

# JEU D'ECHECS QUANTIQUES

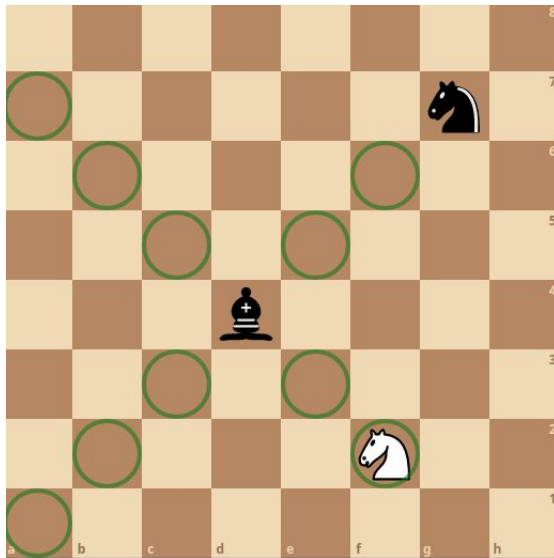
n°xxxxx Quentin Horgues



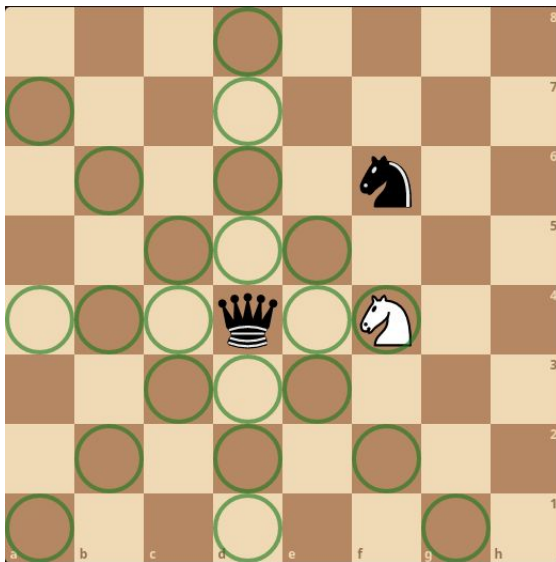
# Présentation des règles du jeu

- 2 joueurs qui joue à tour de rôle
- Un tour = un mouvement
- Objectif : capturer le roi adverse

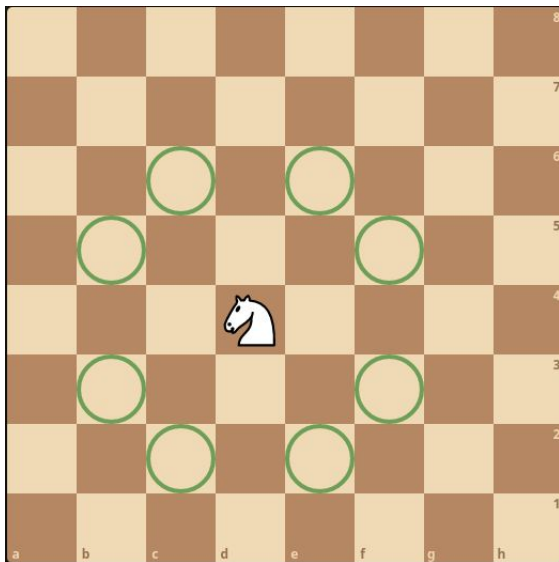
# Mouvement classique : fou



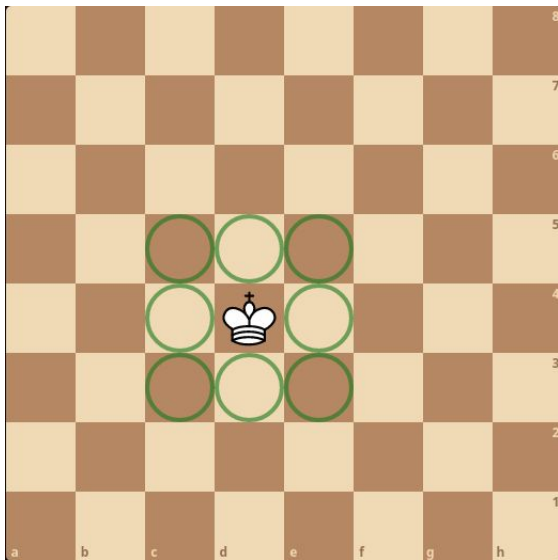
## Mouvement classique : dame



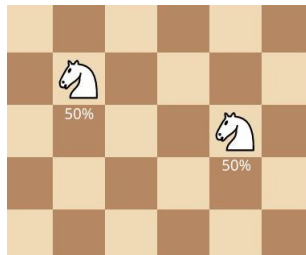
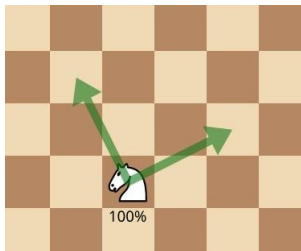
# Mouvement classique : cavalier



# Mouvement classique : roi



# Mouvement split



# Index

- main.cpp
- Board.hpp
- Board.hpp
- Board.hpp
- mesure.hpp
- move.hpp
- get-proba-move.hpp
- Piece.hpp
- Piece.hpp
- get-list-move.hpp
- TypePiece.hpp
- Coord.hpp
- Coord.cpp
- Move.hpp
- Move.cpp



- Qubit.hpp
- Qubit.hpp
- Random.hpp
- Random.cpp
- math-utility.hpp
- math-utility.hpp
- check-path.hpp
- check-path.hpp
- Unitary.hpp
- CMatrix.hpp
- Complex-printer.hpp
- Constexpr.hpp
- ConsoleInterface.hpp
- ConsoleInterface.hpp
- ComputerPlayer.hpp

- ComputerPlayer.tpp
- Matrix.hpp
- Matrix.tpp
- test-move-promotion.cpp
- test-move-enpassant-with-capture.cpp
- test-move-pawn-two-step.cpp
- test-move-promotion-with-capture.cpp
- test-move-promotion-with-split-before.cpp
- test-move-split-with-mesure.cpp
- test-split-in-non-empty-case.cpp
- test-slide-merge-move-through-a-split-piece.cpp
- test-get-proba-move-slide-capture.cpp
- test-get-proba-move-slide-on-same-color.cpp
- test-get-proba-move-slide-without-target.cpp
- test-get-proba-move-jump-same-color.cpp

- test-get-proba-move-jump-capture.cpp
- test-capture-slide-move-true.cpp
- test-capture-slide-move-false.cpp
- test-move-merge-after-split.cpp
- test-split-after-split.cpp
- test-split-after-split2.cpp
- test-multiple-split.cpp
- test-split-takes.cpp

# main.cpp

```
1  #include <iostream>
2  #include <fstream>
3  #include <complex>
4  #include <string>
5  #include <Complex_printer.hpp>
6  #include <CMatrix.hpp>
7  #include <Unitary.hpp>
8  #include <Qubit.hpp>
9  #include <Board.hpp>
10 #include <Piece.hpp>
11 #include <Move.hpp>
12 #include <observer_ptr.hpp>
13 #include <check_path.hpp>
14 #include <Constexpr.hpp>
15 #include <ComputerPlayer.hpp>
16 #include <ConsoleInterface.hpp>
17
18 char piece_to_char(TypePiece piece, Color color = Color::WHITE)
19 {
20     int const offset{(color == Color::BLACK) ? 32 : 0};
21     using enum TypePiece;
22     switch (piece)
23     {
24     case KING:
25         return static_cast<char>('K' + offset);
26     case QUEEN:
27         return static_cast<char>('Q' + offset);
28     case ROOK:
29         return static_cast<char>('R' + offset);
30     case BISHOP:
```

```

31     return static_cast<char>('B' + offset);
32 case KNIGHT:
33     return static_cast<char>('N' + offset);
34 case PAWN:
35     return static_cast<char>('P' + offset);
36 case EMPTY:
37     return ' ';
38 default:
39     return '\0';
40 }
41 }
42
43 template <std::size_t N, std::size_t M>
44 char print_piece(Board<N, M> &board, Coord const &position)
45 {
46     TypePiece piece{board(position.n, position.m).get_type()};
47     Color color{board(position.n, position.m).get_color()};
48     return piece_to_char(piece, color);
49 }
50
51 template <std::size_t N, std::size_t M>
52 void auto_playing(
53     Board<N, M> &board,
54     std::ostream &output,
55     int nb_moves,
56     int search_depth)
57 {
58     std::cout << board << std::endl;
59     while (
60         nb_moves > 0 &&

```

```

61     !board.winning_position(opponent_color(board.get_current_player()))
62 {
63
64     Move m{computer::get_best_move(board, search_depth)};
65
66     std::string data1{{static_cast<char>('a' + m.normal.src.m),
67                       static_cast<char>('0' + N - m.normal.src.n)}};
68
69     std::string data2{{static_cast<char>('a' + m.normal.arv.m),
70                       static_cast<char>('0' + N - m.normal.arv.n)}};
71
72     output << print_piece(board, m.normal.src) << ' ';
73     if (m.type == TypeMove::NORMAL)
74     {
75         output << "N " << data1 << data2 << std::endl;
76     }
77     else if (m.type == TypeMove::PROMOTE)
78     {
79         output << 'P' << piece_to_char(m.promote.piece) << ' '
80             << data1 << data2
81             << std::endl;
82     }
83     else
84     {
85         std::string data3{{static_cast<char>('a' + m.split.arv2.m),
86                           static_cast<char>('0' + N - m.split.arv2.n)}};
87
88         if (m.type == TypeMove::SPLIT)
89         {
90             output << "S " << data1 << '(' << data2 << ':'

```

```

91         << data3 << ')' << std::endl;
92     }
93     else
94     {
95         output << "M " << '(' << data1 << ':' << data2 << ')'
96         << data3 << std::endl;
97     }
98 }
99 board.move(m);
100 /*
101 #if defined(WIN32)
102     int ret = system("cls");
103 #else
104     int ret = system("clear");
105 #endif
106     (void)ret;*/
107     std::cout << board << std::endl;
108     board.change_player();
109     nb_moves--;
110 }
111 }
112
113 int main()
114 {
115
116     Board<> const ChessBoard{
117         {{B_ROOK, B_KNIGHT, B_BISHOP, B_QUEEN, B_KING, B_BISHOP, B_KNIGHT, B_ROOK},
118          {B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN},
119          {Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece()},
120          {Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece()}},

```

```

121     {Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece()},
122     {Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece()},
123     {W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN},
124     {W_ROOK, W_KNIGHT, W_BISHOP, W_QUEEN, W_KING, W_BISHOP, W_KNIGHT, W_ROOK}}};
125
126     /* Board<3> B1{
127         {B_KING, B_ROOK, Piece()},
128         {B_PAWN, Piece(), Piece()},
129         {Piece(), W_QUEEN, W_KING}};
130
131     Board<3> B2{
132         {
133             {B_KING, Piece(), B_QUEEN},
134             {W_PAWN, B_PAWN, W_QUEEN},
135             {Piece(), Piece(), W_KING}
136         }
137     };
138
139     Board<3> B3{
140         {
141             {B_KING, Piece(), Piece()},
142             {Piece(), W_ROOK, Piece()},
143             {Piece(), Piece(), W_KING}
144         }
145     };
146     */
147
148     Board<6, 4> smallBoard{
149         {{B_KNIGHT, B_QUEEN, B_KING, B_BISHOP},
150          {B_PAWN, B_PAWN, B_PAWN, B_PAWN}},

```



```

151     {Piece(), Piece(), Piece(), Piece()},
152     {Piece(), Piece(), Piece(), Piece()},
153     {W_PAWN, W_PAWN, W_PAWN, W_PAWN},
154     {W_KNIGHT, W_QUEEN, W_KING, W_BISHOP}}};
155
156 Board<4> miniBoard{
157     {
158         {B_KING, Piece(), Piece(), B_BISHOP},
159         {Piece(), B_KNIGHT, Piece(), Piece()},
160         {Piece(), Piece(), Piece(), Piece()},
161         {Piece(), Piece(), W_QUEEN, W_KING}
162     }
163 };
164 for (int i = 1; i <= 1; i++)
165 {
166     Board board{ChessBoard};
167     std::ofstream log_file{"partie" + std::to_string(i) + ".txt"};
168     auto_playing(board, log_file, 80, 5);
169     log_file.close();
170 }
171
172 return 0;
173

```

# Board.hpp

```
1  #ifndef GAME_BOARD_HPP
2  #define GAME_BOARD_HPP
3
4  #include <vector>
5  #include <array>
6  #include <utility>
7  #include <optional>
8  #include <complex>
9  #include <cstdint>
10 #include <initializer_list>
11 #include <functional>
12 #include <CMatrix.hpp>
13 #include <Piece.hpp>
14 #include <TypePiece.hpp>
15 #include <Color.hpp>
16 #include <Coord.hpp>
17 #include <forward_list>
18 #include <observer_ptr.hpp>
19 #include <Move.hpp>
20 #include <Constexpr.hpp>
21 #include <check_path.hpp>
22
23 class Piece;
24
25 /**
26  * @brief La classe représentant le plateau de jeu
27  *
28  * @tparam N Le nombre de ligne du plateau
29  * @tparam M Le nombre de colonne du plateau
30  */
```

```

31 template <std::size_t N = 8, std::size_t M = N>
32 class Board final
33 {
34 public:
35     // Constructeur
36     constexpr Board();
37
38     constexpr Board(std::initializer_list<
39                     std::initializer_list<
40                         Piece>> const &board);
41
42     /**
43      * @brief Initialise le plateau à partir d'un ensemble de pièce,
44      * les pièces ne sont pas divisées initialement.
45      *
46      * @param[in] board Double initializer_list sur des Piece
47      */
48     constexpr Board(std::ranges::common_range auto const &board);
49
50     // Copie
51     constexpr Board(Board const &) = default;
52     constexpr Board &operator=(Board const &) = default;
53
54     // Mouvement
55     constexpr Board(Board &&) = default;
56     constexpr Board &operator=(Board &&) = default;
57
58     // Destructeur
59     constexpr ~Board() = default;
60

```

```

61  /**
62   * @brief Retourne un pointeur sur la piece à l'emplacement cible
63   *
64   * @param[in] n L'indice de la ligne
65   * @param[in] m L'indice de la colonne
66   * @return Un pointeur observateur sur une piece
67   * ou Piece() si la case est vide
68   */
69  CONSTEXPR Piece const &
70  operator()(std::size_t n,
71             std::size_t m) const noexcept;
72
73  /**
74   * @brief Retourne le nombre de ligne du plateau
75   *
76   * @return std::size_t Le nombre de ligne
77   */
78  CONSTEXPR static std::size_t numberLines() noexcept;
79
80  /**
81   * @brief Retourne le nombre de colonne
82   *
83   * @return std::size_t Le nombre de colonne
84   */
85  CONSTEXPR static std::size_t numberColumns() noexcept;
86
87  /**
88   * @brief Renvoie la liste dans tous les mouvements
89   * classiques légaux d'une pièce
90   *

```

```

91  * @param pos Position de la pièce
92  *
93  * @return std::forward_list<Coord> La liste de coordonnées des positions
94  * d'arrivées de chaque mouvement possible
95  */
96  CONTEXPR std::forward_list<Coord>
97  get_list_normal_move(Coord const &pos) const;
98
99  /**
100  * @brief Renvoie la liste de toutes les cases qu'une pièce
101  * peut atteindre lors d'un mouvement split, il faut choisir
102  * deux éléments de la liste pour créer un mouvement split.
103  *
104  * @param[in] pos Position de la pièce.
105  *
106  * @return std::forward_list<Coord> La liste de coordonnées
107  * des positions d'arrivées possible sachant que chaque élément
108  * représente une seule des deux cases d'arrivées d'un split move
109  */
110  CONTEXPR std::forward_list<Coord>
111  get_list_split_move(Coord const &pos) const;
112
113  /**
114  * @brief Teste si les mouvement de la piece sont si la
115  * pièce est un pion un mouvement de promotion
116  *
117  * @note peut etre utiliser sans verifier si la pièce est un pion
118  *
119  * @param[in] pos La position de la pièce
120  * @return true si le mouvement possible est un mouvement

```

```

121  * de promotion
122  * @return false sinon
123  */
124  CONSTEXPR bool
125  check_if_use_move_promote(Coord const &pos) const noexcept;
126
127  /**
128   * @brief Renvoie la liste de toutes les promotions
129   *
130   * @warning Ne verifie pas la validité du mouvement,
131   * de la position ou du type de la pièce
132   *
133   * @param[in] pos La position du pion
134   * @return La liste de tout les mouvements de promotion
135   */
136  CONSTEXPR std::forward_list<Move>
137  get_list_promote(Coord const &pos) const noexcept;
138
139  /**
140   * @brief Applique une fonction à l'entièreté des mouvements possible
141   *
142   * @param[in, out] func La fonction à appliquer sur tout les mouvements
143   * Prototype : bool f(Move const&)
144   * Si renvoie true alors le parcourt est interrompue
145   * @param[in] color La couleur du joueur auquel on
146   * récupère les mouvements
147   */
148  template <class UnitaryFunction>
149  CONSTEXPR void
150  all_move(

```

```

151     UnitaryFunction func,
152     std::optional<Color> color_player = std::nullopt) const noexcept;
153
154 /**
155  * @brief Test si un mouvement est réalisable
156  *
157  * @param[in] move Le mouvement à tester
158  * @return true si le mouvement est légal
159  */
160 CONSTEXPR bool move_is_legal(Move const &move) const;
161
162 /**
163  * @brief Réalise un mouvement d'une pièce quelque soit le type
164  * du mouvement (classic, split, merge)
165  *
166  * @warning Ne procède aucune vérification sur la validité du mouvement
167  *
168  * @param[in] mouvement Le mouvement à réaliser
169  * @param[in] val_mes Optionel peut déterminer la mesure de
170  * la présence d'une pièce ou non
171  */
172 CONSTEXPR void
173 move(Move const &movement, std::optional<bool> val_mes = std::nullopt);
174
175 /**
176  * @brief Test si un plateau est gagnant pour une couleur
177  *
178  * @param[in] c Couleur
179  * @return true si la position est gagnante pour la couleur c
180  * @return false sinon

```

```

181  */
182  CONSTEXPR bool winning_position(Color c) const noexcept;
183
184  /**
185   * @brief Recupère la couleur du joueur au tour de jouer
186   *
187   * @return Color la couleur du joueur
188   */
189  CONSTEXPR Color get_current_player() const noexcept;
190
191  /**
192   * @brief Change le joueur actuel et passe la main à l'autre joueur
193   */
194  CONSTEXPR void change_player() noexcept;
195
196  /**
197   * @brief Renvoie la probabilité qu'il y ait une pièce à une position.
198   *
199   * @param pos La position
200   */
201  CONSTEXPR double get_proba(Coord const &pos) const noexcept;
202
203  friend class Piece;
204
205  template <std::size_t _N, std::size_t _M>
206  friend CONSTEXPR bool
207  check_path_straight_1_instance(
208      Board<_N, _M> const &board,
209      Coord const &dpt,
210      Coord const &arv,

```



```

211     std::size_t position,
212     std::optional<Coord>);
213
214     template <std::size_t _N, std::size_t _M>
215     friend CONSTEXPR bool
216     check_path_diagonal_1_instance(
217         Board<_N, _M> const &board,
218         Coord const &dpt,
219         Coord const &arv,
220         std::size_t position,
221         std::optional<Coord>);
222
223     template <std::size_t _N, std::size_t _M>
224     friend CONSTEXPR bool
225     check_path_queen_1_instance(
226         Board<_N, _M> const &board,
227         Coord const &dpt,
228         Coord const &arv,
229         std::size_t position,
230         std::optional<Coord>);
231
232     /**
233     * @brief Fonction qui permet de faire un mouvement de pion quelconque avec une promotion
234     *
235     * @param s Coordonnées de la source
236     * @param t Coordonnées de la cible
237     * @param p Le type de la pièce de la promotion
238     * @param[in] val_mes Optionel peut determiner la mesure de
239     * la présence d'une pièce ou non
240     */

```

```

241 CONSTEXPR void move_promotion(
242     Move const &move,
243     std::optional<bool> val_mes = std::nullopt);
244
245 /**
246  * @brief Renvoie la probabilité que le mouvement soit réalisé
247  *
248  * @param move Le mouvement à réaliser
249  * @return Une probabilité comprise entre 0 et 1
250  */
251 CONSTEXPR double get_proba_move(Move const &move) const noexcept;
252
253 private:
254 /**
255  * @brief Renvoie la probabilité que la fonction mesure renvoie true,
256  * c'est-à-dire la proba que le mouvement soit "réussi"
257  *
258  * @param p Coordonnées de la case
259  * @return La probabilité
260  */
261 CONSTEXPR double
262 get_proba_mesure(Coord const &p) const noexcept;
263
264 /**
265  * @brief Renvoie la probabilité que la fonction mesure_capture_slide renvoie true,
266  * c'est-à-dire la proba que le mouvement de capture_slide soit "réussi"
267  *
268  * @param s Coordonnées de la source
269  * @param t Coordonnées de la cible
270  * @param check_path Fonction qui teste la présence d'une pièce

```

```

271  * entre une source et une cible
272  * @return La probabilité
273  */
274  CONSTEXPR double get_proba_mesure_capture_slide(
275      Coord const &s,
276      Coord const &t,
277      std::function<bool(Board<N, M> const &,
278                          Coord const &,
279                          Coord const &,
280                          std::size_t,
281                          std::optional<Coord>)>
282      check_path) const noexcept;
283
284  /**
285   * @brief Renvoie la probabilité que la fonction mesure_castle renvoie true,
286   * c'est-à-dire la proba que le mouvement de roque soit "réussi"
287   *
288   * @param king Coordonnées du roi
289   * @param rook Coordonnées de la tour
290   * @return La probabilité
291   */
292  CONSTEXPR double get_proba_mesure_castle(
293      Coord const &king,
294      Coord const &rook) const noexcept;
295
296  /**
297   * @brief Vérifie si la case à une possibilité de contenir une pièce,
298   * et si elle n'en a pas modifie m_piece_board en nullptr.
299   *
300   * @param pos Les coordonnées de la position de la case à vérifier.

```

```

301  */
302  void update_case(std::size_t pos) noexcept;
303  /**
304   * @brief Fonction qui permet de mettre à jour le plateau,
305   * car ce mouvement entraîne l'apparition de plusieurs instance de board
306   * identique, il faut donc les concaténer en ajoutant les probas, si la
307   * proba vaut 0, on supprime l'instance
308   *
309   * @warning Complexité en  $k^2 \times N \times M$ , avec  $k$  la taille de  $m\_board$ 
310   * c'est à dire le nombre d'instance du plateau,  $N$  et  $M$  les dimensions
311   * du plateau
312   */
313  void update_board() noexcept;
314  void update_board_classic() noexcept;
315
316  /**
317   * @brief Mouvement classique d'une pièce qui "saute"
318   * (cavalier, roi)
319   *
320   * @warning Ne procède aucune vérification sur la validité du mouvement
321   *
322   * @tparam N Le nombre de ligne du plateau
323   * @tparam M Le nombre de colonnes du plateau
324   * @param[in] s Coordonnées de la source du mouvement
325   * @param[in] t Coordonnées de la cible du mouvement
326   * @param[in] val_mes Optionel peut déterminer la mesure de
327   * la présence d'une pièce ou non
328   */
329  CONSTEXPR void move_classic_jump(
330      Coord const &s,

```

```

331     Coord const &t,
332     std::optional<bool> val_mes = std::nullopt);
333
334 /**
335  * @brief Mouvement "split jump"
336  *
337  * @warning Aucun tests sur la validité du mouvement (cible vide, ect)
338  *
339  * @tparam N Le nombre de lignes du plateau
340  * @tparam M Le nombre de colonnes du plateau
341  * @param[in] s Coordonnées de la source du mouvement
342  * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
343  * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
344  */
345 CONSTEXPR void move_split_jump(
346     Coord const &s,
347     Coord const &t1,
348     Coord const &t2);
349
350 /**
351  * @brief Mouvement de merge
352  *
353  * @warning Aucun test sur la validité du mouvement
354  * (cible vide, pièce identique sur les sources, ect)
355  *
356  * @tparam N Le nombre de lignes du plateau
357  * @tparam M Le nombre de colonnes du plateau
358  * @param[in] s1 Coordonnées de la source 1
359  * @param[in] s2 Coordonnées de la source 2
360  * @param[in] t Coordonnées de la cible

```

```

361  */
362  CONSTEXPR void move_merge_jump(
363      Coord const &s,
364      Coord const &t1,
365      Coord const &t2);
366  /**
367   * @brief Mouvement classique d'un pièce qui "glisse" (fou, dame, tour)
368   *
369   * @warning Ne procède aucune vérification sur la validité du mouvement
370   *
371   * @tparam N Le nombre de lignes
372   * @tparam M Le nombre de colonnes
373   * @param[in] s Coordonnées de la source
374   * @param[in] t Coordonnées de la cible
375   * @param[in] check_path Fonction qui permet de vérifier la présence
376   * d'une pièce entre la source et la cible sur une instance du plateau
377   * @param[in] val_mes Optionel peut déterminer la mesure de
378   * la présence d'une pièce ou non
379   */
380  CONSTEXPR void
381  move_classic_slide(Coord const &s, Coord const &t,
382                    std::function<
383                        bool(
384                            Board<N, M> const &,
385                            Coord const &,
386                            Coord const &,
387                            std::size_t,
388                            std::optional<Coord>)>
389                        check_path,
390                    std::optional<bool> val_mes = std::nullopt);

```

```

391 /**
392  * @brief Mouvement "split slide"
393  * @warning Aucun tests sur la validité du mouvement (cible vide, ect)
394  * @tparam N Le nombre de lignes du plateau
395  * @tparam M Le nombre de colonnes du plateau
396  * @param[in] s Coordonnées de la source du mouvement
397  * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
398  * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
399  * @param[in] check_path Fonction qui permet de vérifier la présence d'une
400  * pièce entre la source et la cible sur une instance du plateau
401  */
402 CONSTEXPR void move_split_slide(
403     Coord const &s,
404     Coord const &t1,
405     Coord const &t2,
406     std::function<
407         bool(
408             Board<N, M> const &,
409             Coord const &,
410             Coord const &,
411             std::size_t,
412             std::optional<Coord>>>
413     check_path);
414
415 /**
416  * @brief Mouvement de merge
417  * @warning Aucun test sur la validité du mouvement
418  * (cible vide, pièce identique sur les sources, ect)
419  * @tparam N Le nombre de lignes du plateau
420  * @tparam M Le nombre de colonnes du plateau

```

```

421 * @param[in] s1 Coordonnées de la source 1
422 * @param[in] s2 Coordonnées de la source 2
423 * @param[in] t Coordonnées de la cible
424 * @param[in] check_path Fonction qui permet de vérifier la présence d'une
425 * pièce entre la source et la cible sur une instance du plateau
426 */
427 CONSTEXPR void move_merge_slide(
428     Coord const &s1,
429     Coord const &s2,
430     Coord const &t,
431     std::function<
432         bool(
433             Board<N, M> const &,
434             Coord const &,
435             Coord const &,
436             std::size_t,
437             std::optional<Coord>>>
438     check_path);
439
440 /**
441  * @brief Le mouvement de pion d'une case fonctionne comme
442  * un mouvement de jump classique, à la différence qu'un
443  * pion ne peut pas capturer une pièce en avançant.
444  * On mesure de la même façon qu'un jump classique en considérant
445  * toutes les pièces comme des pièces alliées
446  *
447  * @warning Ne procède aucune vérification sur la validité du mouvement
448  *
449  * @tparam N Le nombre de lignes du plateau
450  * @tparam M Le nombre de colonnes du plateau

```



```

451  * @param[in] s Coordonnées de la source
452  * @param[in] t Coordonnées de la cible
453  * @param[in] val_mes Optionel peut determiner la mesure de
454  * la présence d'une pièce ou non
455  * @return true si le mouvement à etait effectué
456  * @return false sinon
457  */
458  CONTEXPR bool
459  move_pawn_one_step(Coord const &s, Coord const &t,
460                    std::optional<bool> val_mes = std::nullopt);
461
462  /**
463   * @brief Le mouvement de pion de deux cases fonctionne
464   * comme un mouvement de slide classique, à la différence
465   * qu'un pion ne peut pas capturer une pièce en avançant,
466   * on mesure de la même façon qu'un slide classique en considérant
467   * toutes les pièces comme des pièces alliées.
468   *
469   * @warning Aucun test sur la possibilité de faire un mouvement de deux
470   * cases du pion, pas de mise a jour du board sur la prise en passant
471   *
472   * @tparam N Le nombre de lignes du plateau
473   * @tparam M Le nombre de colonnes du plateau
474   * @param[in] s Coordonnées de la source
475   * @param[in] t Coordonnées de la cible
476   * @param[in] val_mes Optionel peut determiner la mesure de
477   * la présence d'une pièce ou non
478   * @return true si le mouvement à etait effectué
479   * @return false sinon
480   */

```

```

481 CONSTEXPR bool
482 move_pawn_two_step(Coord const &s, Coord const &t,
483                   std::optional<bool> val_mes = std::nullopt);
484 /**
485  * @brief Mouvement de capture dun pion. A la différence
486  * d'un mouvement de "capture jump", on mesure la présence
487  * de la cible car le pion a besoin de la cible pour se déplacer
488  *
489  * @warning Ne procède aucune vérification sur la validité du mouvement
490  *
491  * @param[in] s Coordonnées de la source
492  * @param[in] t Coordonnées de la cible
493  * @param[in] val_mes Optionel peut determiner la mesure de
494  * la présence d'une pièce ou non
495  * @return true si le mouvement à etait effectué
496  * @return false sinon
497  */
498 CONSTEXPR bool capture_pawn(Coord const &s,
499                             Coord const &t,
500                             std::optional<bool> val_mes = std::nullopt);
501
502 /**
503  * @brief Permet d'effectuer un mouvement de prise en passant
504  *
505  * @warning Aucun test sur la validité de la cible et de la cible
506  * en passant, ni sur le type de la pièce
507  *
508  * @tparam N Le nombre de lignes du plateau
509  * @tparam M Le nombre de colonnes du plateau
510  * @param[in] s Les coordonnées de la source

```

```

511 * @param[in] t Les coordonnées de la cible (l'endroit où arrive le pion)
512 * @param[in] ep Les coordonnées du pion capturer "en passant"
513 * @param[in] val_mes Optional peut determiner la mesure de
514 * la présence d'une pièce ou non
515 * @return true si le mouvement à etait effectué
516 * @return false sinon
517 */
518 CONSTEXPR bool
519 move_enpassant(Coord const &s, Coord const &t, Coord const &ep,
520               std::optional<bool> val_mes = std::nullopt);
521
522 /**
523  * @brief Mesure la présence d'une pièce
524  *
525  * @tparam N Le nombre de lignes du plateau
526  * @tparam M Le nombre de colonnes du plateau
527  * @param[in] position La position de la pièce mesurée
528  * @param[in] val_mes Optional peut determiner la mesure de
529  * la présence d'une pièce ou non
530  * @return true Si la pièce est présente sur la case position
531  * @return false Sinon
532  */
533 CONSTEXPR bool mesure(Coord const &p,
534                       std::optional<bool> val_mes = std::nullopt);
535
536 /**
537  * @brief Fonction qui permet de faire une mesure dans le cas
538  * d'un mouvement "capture slide"
539  *
540  * @tparam N Le nombre de lignes du plateau

```

```

541 * @tparam M Le nombre de colonnes du plateau
542 * @param[in] s Coordonnées de la source du mouvement
543 * @param[in] t Coordonnées de la cible du mouvement
544 * @param[in] check_path Fonction qui permet de vérifier si il y a une pièce
545 * entre la source et la cible sur une instance du plateau
546 * @param[in] val_mes Optionel peut determiner la mesure de
547 * la présence d'une pièce ou non
548 * @return true Si la mesure indique de faire le mouvement
549 * @return false Sinon
550 */
551 CONSTEXPR bool
552 mesure_capture_slide(Coord const &s, Coord const &t,
553                     std::function<bool(Board<N, M> const &,
554                                     Coord const &, Coord const &,
555                                     std::size_t,
556                                     std::optional<Coord>)>
557                     check_path,
558                     std::optional<bool> val_mes = std::nullopt);
559
560 /**
561  * @brief Mouvement classique d'une pièce (pion exclu)
562  * @warning Ne procède aucune vérification sur la validité du mouvement
563  * @tparam N Le nombre de ligne du plateau
564  * @tparam M Le nombre de colonnes du plateau
565  * @param[in] s Coordonnées de la source du mouvement
566  * @param[in] t Coordonnées de la cible du mouvement
567  */
568 CONSTEXPR void move_classic(Coord const &s, Coord const &t,
569                             std::optional<bool> val_mes);
570

```

```

571 /**
572  * @brief Mouvement split d'une pièce (pion exclu)
573  * @warning Ne procède aucune vérification sur la validité du mouvement
574  * @tparam N Le nombre de ligne du plateau
575  * @tparam M Le nombre de colonnes du plateau
576  * @param[in] s Coordonnées de la source du mouvement
577  * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
578  * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
579  */
580 CONSTEXPR void move_split(Coord const &s,
581                          Coord const &t1,
582                          Coord const &t2);
583
584 /**
585  * @brief Mouvement de merge de pièce (pion exclu)
586  *
587  * @warning Aucun test sur la validité du mouvement
588  * (cible vide, pièce identique sur les sources, ect)
589  *
590  * @tparam N Le nombre de lignes du plateau
591  * @tparam M Le nombre de colonnes du plateau
592  * @param[in] s1 Coordonnées de la source 1
593  * @param[in] s2 Coordonnées de la source 2
594  * @param[in] t Coordonnées de la cible
595  */
596 CONSTEXPR void move_merge(Coord const &s1,
597                          Coord const &s2,
598                          Coord const &t);
599
600 /**

```

```

601  * @brief Gère tout les mouvements d'un pion
602  * @warning Ne procède aucune vérification sur la validité du mouvement
603  * @tparam N Le nombre de lignes du plateau
604  * @tparam M Le nombre de colonnes du plateau
605  * @param[in] s Coordonnées de la source
606  * @param[in] t Coordonnées de la cible
607  * @param[in] val_mes Optionnel peut determiner la mesure de
608  * la présence d'une pièce ou non
609  * @return true si le mouvement à etait effectué
610  * @return false sinon
611  */
612  CONSTEXPR bool move_pawn(
613      Coord const &s,
614      Coord const &t,
615      std::optional<bool> val_mes = std::nullopt) noexcept;
616
617  /**
618   * @brief Renvoie une position 1D d'une coordonnée 2D
619   *
620   * @param[in] ligne L'indice de ligne
621   * @param[in] colonne L'indice de colonne
622   * @return std::size_t La position en 1D
623   */
624  CONSTEXPR static std::size_t
625  offset(std::size_t ligne,
626         std::size_t colonne) noexcept;
627
628  /**
629   * @brief Initialise un plateau à l'aide d'un container 2D
630   *

```

```

631 * @param[in] board Un container 2D
632 * @param[out] first_instance Le tableau de la première
633 * instance du plateau
634 * @param[out] piece_board Le plateau contenant les pièces
635 */
636 CONSTEXPR static void
637 range_to_2_array(
638     std::ranges::common_range auto const &board,
639     std::array<bool, N * M> &first_instance,
640     std::array<Piece, N * M> &piece_board) noexcept;
641
642 /**
643  * @brief Construit les mailbox en fonction
644  * des dimensions du plateau
645  *
646  * @param[out] S_mailbox La petite mailbox de la taille du plateau
647  * @param[out] L_mailbox La grande mailbox comportant 4 ligne de plus
648  * et 2 colonne de plus
649  * @return CONSTEXPR
650  */
651 CONSTEXPR static void init_mailbox(std::array<int, N * M>
652                                     &S_mailbox,
653                                     std::array<int, (N + 4) * (M + 2)>
654                                     &L_mailbox) noexcept;
655
656 /**
657  * @brief fonction auxiliaire qui permet de modifier un plateau
658  * à l'aide d'un array de la forme du type de retour de la fonction
659  * qubitToArray, le deuxième éléments du tableau à une probabilité non nulle
660  * lors d'un mouvement split

```

```

661  *
662  * @tparam Q La taille du qubit
663  * @tparam N Le nombre de ligne du plateau
664  * @tparam M Le nombre de collone du plateau
665  * @param[in] arrayQubit Type de retour de la fonction quibitToArray,
666  * représente la facon dont va être modifier m_board
667  * @param[in] position_board L'indice du plateau dans le tableau de
668  * toutes les instances du plateau
669  * @param[in] tab_positions Tableau des indices des cases modifiées,
670  * on utilise N*M+1 pour signifier que le qubit en question
671  * est un qubit auxiliaire qui ne modifie pas le board
672  */
673  template <std::size_t Q>
674  constexpr void modify(std::array<std::pair<std::array<bool, Q>,
675                                std::complex<double>>,
676                                2> const &arrayQubit,
677                                std::size_t position_board,
678                                std::array<std::size_t, Q> const &tab_positions);
679
680  /**
681   * @brief Fonction qui permet d'effectuer un mouvement
682   * sur une instance du plateau
683   *
684   * @tparam N Le nombre de ligne du plateau
685   * @tparam M Le nombre de colonnes du plateau
686   * @tparam Q La taille du qubit
687   * @param[in] case_modif Permet d'initialiser le qubit
688   * @param[in] position L'instance du plateau modifiée
689   * @param[in] matrix La matrice du mouvement que l'on veut effectuer
690   * @param[in] tab_positions La position sur le plateau des variables

```



```

691  * utilisées dans le qubit, que l'on met à N*M+1 si les variables
692  * ne représentent pas des pièces
693  */
694  template <std::size_t Q>
695  CONSTEXPR void move_1_instance(std::array<bool, Q> const &case_modif,
696                                std::size_t position,
697                                CMatrix<_2POW(Q)> const &matrix,
698                                std::array<std::size_t, Q> const
699                                &tab_positions) noexcept;
700
701  /**
702   * @brief Mouvement du petit roque.
703   * @warning Aucun test sur la validité du mouvement
704   * @param[in] king Coordonnées du roi
705   * @param[in] rook Coordonnées de la tour
706   * @param[in] val_mes Optionel peut determiner la mesure de
707   * la présence d'une pièce ou non
708   */
709  CONSTEXPR void king_side_castle(
710      Coord const &king,
711      Coord const &rook,
712      std::optional<bool> val_mes = std::nullopt);
713
714  /**
715   * @brief Fonction qui permet de mesurer la possibilité de faire le roque
716   *
717   * @param[in] king Coordonnées du roi
718   * @param[in] rook Coordonnées de la tour
719   * @param[in] val_mes Optionel peut determiner la mesure de
720   * la présence d'une pièce ou non

```

```

721  * @return true si le roque est possible
722  * @return false sinon
723  */
724  CONSTEXPR bool mesure_castle(
725      Coord const &king,
726      Coord const &rook,
727      std::optional<bool> val_mes = std::nullopt);
728
729  /**
730   * @brief Mouvement du grand roque.
731   *
732   * @warning Ne procède aucune vérification sur la validité du mouvement
733   *
734   * @param[in] king Coordonnées du roi
735   * @param[in] rook Coordonnées de la tour
736   * @param[in] val_mes Optionel peut déterminer la mesure de
737   * la présence d'une pièce ou non
738   */
739  CONSTEXPR void queen_side_castle(
740      Coord const &king,
741      Coord const &rook,
742      std::optional<bool> val_mes = std::nullopt);
743
744  /**
745   * @brief Le tableau de toutes les instances possibles
746   * du plateau
747   */
748  std::vector<std::pair<std::array<bool, N * M>,
749                  std::complex<double>>>
750      m_board;

```

```

751
752 /**
753  * @brief Le plateau indiquant le type des pièces
754  */
755 std::array<Piece, N * M> m_piece_board;
756
757 /**
758  * @brief La petite mailbox
759  */
760 std::array<int, N * M> m_S_mailbox;
761
762 /**
763  * @brief La grande mailbox
764  */
765 std::array<int, (N + 4) * (M + 2)> m_L_mailbox;
766
767 /**
768  * @brief Color::WHITE si c'est aux blanc de jouer aux noirs sinon
769  */
770 Color m_color_current_player;
771
772 /**
773  * @brief Vrai si les noirs[0] / blancs[1] peuvent faire le petit roque
774  */
775 std::array<bool, 2> m_k_castle;
776
777 /**
778  * @brief Vrai si les noirs[0] / blancs[1] peuvent faire le grand roque
779  */
780 std::array<bool, 2> m_q_castle;

```

```
781
782  /**
783   * @brief Contient la case sur laquelle il est
784   * possible de faire une prise en passant qui
785   * est vide (la case occupé si le pion avait avancé
786   * d'une seule case)
787   */
788   std::optional<Coord> m_ep;
789 };
790
791 #include "Board.tpp"
792 #include "mesure.tpp"
793 #include "get_proba_move.tpp"
794 #include "move.tpp"
795
796
```

# Board.hpp

```
1 // Lib standard
2 #include <array>
3 #include <initializer_list>
4 #include <algorithm>
5 #include <cassert>
6 #include <utility>
7 #include <functional>
8 #include <cmath>
9 #include <optional>
10 #include <stdexcept>
11
12 // Inclusion projet
13 #include <Qubit.hpp>
14 #include <CMatrix.hpp>
15 #include <Unitary.hpp>
16 #include <TypePiece.hpp>
17 #include <math_utility.hpp>
18 #include <Constexpr.hpp>
19 #include <Move.hpp>
20 #include <Random.hpp>
21 #include "Board.hpp"
22
23 template <std::size_t N, std::size_t M>
24 constexpr Board<N, M>::Board()
25     : m_board(),
26       m_piece_board(),
27       m_S_mailbox(),
28       m_L_mailbox(),
29       m_color_current_player(Color::WHITE),
30       m_k_castle({true, true}),
```

```

31     m_q_castle({true, true}),
32     m_ep()
33 {
34     m_board.push_back(std::pair<std::array<bool, 64>,
35                             std::complex<double>>{{}, 0});
36     init_mailbox(m_S_mailbox, m_L_mailbox);
37 };
38
39 template <std::size_t N, std::size_t M>
40 constexpr Board<N, M>::Board(std::initializer_list<
41                             std::initializer_list<
42                                 Piece>> const &board)
43 : m_board(),
44   m_piece_board(),
45   m_S_mailbox(),
46   m_L_mailbox(),
47   m_color_current_player(Color::WHITE),
48   m_k_castle({true, true}),
49   m_q_castle({true, true}),
50   m_ep()
51 {
52     init_mailbox(m_S_mailbox, m_L_mailbox);
53     m_board.push_back(std::pair<std::array<bool, N * M>,
54                             std::complex<double>>{{false}, 1.});
55     range_to_2_array(board, m_board[0].first, m_piece_board);
56 };
57
58 template <std::size_t N, std::size_t M>
59 constexpr Board<N, M>::Board(std::ranges::common_range auto const &board)
60 : m_board(),

```

```

61     m_piece_board(),
62     m_S_mailbox(),
63     m_L_mailbox(),
64     m_color_current_player(Color::WHITE),
65     m_k_castle({true, true}),
66     m_q_castle({true, true}),
67     m_ep()
68 {
69     init_mailbox(m_S_mailbox, m_L_mailbox);
70     m_board.push_back(std::pair<std::array<bool, N * M>,
71                             std::complex<double>>{{false}, 1.});
72     range_to_2_array(board, m_board[0].first, m_piece_board);
73 };
74
75 template <std::size_t N, std::size_t M>
76 constexpr std::size_t
77 Board<N, M>::offset(std::size_t ligne, std::size_t colonne) noexcept
78 {
79     return ligne * M + colonne;
80 };
81
82 template <std::size_t N, std::size_t M>
83 constexpr std::size_t Board<N, M>::numberLines() noexcept
84 {
85     return N;
86 };
87
88 template <std::size_t N, std::size_t M>
89 constexpr std::size_t Board<N, M>::numberColumns() noexcept
90 {

```

```

91     return M;
92 };
93
94 template <std::size_t N, std::size_t M>
95 CONSTEXPR void
96 Board<N, M>::range_to_2_array(
97     std::ranges::common_range auto const &board,
98     std::array<bool, N * M> &first_instance,
99     std::array<Piece, N * M> &piece_board) noexcept
100 {
101
102     auto it_tab{std::begin(first_instance)};
103     auto it_piece_dst{std::begin(piece_board)};
104     assert(std::size(board) <= N && "Value entry out of Board");
105     for (auto const &e : board)
106     {
107         auto const it_tab_begin_line{it_tab};
108         assert(std::size(e) <= M && "Value entry out of Board");
109         std::for_each(std::begin(e),
110                     std::end(e),
111                     [it_tab](Piece piece) mutable
112                     -> void
113                     {
114                         if (piece.get_type() != TypePiece::EMPTY)
115                         {
116                             *it_tab = true;
117                         }
118                         else
119                         {
120                             *it_tab = false;

```



```

121         }
122         it_tab++; });
123     std::move(std::begin(e), std::end(e), it_piece_dst);
124     it_tab = it_tab_begin_line + M;
125     it_piece_dst += M;
126 }
127 }
128
129 template <std::size_t N, std::size_t M>
130 constexpr void
131 Board<N, M>::init_mailbox(
132     std::array<int, N * M> &S_mailbox,
133     std::array<int, (N + 4) * (M + 2)> &L_mailbox) noexcept
134 {
135     std::fill(std::begin(L_mailbox),
136         std::begin(L_mailbox) + (2 * (M + 2) + 1), -1);
137
138     std::fill(std::end(L_mailbox) - (2 * (M + 2) + 1),
139         std::end(L_mailbox), -1);
140     auto offset_L_mailboard{
141         [](std::size_t n, std::size_t m)
142             -> std::size_t
143         {
144             return n * (M + 2) + m;
145         }};
146     for (std::size_t i{0}; i < N; i++)
147     {
148         L_mailbox[offset_L_mailboard(i + 2, 0)] = -1;
149         L_mailbox[offset_L_mailboard(i + 2, M + 1)] = -1;
150         for (std::size_t j{0}; j < M; j++)

```

```

151     {
152         L_mailbox[offset_L_mailboard(i + 2, j + 1)] =
153             static_cast<int>(Board<N, M>::offset(i, j));
154         S_mailbox[Board<N, M>::offset(i, j)] =
155             static_cast<int>(offset_L_mailboard(i + 2, j + 1));
156     }
157 }
158 }
159
160 template <std::size_t N, std::size_t M>
161 CONSTEXPR double Board<N, M>::get_proba(Coord const &pos) const noexcept
162 {
163     double acc{0.};
164     for (auto const &e : m_board)
165     {
166         acc += pow(abs(e.second), 2.) * e.first[offset(pos.n, pos.m)];
167     }
168     return acc;
169 }
170
171 template <std::size_t N, std::size_t M>
172 CONSTEXPR std::forward_list<Coord>
173 Board<N, M>::get_list_normal_move(Coord const &pos) const
174 {
175     if ((*this)(pos.n, pos.m).get_type() != TypePiece::EMPTY)
176     {
177         return (*this)(pos.n, pos.m).get_list_normal_move(*this, pos);
178     }
179     return std::forward_list<Coord>{};
180 }

```

```

181
182 template <std::size_t N, std::size_t M>
183 CONSTEXPR std::forward_list<Coord>
184 Board<N, M>::get_list_split_move(Coord const &pos) const
185 {
186     if ((*this)(pos.n, pos.m).get_type() != TypePiece::EMPTY)
187     {
188         return (*this)(pos.n, pos.m).get_list_split_move(*this, pos);
189     }
190     return std::forward_list<Coord>{};
191 }
192
193 template <std::size_t N, std::size_t M>
194 CONSTEXPR Color Board<N, M>::get_current_player() const noexcept
195 {
196     return m_color_current_player;
197 }
198
199 template <std::size_t N, std::size_t M>
200 CONSTEXPR void Board<N, M>::change_player() noexcept
201 {
202     if (get_current_player() == Color::WHITE)
203     {
204         m_color_current_player = Color::BLACK;
205     }
206     else
207     {
208         m_color_current_player = Color::WHITE;
209     }
210 }

```

```

211
212 template <std::size_t N, std::size_t M>
213 template <std::size_t Q>
214 constexpr void Board<N, M>::modify(
215     std::array<
216         std::pair<
217             std::array<bool, Q>,
218             std::complex<double>>>,
219         2> const &arrayQubit,
220     std::size_t position_board,
221     std::array<std::size_t, Q> const &tab_positions)
222 {
223     if (!complex_equal(arrayQubit[1].second, 0i))
224     {
225         std::pair<std::array<bool, N * M>, std::complex<double>> new_b{};
226         std::copy(std::begin(m_board[position_board].first),
227             std::end(m_board[position_board].first),
228             std::begin(new_b.first));
229         new_b.second = m_board[position_board].second;
230         for (std::size_t i{0}; i < Q; i++)
231         {
232             if (tab_positions[i] < N * M + 1)
233             {
234                 /*Ca nous permet de mettre des variables
235                 dans les qubits qui ne sont pas prises
236                 en compte lors de la modif du plateau */
237                 new_b.first[tab_positions[i]] = arrayQubit[1].first[i];
238             }
239         }
240         new_b.second *= arrayQubit[1].second;

```

```

241     m_board.push_back(new_b);
242 }
243 for (std::size_t i{0}; i < Q; i++)
244 {
245     if (tab_positions[i] < N * M + 1)
246     {
247         m_board[position_board].first[tab_positions[i]] =
248             arrayQubit[0].first[i];
249     }
250 }
251 m_board[position_board].second *= arrayQubit[0].second;
252 }
253
254 template <std::size_t N, std::size_t M>
255 template <std::size_t Q>
256 constexpr void
257 Board<N, M>::move_1_instance(std::array<bool, Q> const &case_modif,
258                             std::size_t position,
259                             CMatrix<_2POW(Q)> const &matrix,
260                             std::array<std::size_t, Q> const
261                                 &tab_positions) noexcept
262 {
263     Qubit<Q> q{case_modif};
264     auto x{qubitToArray(matrix * q)};
265     modify(std::move(x), position, tab_positions);
266 }
267
268 template <std::size_t N, std::size_t M>
269 constexpr Piece const &
270 Board<N, M>::operator()(std::size_t n, std::size_t m) const noexcept

```

```

271 {
272     return m_piece_board[offset(n, m)];
273 }
274
275 template <std::size_t N, std::size_t M>
276 void Board<N, M>::update_board() noexcept
277 {
278     std::size_t size_board = std::size(m_board);
279     for (std::size_t i{size_board}; i > 0; i--)
280     {
281         using namespace std::complex_literals;
282         if (complex_equal(m_board[i - 1].second, 0i))
283         {
284             m_board.erase(std::begin(m_board) + i - 1);
285         }
286         else
287         {
288             for (std::size_t j{i - 1}; j > 0; j--)
289             {
290                 if (m_board[i - 1].first == m_board[j - 1].first)
291                 {
292                     m_board[j - 1].second += m_board[i - 1].second;
293                     m_board.erase(std::begin(m_board) + i - 1);
294                     break;
295                 }
296             }
297         }
298     }
299 }
300 template <std::size_t N, std::size_t M>

```

```

301 CONSTEXPR bool Board<N, M>::winning_position(Color c) const noexcept
302 {
303     for (std::size_t i{0}; i < N * M; i++)
304     {
305         if (m_piece_board[i].get_type() == TypePiece::KING &&
306             m_piece_board[i].get_color() != c)
307         {
308             return false;
309         }
310     }
311     return true;
312 }
313
314 template <std::size_t N, std::size_t M>
315 CONSTEXPR bool
316 Board<N, M>::move_is_legal(Move const &move) const
317 {
318     if (move.type == TypeMove::NORMAL)
319     {
320         auto list{get_list_normal_move(move.normal.src)};
321         return std::find(
322             std::begin(list),
323             std::end(list),
324             move.normal.arv) !=
325             std::end(list);
326     }
327     else if (move.type == TypeMove::SPLIT)
328     {
329         std::forward_list<Coord> list{get_list_split_move(move.split.src)};
330         std::array<bool, 2> found{};

```

```

331     std::array<Coord, 2> arv{{move.split.arv1, move.split.arv2}};
332     for (Coord const &e : list)
333     {
334         for (std::size_t i{0}; i < std::size(arv); i++)
335         {
336             if (e == arv[i])
337             {
338                 found[i] = true;
339             }
340         }
341         if (std::all_of(
342             std::begin(found),
343             std::end(found),
344             [](bool v) -> bool
345             { return v; }))
346         {
347             break;
348         }
349     }
350 }
351 else if (move.type == TypeMove::MERGE)
352 {
353     std::forward_list<Coord> list1{
354         get_list_split_move(move.merge.src1)};
355     std::forward_list<Coord> list2{
356         get_list_split_move(move.merge.src2)};
357
358     return std::find(
359         std::begin(list1),
360         std::end(list1),

```



```

361         move.merge.arv) !=
362         std::end(list1) &&
363     std::find(
364         std::begin(list2),
365         std::end(list2),
366         move.merge.arv) !=
367         std::end(list2);
368 }
369 }
370
371 template <std::size_t N, std::size_t M>
372 void Board<N, M>::update_case(std::size_t pos) noexcept
373 {
374     std::size_t size_board{std::size(m_board)};
375     for (std::size_t i{0}; i < size_board; i++)
376     {
377         if (m_board[i].first[pos])
378         {
379             return;
380         }
381     }
382     m_piece_board[pos] = Piece();
383 }
384
385 template <std::size_t N, std::size_t M>
386 constexpr bool
387 Board<N, M>::check_if_use_move_promote(Coord const &pos) const noexcept
388 {
389     return (*this)(pos.n, pos.m).check_if_use_move_promote(*this, pos);
390 }

```

```

391
392 template <std::size_t N, std::size_t M>
393 CONSTEXPR std::forward_list<Move>
394 Board<N, M>::get_list_promote(Coord const &pos) const noexcept
395 {
396     return (*this)(pos.n, pos.m).get_list_promote(*this, pos);
397 }
398
399 template <std::size_t N, std::size_t M>
400 template <class UnitaryFunction>
401 CONSTEXPR void
402 Board<N, M>::all_move(
403     UnitaryFunction func,
404     std::optional<Color> color_player) const noexcept
405 {
406     std::unordered_map<TypePiece,
407         std::unordered_map<
408             Coord,
409             std::vector<Coord>,
410             Coord_hash>>
411         move_merge{};
412
413     for (std::size_t i{0}; i < numberLines(); i++)
414     {
415         for (std::size_t j{0}; j < numberColumns(); j++)
416         {
417             Piece const &piece{(*this)(i, j)};
418             double p_proba{get_proba(Coord(i, j))};
419             if (piece.get_type() != TypePiece::EMPTY &&
420                 piece.get_color() == color_player

```

```

421                                     .value_or(get_current_player()))
422 {
423     if (!check_if_use_move_promote(Coord(i, j)))
424     {
425         std::forward_list<Coord> move_normal{
426             get_list_normal_move(Coord(i, j))};
427
428         for (Coord const &c : move_normal)
429         {
430             if (func(Move_classic(Coord(i, j), c)))
431             {
432                 return;
433             }
434         }
435
436         std::forward_list<Coord> move_split{
437             get_list_split_move(Coord(i, j))};
438
439         std::forward_list<Coord>::
440             const_iterator it_move{
441                 std::cbegin(move_split)};
442         for (; it_move != std::cend(move_split);
443             it_move++)
444         {
445             for (std::forward_list<Coord>::
446                 const_iterator it2{
447                     std::cbegin(move_split)};
448                 it2 != cend(move_split); it2++)
449             {
450                 if (it_move == it2)

```

```

451     {
452         continue;
453     }
454     if (func(Move_split(Coord(i, j), *it_move, *it2)))
455     {
456         return;
457     }
458 }
459 if (piece.get_type() != TypePiece::PAWN)
460 {
461     if (move_merge
462         .contains(piece.get_type()))
463     {
464         if (move_merge
465             .at(piece.get_type())
466             .contains(*it_move))
467         {
468             for (Coord const &c :
469                 move_merge
470                     .at(piece.get_type())
471                     .at(*it_move))
472             {
473                 if (p_proba < 1. ||
474                     get_proba(
475                         Coord(
476                             c.n,
477                             c.m)) < 1.)
478                 {
479                     if (func(Move_merge(
480                         Coord(i, j),

```

```

481             c,
482             *it_move)))
483         {
484             return;
485         }
486     }
487 }
488 }
489 else
490 {
491     move_merge
492         .at(piece.get_type())
493         .insert({*it_move,
494                 std::vector{
495                     Coord(i, j)}});
496 }
497 }
498 else
499 {
500     move_merge
501         .insert(
502             std::make_pair(
503                 piece
504                     .get_type(),
505                 std::unordered_map<
506                     Coord,
507                     std::vector<Coord>,
508                     Coord_hash>{
509                         std::make_pair(
510                             *it_move,

```

```

511         std::vector{
512             Coord(i, j)}}}));
513     }
514 }
515 }
516 }
517 else
518 {
519     std::forward_list<Move> list{
520         get_list_promote(Coord(i, j))};
521
522     for (Move const &move : list)
523     {
524         if (func(move))
525         {
526             return;
527         }
528     }
529 }
530 }
531 }
532 }
533 }
534
535 template <std::size_t N, std::size_t M>
536 void Board<N, M>::update_board_classic() noexcept
537 {
538     std::size_t size_board = std::size(m_board);
539     for (std::size_t i{size_board}; i > 0; i--)
540     {

```

```

541     using namespace std::complex_literals;
542     if (complex_equal(m_board[i - 1].second, 0i))
543     {
544         m_board.erase(std::begin(m_board) + i - 1);
545     }
546     else
547     {
548         for (std::size_t j{i - 1}; j > 0; j--)
549         {
550             if (m_board[i - 1].first == m_board[j - 1].first)
551             {
552                 double inter = std::pow(std::abs(m_board[j - 1].second), 2) +
553                     std::pow(std::abs(m_board[i - 1].second), 2);
554                 m_board[j - 1].second = std::pow(inter, 1. / 2);
555                 m_board.erase(std::begin(m_board) + i - 1);
556                 break;
557             }
558         }
559     }
560 }
561 /*std::size_t n_size_board = std::size(m_board);
562 double all_proba {0};
563 for (std::size_t i{0}; i<n_size_board; i++)
564 {
565     all_proba += std::pow(std::abs(m_board[i].second), 2);
566 }
567 all_proba = std::pow(all_proba, 1./2);
568 for (std::size_t i{0}; i<n_size_board; i++)
569 {
570     m_board[i].second /= all_proba;

```

571 }\*/  
572



# measure.hpp

```
1 // Lib standard
2 #include <cassert>
3 #include <utility>
4 #include <functional>
5 #include <cmath>
6 #include <optional>
7 #include <stdexcept>
8
9 // Inclusion projet
10 #include <Piece.hpp>
11 #include <TypePiece.hpp>
12 #include <Constexpr.hpp>
13 #include "Board.hpp"
14
15 template <std::size_t N, std::size_t M>
16 constexpr bool Board<N, M>::mesure(Coord const &p,
17                                     std::optional<bool> val_mes)
18 {
19
20     Piece p_actuelle = (*this)(p.n, p.m);
21     if (p_actuelle.get_type() == TypePiece::EMPTY)
22     {
23         return false;
24     }
25     else
26     {
27         bool mes;
28         if (val_mes == std::nullopt)
29         {
30             double x = rnd::randreal(0., 1.);
```

```

31     std::size_t indice_mes = 0;
32     double pow_coef{
33         std::pow(
34             std::abs(m_board[indice_mes].second), 2));
35     std::size_t size_board{std::size(m_board)};
36     while (x - pow_coef > 0 && indice_mes < size_board - 1)
37     {
38         x -= pow_coef;
39         indice_mes++;
40         if (indice_mes >= size_board)
41         {
42             throw std::runtime_error("Indice mesuré de mesure trop grand");
43         }
44         pow_coef = std::pow(std::abs(m_board[indice_mes].second), 2);
45     }
46     mes = m_board[indice_mes].first[offset(p.n, p.m)];
47 }
48 else
49 {
50     mes = val_mes.value();
51 }
52 double proba_delete = 0;
53 // std::size_t nbr_elt_suppr{0};
54 for (std::size_t i{std::size(m_board)}; i > 0; i--)
55 {
56     if (m_board[i - 1].first[offset(p.n, p.m)] != mes) //?
57     {
58         proba_delete +=
59             std::pow(std::abs(m_board[i - 1].second), 2);
60

```

```

61         m_board.erase(
62             std::begin(m_board) + i - 1);
63     }
64 }
65 for (auto &e : m_board)
66 {
67     e.second /= std::sqrt(1. - proba_delete);
68 }
69 for (std::size_t i{0}; i < N * M; i++)
70 {
71     if (p_actuelle.get_type() != TypePiece::EMPTY)
72     {
73         update_case(i);
74     }
75 }
76 return mes;
77 }
78 }
79
80 template <std::size_t N, std::size_t M>
81 constexpr bool
82 Board<N, M>::mesure_capture_slide(
83     Coord const &s,
84     Coord const &t,
85     std::function<bool(Board<N, M> const &,
86         Coord const &,
87         Coord const &,
88         std::size_t,
89         std::optional<Coord>)>
90     check_path,

```

```

91     std::optional<bool> val_mes)
92 {
93     std::size_t position = offset(s.n, s.m);
94     Piece p_actuelle = (*this)(s.n, s.m);
95     if (p_actuelle.get_type() == TypePiece::EMPTY)
96     {
97         return false;
98     }
99     else
100     {
101         bool mes;
102         if (val_mes == std::nullopt)
103         {
104             double x = rnd::randreal(0., 1.);
105             std::size_t indice_mes = 0;
106             double pow_coef{
107                 std::pow(std::abs(m_board[0].second), 2)};
108             std::size_t size_board{std::size(m_board)};
109             while (x - pow_coef > 0 && indice_mes < size_board - 1)
110             {
111                 x -= pow_coef;
112                 indice_mes++;
113                 if (indice_mes >= size_board)
114                 {
115                     throw std::runtime_error("Indice mesuré de mesure trop grand");
116                 }
117                 pow_coef = std::pow(std::abs(m_board[indice_mes].second), 2);
118             }
119             // indice_suppr++,
120         }

```

```

121     mes = m_board[indice_mes].first[position] &&
122         check_path(*this, s, t, indice_mes, std::nullopt);
123 }
124 else
125 {
126     mes = val_mes.value();
127 }
128 double proba_delete = 0;
129 // std::size_t nbr_elt_suppr{0};
130 for (std::size_t i{std::size(m_board)}; i > 0; i--)
131 {
132     if ((m_board[i - 1].first[position] &&
133         check_path(*this, s, t, i - 1, std::nullopt)) != mes)
134     {
135         proba_delete +=
136             std::pow(std::abs(m_board[i - 1].second), 2);
137
138         m_board.erase(
139             std::begin(m_board) + i - 1);
140     }
141 }
142 for (auto &e : m_board)
143 {
144     e.second /= std::sqrt(1. - proba_delete);
145 }
146 for (std::size_t i{0}; i < N * M; i++)
147 {
148     if (p_actuelle.get_type() != TypePiece::EMPTY)
149     {
150         update_case(i);

```

```

151     }
152 }
153 return mes;
154 }
155 }
156
157 template <std::size_t N, std::size_t M>
158 CONSTEXPR bool
159 Board<N, M>::mesure_castle(
160     Coord const &king,
161     Coord const &rook,
162     std::optional<bool> val_mes)
163 {
164     std::size_t position_king = offset(king.n, king.m);
165     std::size_t position_rook = offset(rook.n, rook.m);
166
167     bool mes;
168     if (val_mes == std::nullopt)
169     {
170         double x = rnd::randreal(0., 1.);
171         std::size_t indice_mes = 0;
172         std::size_t size_board{std::size(m_board)};
173         double pow_coef{
174             std::pow(std::abs(m_board[0].second), 2)};
175
176         while (x - pow_coef > 0 && indice_mes < size_board - 1)
177         {
178             x -= pow_coef;
179             indice_mes++;
180             if (indice_mes >= size_board)

```

```

181     {
182         throw std::runtime_error("Indice mesuré de mesure trop grand");
183     }
184     pow_coef = std::pow(std::abs(m_board[indice_mes].second), 2);
185 }
186 mes = m_board[indice_mes].first[position_king] &&
187       m_board[indice_mes].first[position_rook] &&
188       check_path_straight_1_instance(*this, king, rook, indice_mes, std::nullopt);
189 }
190 else
191 {
192     mes = val_mes.value();
193 }
194 double proba_delete = 0;
195 // std::size_t nbr_elt_suppr{0};
196 for (std::size_t i{std::size(m_board)}; i > 0; i--)
197 {
198     if ((m_board[i - 1].first[position_king] &&
199         m_board[i - 1].first[position_rook] &&
200         check_path_straight_1_instance(*this, king, rook, i - 1, std::nullopt)) != mes)
201     {
202         proba_delete +=
203             std::pow(std::abs(m_board[i - 1].second), 2);
204     }
205     m_board.erase(
206         std::begin(m_board) + i - 1);
207 }
208 }
209 for (auto &e : m_board)
210 {

```

```
211     e.second /= std::sqrt(1. - proba_delete);
212 }
213 for (std::size_t i{0}; i < N * M; i++)
214 {
215     if (m_piece_board[i].get_type() != TypePiece::EMPTY)
216     {
217         update_case(i);
218     }
219 }
220 return mes;
221
```



# move.hpp

```
1 // Lib standard
2 #include <array>
3 #include <algorithm>
4 #include <cassert>
5 #include <utility>
6 #include <functional>
7 #include <cmath>
8 #include <optional>
9 #include <stdexcept>
10
11 // Inclusion projet
12 #include <Piece.hpp>
13 #include <CMatrx.hpp>
14 #include <Unitary.hpp>
15 #include <TypePiece.hpp>
16 #include <math_utility.hpp>
17 #include <Constexpr.hpp>
18 #include <Move.hpp>
19 #include "Board.hpp"
20
21 template <std::size_t N, std::size_t M>
22 CONSTEXPR void Board<N, M>::king_side_castle(Coord const &k,
23                                               Coord const &r,
24                                               std::optional<bool> val_mes)
25 {
26     std::size_t king = offset(k.n, k.m);
27     std::size_t new_king = offset(k.n, k.m + 2);
28     std::size_t rook = offset(r.n, r.m);
29     std::size_t new_rook = offset(r.n, r.m - 2);
30     if (mesure_castle(k, r, val_mes))
```

```

31 {
32     for (std::size_t i{0}; i < std::size(m_board); i++)
33     {
34         move_1_instance(
35             std::array<bool, 2>{true, false}, i,
36             MATRIX_ISWAP,
37             std::array<std::size_t, 2>{king, new_king});
38         move_1_instance(
39             std::array<bool, 2>{true, false}, i,
40             MATRIX_ISWAP,
41             std::array<std::size_t, 2>{rook, new_rook});
42     }
43     m_piece_board[new_king] = std::move(m_piece_board[king]);
44     m_piece_board[new_rook] = std::move(m_piece_board[rook]);
45     m_piece_board[king] = Piece();
46     m_piece_board[rook] = Piece();
47 }
48 }
49 template <std::size_t N, std::size_t M>
50 constexpr void Board<N, M>::queen_side_castle(Coord const &k,
51                                                  Coord const &r,
52                                                  std::optional<bool> val_mes)
53 {
54     std::size_t king = offset(k.n, k.m);
55     std::size_t new_king = offset(k.n, k.m - 2);
56     std::size_t rook = offset(r.n, r.m);
57     std::size_t new_rook = offset(r.n, r.m + 3);
58     if (measure_castle(k, r, val_mes))
59     {
60         for (std::size_t i{0}; i < std::size(m_board); i++)

```

```

61     {
62         move_1_instance(
63             std::array<bool, 2>{true, false}, i,
64             MATRIX_ISWAP,
65             std::array<std::size_t, 2>{king, new_king});
66         move_1_instance(
67             std::array<bool, 2>{true, false}, i,
68             MATRIX_ISWAP,
69             std::array<std::size_t, 2>{rook, new_rook});
70     }
71     m_piece_board[new_king] = std::move(m_piece_board[king]);
72     m_piece_board[new_rook] = std::move(m_piece_board[rook]);
73     m_piece_board[king] = Piece();
74     m_piece_board[rook] = Piece();
75 }
76 }
77 template <std::size_t N, std::size_t M>
78 constexpr void
79 Board<N, M>::move_classic_jump(Coord const &s, Coord const &t,
80                                std::optional<bool> val_mes)
81 {
82     std::size_t source = offset(s.n, s.m);
83     std::size_t target = offset(t.n, t.m);
84
85     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
86         (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
87          m_piece_board[target].get_color() == m_piece_board[source].get_color()))
88     {
89         std::size_t const size_board{std::size(m_board)};
90         for (std::size_t i{0}; i < size_board; i++)

```

```

91     {
92         move_1_instance(
93             std::array<bool, 2>{m_board[i].first[source], false}, i,
94             MATRIX_ISWAP,
95             std::array<std::size_t, 2>{source, target});
96     }
97     m_piece_board[target] = std::move(m_piece_board[source]);
98     m_piece_board[source] = Piece();
99 }
100 else
101 {
102     if (m_piece_board[source].same_color(m_piece_board[target]))
103     {
104         std::optional<bool> negation;
105         if (val_mes.has_value())
106         {
107             negation = !val_mes.value();
108         }
109         if (!mesure(t, negation))
110         {
111             for (std::size_t i{0}; i < std::size(m_board); i++)
112             {
113                 move_1_instance(
114                     std::array<bool, 2>{m_board[i].first[source], false},
115                     i, MATRIX_ISWAP,
116                     std::array<std::size_t, 2>{source, target});
117             }
118             m_piece_board[target] = std::move(m_piece_board[source]);
119             m_piece_board[source] = Piece();
120         }

```

```

121     }
122     else
123     {
124         if (measure(s, val_mes))
125         {
126             for (std::size_t i{0}; i < std::size(m_board); i++)
127             {
128                 constexpr CMatrix<8> A{
129                     MATRIX_JUMP
130                     .tensoriel_product(
131                         CMatrix<2>::identity()) *
132                         CMatrix<2>::identity()
133                     .tensoriel_product(MATRIX_JUMP)};
134
135                 move_1_instance(
136                     std::array<bool, 3>{m_board[i].first[source],
137                                         m_board[i].first[target],
138                                         false},
139                     i,
140                     A,
141                     std::array<std::size_t, 3>{source,
142                                                 target,
143                                                 N * M + 1});
144             }
145             m_piece_board[target] = std::move(m_piece_board[source]);
146             m_piece_board[source] = Piece();
147         }
148     }
149 }
150 }

```

```

151
152 template <std::size_t N, std::size_t M>
153 CONSTEXPR bool
154 Board<N, M>::move_pawn_one_step(Coord const &s, Coord const &t,
155                                 std::optional<bool> val_mes)
156 {
157     std::size_t source = offset(s.n, s.m);
158     std::size_t target = offset(t.n, t.m);
159
160     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
161         (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
162          m_piece_board[target].get_color() == m_piece_board[source].get_color()))
163     {
164         std::size_t const size_board{std::size(m_board)};
165         for (std::size_t i{0}; i < size_board; i++)
166         {
167             move_1_instance(
168                 std::array<bool, 2>{m_board[i].first[source], false},
169                 i, MATRIX_ISWAP,
170                 std::array<std::size_t, 2>{source, target});
171         }
172         m_piece_board[target] = std::move(m_piece_board[source]);
173         m_piece_board[source] = Piece();
174         return true;
175     }
176     else
177     {
178         std::optional<bool> negation;
179         if (val_mes.has_value())
180         {

```

```

181     negation = !val_mes.value();
182 }
183 if (!mesure(t, negation))
184 {
185     for (std::size_t i{0}; i < std::size(m_board); i++)
186     {
187         move_1_instance(
188             std::array<bool, 2>{m_board[i].first[source], false},
189             i, MATRIX_ISWAP,
190             std::array<std::size_t, 2>{source, target});
191     }
192     m_piece_board[target] = std::move(m_piece_board[source]);
193     m_piece_board[source] = Piece();
194     return true;
195 }
196 }
197 return false;
198 }
199
200 template <std::size_t N, std::size_t M>
201 constexpr bool
202 Board<N, M>::move_pawn_two_step(Coord const &s, Coord const &t,
203                                 std::optional<bool> val_mes)
204 {
205     {
206         std::size_t source = offset(s.n, s.m);
207         std::size_t target = offset(t.n, t.m);
208         if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
209             (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
210              m_piece_board[target].get_color() == m_piece_board[source].get_color()))

```

```

211 {
212     std::size_t const size_board{std::size(m_board)};
213     for (std::size_t i{0}; i < size_board; i++)
214     {
215         move_1_instance(
216             std::array<bool, 3>{!check_path_straight_1_instance(
217                 *this, s, t, i, std::nullopt),
218                 false,
219                 m_board[i].first[source]},
220             i, MATRIX_SLIDE,
221             std::array<std::size_t, 3>{N * M + 1, target, source});
222     }
223     m_piece_board[target] = m_piece_board[source];
224     update_case(source);
225     return true;
226 }
227 else
228 {
229     std::optional<bool> negation;
230     if (val_mes.has_value())
231     {
232         negation = !val_mes.value();
233     }
234     if (!measure(t, negation))
235     {
236         for (std::size_t i{0}; i < std::size(m_board); i++)
237         {
238             move_1_instance(
239                 std::array<bool, 3>{!check_path_straight_1_instance(
240                     *this, s, t, i, std::nullopt),

```



```

241                                     false,
242                                     m_board[i].first[source]],
243     i, MATRIX_SLIDE,
244     std::array<std::size_t, 3>{N * M + 1, target, source});
245 }
246 m_piece_board[target] = std::move(m_piece_board[source]);
247 update_case(source);
248 return true;
249 }
250 }
251 return false;
252 }
253
254 template <std::size_t N, std::size_t M>
255 CONSTEXPR bool Board<N, M>::capture_pawn(
256     Coord const &s,
257     Coord const &t,
258     std::optional<bool> val_mes)
259 {
260     std::size_t source = offset(s.n, s.m);
261     std::size_t target = offset(t.n, t.m);
262     if (measure(s, val_mes))
263     {
264         for (std::size_t i{0}; i < std::size(m_board); i++)
265         {
266             if (m_board[i].first[target])
267             {
268                 CONSTEXPR CMatrix<8> A{
269                     MATRIX_JUMP
270                     .tensoriel_product(

```

```

271         CMatrix<2>::identity()) *
272         CMatrix<2>::identity()
273         .tensoriel_product(MATRIX_JUMP));
274
275     move_1_instance(
276         std::array<bool, 3>{
277             m_board[i].first[source],
278             m_board[i].first[target],
279             false},
280         i, A,
281         std::array<std::size_t, 3>{source, target, N * M + 1});
282     }
283 }
284 m_piece_board[target] = m_piece_board[source];
285 update_case(source);
286 return true;
287 }
288 return false;
289 }
290
291 template <std::size_t N, std::size_t M>
292 constexpr bool
293 Board<N, M>::move_ennassant(Coord const &s, Coord const &t, Coord const &ep,
294                             std::optional<bool> val_mes)
295 {
296     std::size_t source = offset(s.n, s.m);
297     std::size_t target = offset(t.n, t.m);
298     std::size_t enpassant = offset(ep.n, ep.m);
299
300     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||

```

```

301     (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
302      m_piece_board[target].get_color() == m_piece_board[source].get_color()))
303 {
304
305     std::size_t const size_board{std::size(m_board)};
306     for (std::size_t i{0}; i < size_board; i++)
307     {
308         if (m_board[i].first[enpassant])
309         {
310             // permet d'enlever le pion pris en passant
311             move_1_instance(
312                 std::array<bool, 2>{m_board[i].first[enpassant], false},
313                 i, MATRIX_ISWAP,
314                 std::array<std::size_t, 2>{enpassant, N * M + 1});
315             move_1_instance(
316                 std::array<bool, 2>{m_board[i].first[source], false},
317                 i, MATRIX_ISWAP,
318                 std::array<std::size_t, 2>{source, target});
319         }
320     }
321     m_piece_board[target] = std::move(m_piece_board[source]);
322     m_piece_board[source] = Piece();
323     m_piece_board[enpassant] = Piece();
324     return true;
325 }
326 else
327 {
328     if (m_piece_board[source].same_color(m_piece_board[target]))
329     {
330         std::optional<bool> negation;

```

```

331     if (val_mes.has_value())
332     {
333         negation = !val_mes.value();
334     }
335     if (!mesure(t, negation))
336     {
337
338         for (std::size_t i{0}; i < std::size(m_board); i++)
339         {
340             if (m_board[i].first[enpassant])
341             {
342                 // permet d'enlever le pion pris en passant
343                 move_1_instance(
344                     std::array<bool, 2>{
345                         m_board[i].first[enpassant],
346                         false},
347                     i, MATRIX_ISWAP,
348                     std::array<std::size_t, 2>{enpassant, N * M + 1});
349
350                 move_1_instance(
351                     std::array<bool, 2>{
352                         m_board[i].first[source],
353                         false},
354                     i, MATRIX_ISWAP,
355                     std::array<std::size_t, 2>{source, target});
356             }
357         }
358         m_piece_board[target] = m_piece_board[source];
359         m_piece_board[source] = Piece();
360         return true;

```

```

361     }
362     return false;
363 }
364 else
365 {
366     if (mesure(s, val_mes))
367     {
368         for (std::size_t i{0}; i < std::size(m_board); i++)
369         {
370             if (m_board[i].first[enpassant] ||
371                 m_board[i].first[target])
372             {
373                 // permet d'enlever le pion pris en passant
374                 move_1_instance(
375                     std::array<bool, 2>{
376                         m_board[i].first[enpassant], false},
377                     i, MATRIX_ISWAP,
378                     std::array<std::size_t, 2>{
379                         enpassant, N * M + 1});
380                 // ces deux instructions représentent un mouvement
381                 // de capture jump
382                 move_1_instance(
383                     std::array<bool, 2>{
384                         m_board[i].first[target], false},
385                     i, MATRIX_ISWAP,
386                     std::array<std::size_t, 2>{
387                         target, N * M + 1});
388
389                 move_1_instance(
390                     std::array<bool, 2>{

```

```

391         m_board[i].first[source], false},
392         i, MATRIX_ISWAP,
393         std::array<std::size_t, 2>{
394             source, target});
395     }
396 }
397 m_piece_board[target] = std::move(m_piece_board[source]);
398 m_piece_board[source] = Piece();
399 m_piece_board[enpassant] = Piece();
400 return true;
401 }
402 }
403 }
404 return false;
405 }
406
407 template <std::size_t N, std::size_t M>
408 CONSTEXPR void
409 Board<N, M>::move_split_jump(Coord const &s, Coord const &t1, Coord const &t2)
410 {
411     std::size_t const source = offset(s.n, s.m);
412     std::size_t const target1 = offset(t1.n, t1.m);
413     std::size_t const target2 = offset(t2.n, t2.m);
414     std::size_t const size_board{std::size(m_board)};
415     for (std::size_t i{0}; i < size_board; i++)
416     {
417         move_1_instance(
418             std::array<bool, 3>{
419                 m_board[i].first[target2],
420                 m_board[i].first[target1],

```

```

421         m_board[i].first[source]},
422         i, MATRIX_SPLIT,
423         std::array<std::size_t, 3>{target2, target1, source});
424     }
425     m_piece_board[target1] = m_piece_board[source];
426
427     if (m_piece_board[target1].get_type() == TypePiece::EMPTY &&
428         m_piece_board[target2].get_type() == TypePiece::EMPTY)
429     {
430         m_piece_board[target2] = std::move(m_piece_board[source]);
431         m_piece_board[source] = Piece();
432         update_case(target1);
433         update_case(target2);
434     }
435     else
436     {
437         m_piece_board[target2] = m_piece_board[source];
438         update_case(source);
439         update_case(target1);
440         update_case(target2);
441         update_board();
442     }
443 }
444
445 template <std::size_t N, std::size_t M>
446 constexpr void
447 Board<N, M>::move_merge_jump(Coord const &s1, Coord const &s2, Coord const &t)
448 {
449     std::size_t source1 = offset(s1.n, s1.m);
450     std::size_t source2 = offset(s2.n, s2.m);

```

```

451     std::size_t target = offset(t.n, t.m);
452     std::size_t const size_board{std::size(m_board)};
453     for (std::size_t i{0}; i < size_board; i++)
454     {
455         move_1_instance(
456             std::array<bool, 3>{
457                 m_board[i].first[source1],
458                 m_board[i].first[source2],
459                 m_board[i].first[target]},
460             i, MATRIX_MERGE,
461             std::array<std::size_t, 3>{source1, source2, target});
462     }
463     m_piece_board[target] = m_piece_board[source1];
464     update_board();
465     update_case(target);
466     update_case(source1);
467     update_case(source2);
468 }
469
470 template <std::size_t N, std::size_t M>
471 constexpr void
472 Board<N, M>::move_classic_slide(
473     Coord const &s,
474     Coord const &t,
475     std::function<
476         bool(
477             Board<N, M> const &,
478             Coord const &,
479             Coord const &,
480             std::size_t,

```



```

481         std::optional<Coord>>
482         check_path,
483         std::optional<bool> val_mes)
484 {
485     std::size_t source = offset(s.n, s.m);
486     std::size_t target = offset(t.n, t.m);
487     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
488         (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
489          m_piece_board[target].get_color() == m_piece_board[source].get_color()))
490     {
491         std::size_t const size_board{std::size(m_board)};
492         for (std::size_t i{0}; i < size_board; i++)
493         {
494             move_1_instance(
495                 std::array<bool, 3>{
496                     !check_path(
497                         *this, s, t, i, std::nullopt),
498                     false,
499                     m_board[i].first[source]},
500                 i, MATRIX_SLIDE,
501                 std::array<std::size_t, 3>{N * M + 1, target, source});
502         }
503         m_piece_board[target] = m_piece_board[source];
504         update_case(source);
505     }
506     else
507     {
508         if (m_piece_board[source].same_color(m_piece_board[target]))
509         {
510             std::optional<bool> negation;

```

```

511     if (val_mes.has_value())
512     {
513         negation = !val_mes.value();
514     }
515     if (!measure(t, negation))
516     {
517         for (std::size_t i{0}; i < std::size(m_board); i++)
518         {
519             move_1_instance(
520                 std::array<bool, 3>{
521                     !check_path(*this, s, t, i, std::nullopt),
522                     false,
523                     m_board[i].first[source]},
524                     i, MATRIX_SLIDE,
525                     std::array<std::size_t, 3>{N * M + 1, target, source});
526         }
527         m_piece_board[target] = std::move(m_piece_board[source]);
528         m_piece_board[source] = Piece();
529     }
530 }
531 else
532 {
533     if (measure_capture_slide(s, t, check_path, val_mes))
534     {
535         for (std::size_t i{0}; i < std::size(m_board); i++)
536         {
537             auto const A{
538                 MATRIX_ISWAP.tensoriel_product(
539                     CMatrix<2>::identity()) *
540                     CMatrix<2>::identity()

```

```

541         .tensoriel_product(MATRIX_ISWAP)}};
542
543     move_1_instance(
544         std::array<bool, 3>{
545             m_board[i].first[source],
546             m_board[i].first[target], false},
547         i, A,
548         std::array<std::size_t, 3>{source, target, N * M + 1});
549     }
550     m_piece_board[target] = std::move(m_piece_board[source]);
551     m_piece_board[source] = Piece();
552 }
553 }
554 }
555 }
556
557 template <std::size_t N, std::size_t M>
558 constexpr void
559 Board<N, M>::move_split_slide(
560     Coord const &s,
561     Coord const &t1,
562     Coord const &t2,
563     std::function<
564         bool(
565             Board<N, M> const &,
566             Coord const &,
567             Coord const &,
568             std::size_t,
569             std::optional<Coord>>>
570     check_path)

```

```

571 {
572     std::size_t source = offset(s.n, s.m);
573     std::size_t target1 = offset(t1.n, t1.m);
574     std::size_t target2 = offset(t2.n, t2.m);
575     std::size_t const size_board{std::size(m_board)};
576     for (std::size_t i{0}; i < size_board; i++)
577     {
578         move_1_instance(
579             std::array<bool, 5>{
580                 !check_path(*this, s, t2, i, t1),
581                 !check_path(*this, s, t1, i, t2),
582                 m_board[i].first[target2],
583                 m_board[i].first[target1],
584                 m_board[i].first[source]},
585             i, MATRIX_SPLIT_SLIDE,
586             std::array<std::size_t, 5>{
587                 N * M + 1,
588                 N * M + 1,
589                 target2,
590                 target1,
591                 source});
592         /*La matrice split slide est créée pour
593         être utilisé sans appliquer de porte cnot a
594         un chemin, nos fonctions check_path renvoie
595         un résultat où l'on a appliquer la porte cnot */
596     }
597     if (m_piece_board[target1].get_type() == TypePiece::EMPTY &&
598         m_piece_board[target2].get_type() == TypePiece::EMPTY)
599     {
600         m_piece_board[target1] = m_piece_board[source];

```

```

601     m_piece_board[target2] = m_piece_board[source];
602     update_case(source);
603     update_case(target1);
604     update_case(target2);
605 }
606 else
607 {
608     m_piece_board[target1] = m_piece_board[source];
609     m_piece_board[target2] = m_piece_board[source];
610     update_case(source);
611     update_case(target1);
612     update_case(target2);
613     update_board();
614 }
615 }
616
617 template <std::size_t N, std::size_t M>
618 CONSTEXPR void
619 Board<N, M>::move_merge_slide(
620     Coord const &s1,
621     Coord const &s2,
622     Coord const &t,
623     std::function<
624         bool(
625             Board<N, M> const &,
626             Coord const &,
627             Coord const &,
628             std::size_t,
629             std::optional<Coord>>>
630     check_path)

```

```

631 {
632     std::size_t const source1 = offset(s1.n, s1.m);
633     std::size_t const source2 = offset(s2.n, s2.m);
634     std::size_t const target = offset(t.n, t.m);
635     std::size_t const size_board{std::size(m_board)};
636     for (std::size_t i{0}; i < size_board; i++)
637     {
638         move_1_instance(
639             std::array<bool, 5>{
640                 !check_path(*this, s2, t, i, s1),
641                 !check_path(*this, s1, t, i, s2),
642                 m_board[i].first[source1],
643                 m_board[i].first[source2],
644                 m_board[i].first[target]},
645             i, MATRIX_MERGE_SLIDE,
646             std::array<std::size_t, 5>{
647                 N * M + 1,
648                 N * M + 1,
649                 source1,
650                 source2,
651                 target});
652         /* La matrice merge slide est créée pour
653            être utilisé sans appliquer de porte cnot au chemin,
654            nos fonctions check_path renvoie un résultat où l'on
655            a appliqué la porte cnot */
656     }
657     m_piece_board[target] = m_piece_board[source1];
658     update_board();
659     update_case(target);
660     update_case(source1);

```

```

661     update_case(source2);
662 }
663
664 template <std::size_t N, std::size_t M>
665 CONSTEXPR bool Board<N, M>::move_pawn(
666     Coord const &s,
667     Coord const &t,
668     std::optional<bool> val_mes) noexcept
669 {
670     bool move_exec{false};
671     auto abs_diff{[](std::size_t x, std::size_t y) -> std::size_t
672         {
673             return (x >= y) ? x - y : y - x;
674         }};
675     std::size_t diff_line{abs_diff(s.n, t.n)};
676     if (diff_line == 2)
677     {
678         move_exec = move_pawn_two_step(s, t, val_mes);
679         m_ep = Coord((s.n + t.n) / 2, s.m);
680     }
681     else if (diff_line == 1)
682     {
683         std::size_t diff_col{abs_diff(s.m, t.m)};
684         if (diff_col == 1)
685         {
686
687             if (m_ep != std::nullopt && m_ep == t)
688             {
689                 int step{
690                     ((*this)(s.n, s.m).get_color() ==

```

```

691         Color::WHITE)
692         ? -1
693         : 1};
694     Coord ep;
695     ep.m = m_ep->m;
696     ep.n = m_ep->n - step;
697     move_exec = move_enpassant(s, t, ep, val_mes);
698 }
699 else
700 {
701     move_exec = capture_pawn(s, t, val_mes);
702 }
703 }
704 else
705 {
706     move_exec = move_pawn_one_step(s, t, val_mes);
707 }
708 m_ep = std::nullopt;
709 }
710 return move_exec;
711 }
712
713 template <std::size_t N, std::size_t M>
714 constexpr void
715 Board<N, M>::move_promotion(
716     Move const &move,
717     std::optional<bool> val_mes)
718 {
719     TypePiece p{move.promote.piece};
720     Coord s{move.promote.src};

```



```

721   Coord t{move.promote.arv};
722
723   if (p == TypePiece::QUEEN ||
724       p == TypePiece::KNIGHT ||
725       p == TypePiece::BISHOP ||
726       p == TypePiece::ROOK)
727   {
728       if (move_pawn(s, t, val_mes))
729       {
730           Piece piece{p, m_piece_board[offset(t.n, t.m)].get_color()};
731           m_piece_board[offset(t.n, t.m)] = std::move(piece);
732       }
733   }
734   else
735   {
736       throw std::runtime_error("Piece non valide pour faire une promotion");
737   }
738 }
739
740 template <std::size_t N, std::size_t M>
741 constexpr void
742 Board<N, M>::move_classic(
743     Coord const &s,
744     Coord const &t,
745     std::optional<bool> val_mes)
746 {
747     TypePiece piece{(*this)(s.n, s.m).get_type()};
748     switch (piece)
749     {
750     case TypePiece::KING:

```

```

751     if constexpr (N == 8 && M == 8)
752     {
753         auto abs_diff{[] (std::size_t x, std::size_t y) -> std::size_t
754             {
755                 return (x >= y) ? x - y : y - x;
756             }};
757         std::size_t diff_column{abs_diff(s.m, t.m)};
758         if (diff_column == 2)
759         {
760             if (s.m > t.m)
761             {
762                 queen_side_castle(s, Coord(s.n, t.m - 2), val_mes);
763             }
764             else
765             {
766                 king_side_castle(s, Coord(s.n, t.m + 1), val_mes);
767             }
768         }
769         else
770         {
771             move_classic_jump(s, t, val_mes);
772         }
773     }
774     else
775     {
776         move_classic_jump(s, t, val_mes);
777     }
778     break;
779 case TypePiece::KNIGHT:
780     move_classic_jump(s, t, val_mes);

```

```

781     break;
782     case TypePiece::BISHOP:
783         move_classic_slide(
784             s, t,
785             &check_path_diagonal_1_instance<N, M>,
786             val_mes);
787         break;
788     case TypePiece::ROOK:
789         move_classic_slide(
790             s, t,
791             &check_path_straight_1_instance<N, M>,
792             val_mes);
793         break;
794     case TypePiece::QUEEN:
795         move_classic_slide(
796             s, t,
797             &check_path_queen_1_instance<N, M>,
798             val_mes);
799         break;
800     case TypePiece::PAWN:
801         move_pawn(s, t, val_mes);
802         return;
803     case TypePiece::EMPTY:
804     default:
805         break;
806 }
807 m_ep = std::nullopt;
808 }
809
810 template <std::size_t N, std::size_t M>

```

```

811 CONSTEXPR void Board<N, M>::move_split(Coord const &s,
812                                         Coord const &t1,
813                                         Coord const &t2)
814 {
815     TypePiece piece{(*this)(s.n, s.m).get_type()};
816     switch (piece)
817     {
818     case TypePiece::KING:
819     case TypePiece::KNIGHT:
820         move_split_jump(s, t1, t2);
821         break;
822     case TypePiece::BISHOP:
823         move_split_slide(s, t1, t2, &check_path_diagonal_1_instance<N, M>);
824         break;
825     case TypePiece::ROOK:
826         move_split_slide(s, t1, t2, &check_path_straight_1_instance<N, M>);
827         break;
828     case TypePiece::QUEEN:
829         move_split_slide(s, t1, t2, &check_path_queen_1_instance<N, M>);
830         break;
831     case TypePiece::PAWN:
832     case TypePiece::EMPTY:
833     default:
834         break;
835     }
836     m_ep = std::nullopt;
837 }
838
839 template <std::size_t N, std::size_t M>
840 CONSTEXPR void Board<N, M>::move_merge(Coord const &s1,

```

```

841             Coord const &s2,
842             Coord const &t)
843 {
844     TypePiece piece1{(*this)(s1.n, s1.m).get_type()};
845     TypePiece piece2{(*this)(s2.n, s2.m).get_type()};
846     if (piece1 != piece2)
847     {
848         return;
849     }
850     switch (piece1)
851     {
852     case TypePiece::KING:
853     case TypePiece::KNIGHT:
854         move_merge_jump(s1, s2, t);
855         break;
856     case TypePiece::BISHOP:
857         move_merge_slide(s1, s2, t, &check_path_diagonal_1_instance<N, M>);
858         break;
859     case TypePiece::ROOK:
860         move_merge_slide(s1, s2, t, &check_path_straight_1_instance<N, M>);
861         break;
862     case TypePiece::QUEEN:
863         move_merge_slide(s1, s2, t, &check_path_queen_1_instance<N, M>);
864         break;
865     case TypePiece::PAWN:
866     case TypePiece::EMPTY:
867     default:
868         break;
869     }
870     m_ep = std::nullopt;

```

```

871 }
872
873 template <std::size_t N, std::size_t M>
874 CONSTEXPR void Board<N, M>::move(Move const &movement, std::optional<bool> val_mes)
875 {
876     if (std::empty(m_board))
877     {
878         throw std::runtime_error("Le plateau est vide impossible de réaliser l'opération move");
879     }
880     switch (movement.type)
881     {
882     case TypeMove::NORMAL:
883         move_classic(movement.normal.src, movement.normal.arv, val_mes);
884         break;
885     case TypeMove::SPLIT:
886         move_split(movement.split.src,
887                     movement.split.arv1,
888                     movement.split.arv2);
889         break;
890     case TypeMove::MERGE:
891         move_merge(movement.merge.src1,
892                     movement.merge.src2,
893                     movement.merge.arv);
894         break;
895     case TypeMove::PROMOTE:
896         move_promotion(movement, val_mes);
897         break;
898     default:
899         break;
900 }

```

```
901 update_board_classic();
902 // update_board();
903
```

# get-proba-move.hpp

```
1 // Lib standard
2 #include <cassert>
3 #include <utility>
4 #include <functional>
5 #include <cmath>
6 #include <optional>
7 #include <stdexcept>
8
9 // Inclusion projet
10 #include <Piece.hpp>
11 #include <TypePiece.hpp>
12 #include <Constexpr.hpp>
13 #include "Board.hpp"
14 #include <math_utility.hpp>
15
16 template <std::size_t N, std::size_t M>
17 constexpr double
18 Board<N, M>::get_proba_mesure(Coord const &p) const noexcept
19 {
20     double res{0};
21     std::size_t size_board = std::size(m_board);
22     for (std::size_t i{0}; i < size_board; i++)
23     {
24         if (m_board[i].first[offset(p.n, p.m)])
25         {
26             res += std::pow(std::abs(m_board[i].second), 2);
27         }
28     }
29     return res;
30 }
```



```

31
32 template <std::size_t N, std::size_t M>
33 CONSTEXPR double
34 Board<N, M>::get_proba_mesure_capture_slide(
35     Coord const &s,
36     Coord const &t,
37     std::function<bool(Board<N, M> const &,
38                         Coord const &,
39                         Coord const &,
40                         std::size_t,
41                         std::optional<Coord>)>
42     check_path) const noexcept
43 {
44     double res{0};
45     std::size_t position = offset(s.n, s.m);
46     std::size_t size_board = std::size(m_board);
47     for (std::size_t i{0}; i < size_board; i++)
48     {
49         if (m_board[i].first[position] &&
50             check_path(*this, s, t, i, std::nullopt))
51         {
52             res += std::pow(std::abs(m_board[i].second), 2);
53         }
54     }
55     return res;
56 }
57
58 template <std::size_t N, std::size_t M>
59 CONSTEXPR double
60 Board<N, M>::get_proba_mesure_castle(

```

```

61     Coord const &king,
62     Coord const &rook) const noexcept
63 {
64     double res{0};
65     std::size_t size_board = std::size(m_board);
66     std::size_t position_king = offset(king.n, king.m);
67     std::size_t position_rook = offset(rook.n, rook.m);
68     for (std::size_t i{0}; i < size_board; i++)
69     {
70         if (m_board[i].first[position_king] &&
71             m_board[i].first[position_rook] &&
72             check_path_straight_1_instance(*this, king, rook, i, std::nullopt))
73         {
74             res += std::pow(std::abs(m_board[i].second), 2);
75         }
76     }
77     return res;
78 }
79
80 template <std::size_t N, std::size_t M>
81 constexpr double Board<N, M>::get_proba_move(Move const &move) const noexcept
82 {
83     Coord s;
84     Coord t;
85     switch (move.type)
86     {
87     case TypeMove::NORMAL:
88         s = move.normal.src;
89         t = move.normal.arv;
90         break;

```

```

91  case TypeMove::PROMOTE:
92      s = move.promote.src;
93      t = move.promote.arv;
94      break;
95  case TypeMove::MERGE:
96      return 1.;
97  case TypeMove::SPLIT:
98      return 1.;
99  default:
100     return 1.;
101 }
102 TypePiece piece{(*this)(s.n, s.m).get_type()};
103 TypePiece piece_target{(*this)(t.n, t.m).get_type()};
104 if ((piece == piece_target) && (*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
105 {
106     return 1.;
107 }
108 switch (piece)
109 {
110 case TypePiece::KING:
111     if constexpr (N == 8 && M == 8)
112     {
113         auto abs_diff{[](std::size_t x, std::size_t y) -> std::size_t
114             {
115                 return (x >= y) ? x - y : y - x;
116             }};
117         std::size_t diff_colum{abs_diff(s.m, t.m)};
118         if (diff_colum == 2)
119         {
120             if (s.m > t.m)

```

```

121     {
122         return get_proba_mesure_castle(s, Coord(s.n, t.m - 2));
123     }
124     else
125     {
126         return get_proba_mesure_castle(s, Coord(s.n, t.m + 1));
127     }
128 }
129 else
130 {
131     if (piece_target != TypePiece::EMPTY)
132     {
133         if ((*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
134         {
135             return 1. - get_proba_mesure(t);
136         }
137         else
138         {
139             return get_proba_mesure(s);
140         }
141     }
142     else
143     {
144         return 1.;
145     }
146 }
147 }
148 else
149 {
150     if (piece_target != TypePiece::EMPTY)

```

```

151     {
152         if ((*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
153         {
154             return 1. - get_proba_mesure(t);
155         }
156         else
157         {
158             return get_proba_mesure(s);
159         }
160     }
161     else
162     {
163         return 1.;
164     }
165 }
166 break;
167 case TypePiece::KNIGHT:
168     if (piece_target != TypePiece::EMPTY)
169     {
170         if ((*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
171         {
172             return 1. - get_proba_mesure(t);
173         }
174         else
175         {
176             return get_proba_mesure(s);
177         }
178     }
179     else
180     {

```

```

181     return 1.;
182 }
183
184 break;
185 case TypePiece::BISHOP:
186     if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
187     {
188         return 1.;
189     }
190     else
191     {
192         if (!(*this)(t.n, t.m).same_color((*this)(s.n, s.m)))
193         {
194             return get_proba_mesure_capture_slide(
195                 s, t,
196                 &check_path_diagonal_1_instance<N, M>);
197         }
198         else
199         {
200             return 1. - get_proba_mesure(t);
201         }
202     }
203     break;
204 case TypePiece::ROOK:
205     if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
206     {
207         return 1.;
208     }
209     else
210     {

```

```

211     if (!(*this)(t.n, t.m).same_color((*this)(s.n, s.m)))
212     {
213         return get_proba_mesure_capture_slide(
214             s, t,
215             &check_path_straight_1_instance<N, M>);
216     }
217     else
218     {
219         return 1. - get_proba_mesure(t);
220     }
221 }
222 break;
223 case TypePiece::QUEEN:
224     if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
225     {
226         return 1.;
227     }
228     else
229     {
230         if (!(*this)(t.n, t.m).same_color((*this)(s.n, s.m)))
231         {
232             return get_proba_mesure_capture_slide(
233                 s, t,
234                 &check_path_queen_1_instance<N, M>);
235         }
236         else
237         {
238             return 1. - get_proba_mesure(t);
239         }
240     }

```

```

241     break;
242 case TypePiece::PAWN:
243     if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY ||
244         ((*this)(t.n, t.m).get_type() == (*this)(s.n, s.m).get_type() &&
245         (*this)(t.n, t.m).get_color() == (*this)(s.n, s.m).get_color()))
246     {
247         return 1.;
248     }
249     else
250     {
251         return 1. - get_proba_mesure(t);
252     }
253 case TypePiece::EMPTY:
254 default:
255     break;
256 }
257 return 1.;
258

```



# Piece.hpp

```
1  #ifndef PIECE_HPP
2  #define PIECE_HPP
3
4  #include <Color.hpp>
5  #include <Coord.hpp>
6  #include <concepts>
7  #include <vector>
8  #include <forward_list>
9  #include <constexpr.hpp>
10 #include <Move.hpp>
11 #include "TypePiece.hpp"
12
13 template <std::size_t N, std::size_t M>
14 class Board;
15
16 /**
17  * @brief Class conservant le type et la couleur d'une pièce.
18  * Elle permet aussi de recuperer ses mouvement possible
19  */
20 class Piece
21 {
22 public:
23     constexpr inline Piece() noexcept;
24
25     /**
26      * @brief Construit une piece à l'aide de son type et de sa couleur
27      *
28      * @param[in] piece Le type de la piece
29      * @param[in] color La couleur de la piece
30      */
```

```

31 constexpr inline Piece(TypePiece piece, Color color) noexcept;
32 constexpr inline Piece(Piece const &) = default;
33 constexpr inline Piece &operator=(Piece const &) = default;
34
35 /**
36  * @brief Déplacement d'un objet piece vers un autre objet piece
37  */
38 constexpr inline Piece(Piece &&) = default;
39 constexpr inline Piece &operator=(Piece &&) = default;
40 constexpr inline ~Piece() = default;
41
42 /**
43  * @brief Renvoie le type de la piece
44  *
45  * @return Une énumération TypePiece
46  */
47 constexpr inline TypePiece get_type() const noexcept;
48
49 /**
50  * @brief Renvoie la couleur d'un piece
51  *
52  * @return Color La couleur de la piece
53  */
54 constexpr inline Color get_color() const noexcept;
55
56 /**
57  * @brief Récupère la liste des mouvements classiques pour une piece
58  *
59  * @tparam N Le nombre de ligne du plateau
60  * @tparam M Le nombre de colonne du plateau

```

```

61  * @param[in] board Le plateau
62  * @param[in] pos la position de la piece sur le plateau
63  * @warning Le resultat n'est pas garanti si la position ne concorde pas
64  * avec la position de la piece sur le plateau
65  * @return std::forward_list<Coord> La liste de coordonnées des positions
66  * d'arrivées de chaque mouvement possible
67  */
68  template <std::size_t N, std::size_t M>
69  CONSTEXPR std::forward_list<Coord>
70  get_list_normal_move(Board<N, M> const &board,
71                      Coord const &pos) const noexcept;
72
73  /**
74   * @brief Récupère la liste des mouvements split pour une piece
75   *
76   * @tparam N Le nombre de ligne du plateau
77   * @tparam M Le nombre de colonne du plateau
78   * @param[in] board Le plateau
79   * @param[in] pos la position de la piece sur le plateau
80   * @warning Le resultat n'est pas garanti si la position ne concorde pas
81   * avec la position de la piece sur le plateau
82   * @warning Ne renvoie qu'une liste de coordonnées uniques car
83   * l'ensemble des mouvements possibles est toutes les couples
84   * de coordonnées de la liste
85   * @return std::forward_list<Coord> La liste de coordonnées
86   * des positions d'arrivées possible sachant que chaque élément
87   * représente une seule des deux cases d'arrivées d'un split move
88   */
89  template <std::size_t N, std::size_t M>
90  CONSTEXPR std::forward_list<Coord>

```

```

91  get_list_split_move(Board<N, M> const &board,
92                      Coord const &pos) const noexcept;
93
94  /**
95   * @brief Teste si la piece est blanche
96   *
97   * @return bool true si la piece est blanche, false sinon
98   */
99  constexpr inline bool is_white() const noexcept;
100
101  /**
102   * @brief Teste si la piece est noire
103   *
104   * @return bool true si la piece est noire, false sinon
105   */
106  constexpr inline bool is_black() const noexcept;
107
108  /**
109   * @brief Teste si l'autre piece est de la meme couleur
110   *
111   * @param[in] other L'autre pièce à comparer
112   *
113   * @return bool true si la piece est de la meme couleur, false sinon
114   */
115  constexpr inline bool same_color(Piece const &other) const noexcept;
116
117  /**
118   * @brief Teste si les mouvement de la piece sont si la
119   * pièce est un pion un mouvement de promotion
120   *

```

```

121  * @note peut etre utiliser sans verifier si la pièce est un pion
122  *
123  * @param[in] board Le plateau
124  * @param[in] pos La position de la pièce
125  * @return true si le mouvement possible est un mouvement
126  * de promotion
127  * @return false sinon
128  */
129  template <std::size_t N, std::size_t M>
130  CONSTEXPR bool
131  check_if_use_move_promote(Board<N, M> const &board,
132                           Coord const &pos) const noexcept;
133
134  /**
135   * @brief Renvoie la liste de toutes les promotions
136   *
137   * @warning Ne verifie pas la validité du mouvement,
138   * de la position ou du type de la pièce
139   *
140   * @param[in] board Le plateau
141   * @param[in] pos La position du pion
142   * @return La liste de tout les mouvements de promotion
143   */
144  template <std::size_t N, std::size_t M>
145  CONSTEXPR std::forward_list<Move>
146  get_list_promote(Board<N, M> const &board,
147                  Coord const &pos) const noexcept;
148
149 private:
150  /**

```

```

151  * @brief Enumération utilisé dans le template des fonctions
152  * qui récupère les listes de mouvement pour modifier leurs
153  * comportement lors de la prise de pièce.
154  */
155  enum class Move_Mode
156  {
157      /**
158       * @brief Dans le cas du mouvement classique on
159       * autorise la prise de pièce.
160       */
161      NORMAL,
162
163      /**
164       * @brief Dans le cas du split on interdit la
165       * prise de pièce.
166       */
167      SPLIT
168  };
169
170  /**
171   * @brief renvoie la valeur absolue d'une soustraction sur des types size_t
172   *
173   * @param[in] x Variable de type size_t
174   * @param[in] y Variable de type size_t
175   * @return std::size_t Retourne l'opération  $|x - y|$ 
176   */
177  constexpr inline static std::size_t
178  abs_substracte(std::size_t x, std::size_t y) noexcept;
179
180  /**

```

```

181  * @brief renvoie la distance entre deux coordonnées
182  * à l'aide de la norme 2
183  *
184  * @param[in] x Première coordonnée
185  * @param[in] y Seconde coordonnée
186  * @return double le résultat de la norme 1
187  */
188  constexpr inline static double
189  norm(Coord const &x, Coord const &y) noexcept;
190
191  /**
192   * @brief Récupère la liste de mouvement d'une pièce pour
193   * les déplacements indiqué de manière récursive
194   * c'est à dire tout les mouvements infini (ex : diagonale du fou)
195   *
196   * @tparam MOVE Le type de mouvement utiliser entre normal
197   * et split (split ne prend pas les mouvements de capture de pièce)
198   * @tparam N Le nombre de ligne du plateau
199   * @tparam M Le nombre de colonne du plateau
200   * @tparam SIZE La taille de la liste des mouvements possible
201   * @param[in] board Le plateau
202   * @param[in] pos La position de la pièce
203   * @param[in] list_move La liste de tout les mouvements possible
204   * @return std::forward_list<Coord> La liste de coordonnées
205   * des positions d'arrivées possible
206   */
207  template <Move_Mode MOVE, std::size_t N, std::size_t M, std::size_t SIZE>
208  CONSTEXPR std::forward_list<Coord>
209  get_list_move_rec(Board<N, M> const &board, Coord const &pos,
210                  std::array<int, SIZE> const &list_move) const noexcept;

```

```

211
212 /**
213  * @brief Récupère la liste de mouvement du roi
214  *
215  * @warning Ne récupère les mouvements du roque que pour un tableau
216  * de taille 8 x 8
217  *
218  * @warning ne verifie pas si c'est bien un roi
219  *
220  * @tparam MOVE Le type de mouvement utiliser entre normal
221  * et split (split ne prend pas les mouvements de capture de pièce)
222  * @tparam N Le nombre de ligne du plateau
223  * @tparam M Le nombre de colonne du plateau
224  * @param[in] board Le plateau
225  * @param[in] pos La position du roi
226  * @return std::forward_list<Coord> La liste de coordonnées
227  * des positions d'arrivées possible pour le roi
228  */
229 template <Move_Mode MOVE, std::size_t N, std::size_t M>
230 CONSTEXPR std::forward_list<Coord>
231 get_list_move_king(Board<N, M> const &board,
232                   Coord const &pos) const noexcept;
233
234 /**
235  * @brief Récupère la liste de mouvement du cavalier
236  *
237  * @warning ne verifie pas si c'est bien un cavalier
238  *
239  * @tparam MOVE Le type de mouvement utiliser entre normal
240  * et split (split ne prend pas les mouvements de capture de pièce)

```



```

241  * @tparam N Le nombre de ligne du plateau
242  * @tparam M Le nombre de colonne du plateau
243  * @param[in] board Le plateau
244  * @param[in] pos La position du cavalier
245  * @return std::forward_list<Coord> La liste de coordonnées
246  * des positions d'arrivées possible pour le cavalier
247  */
248  template <Move_Mode MOVE, std::size_t N, std::size_t M>
249  CONSTEXPR std::forward_list<Coord>
250  get_list_move_knight(Board<N, M> const &board,
251                      Coord const &pos) const noexcept;
252
253  /**
254   * @brief Récupère la liste de mouvement du fou
255   *
256   * @warning ne verifie pas si c'est bien un fou
257   *
258   * @tparam MOVE Le type de mouvement utiliser entre normal
259   * et split (split ne prend pas les mouvements de capture de pièce)
260   * @tparam N Le nombre de ligne du plateau
261   * @tparam M Le nombre de colonne du plateau
262   * @param[in] board Le plateau
263   * @param[in] pos La position du fou
264   * @return std::forward_list<Coord> La liste de coordonnées
265   * des positions d'arrivées possible pour le fou
266   */
267  template <Move_Mode MOVE, std::size_t N, std::size_t M>
268  CONSTEXPR std::forward_list<Coord>
269  get_list_move_bishop(Board<N, M> const &board,
270                      Coord const &pos) const noexcept;

```

```

271
272 /**
273  * @brief Récupère la liste de mouvement de la tour
274  *
275  * @warning ne verifie pas si c'est bien une tour
276  *
277  * @tparam MOVE Le type de mouvement utiliser entre normal
278  * et split (split ne prend pas les mouvements de capture de pièce)
279  * @tparam N Le nombre de ligne du plateau
280  * @tparam M Le nombre de colonne du plateau
281  * @param[in] board Le plateau
282  * @param[in] pos La position de la tour
283  * @return std::forward_list<Coord> La liste de coordonnées
284  * des positions d'arrivées possible pour la tour
285  */
286 template <Move_Mode MOVE, std::size_t N, std::size_t M>
287 CONSTEXPR std::forward_list<Coord>
288 get_list_move_rook(Board<N, M> const &board,
289                  Coord const &pos) const noexcept;
290
291 /**
292  * @brief Récupère la liste de mouvement de la dame
293  *
294  * @warning ne verifie pas si c'est bien une dame
295  *
296  * @tparam MOVE Le type de mouvement utiliser entre normal
297  * et split (split ne prend pas les mouvements de capture de pièce)
298  * @tparam N Le nombre de ligne du plateau
299  * @tparam M Le nombre de colonne du plateau
300  * @param[in] board Le plateau

```

```

301  * @param[in] pos La position de la dame
302  * @return std::forward_list<Coord> La liste de coordonnées
303  * des positions d'arrivées possible pour la dame
304  */
305  template <Move_Mode MOVE, std::size_t N, std::size_t M>
306  CONSTEXPR std::forward_list<Coord>
307  get_list_move_queen(Board<N, M> const &board,
308                    Coord const &pos) const noexcept;
309
310  /**
311   * @brief Récupère la liste de mouvement du pion
312   *
313   * @warning ne verifie pas si c'est bien un pion
314   *
315   * @tparam N Le nombre de ligne du plateau
316   * @tparam M Le nombre de colonne du plateau
317   * @param[in] board Le plateau
318   * @param[in] pos La position du pion
319   * @return std::forward_list<Coord> La liste de coordonnées
320   * des positions d'arrivées possible pour le pion
321   */
322  template <std::size_t N, std::size_t M>
323  CONSTEXPR std::forward_list<Coord>
324  get_list_move_pawn(Board<N, M> const &board,
325                   Coord const &pos) const noexcept;
326
327  /**
328   * @brief Récupère la liste de mouvement pour
329   * n'importe qu'elle pièce
330   *

```

```

331  * @tparam MOVE Le type de mouvement utiliser entre normal
332  * et split (split ne prend pas les mouvements de capture de pièce)
333  * @tparam N Le nombre de ligne du plateau
334  * @tparam M Le nombre de colonne du plateau
335  * @param[in] board Le plateau
336  * @param[in] pos La position de la pièce
337  * @param[in] list_move La liste de tout les mouvements possible
338  * @return std::forward_list<Coord> La liste de coordonnées
339  * des positions d'arrivées possible
340  */
341  template <Move_Mode MOVE, std::size_t N, std::size_t M>
342  CONSTEXPR std::forward_list<Coord>
343  get_list_move(Board<N, M> const &board,
344              Coord const &pos) const noexcept;
345
346  /**
347   * @brief La couleur de la pièce (WHITE/BLACK)
348   */
349  Color m_color;
350
351  /**
352   * @brief Le type de la piece
353   * ( KING / QUEEN / ROOK / BISHOP / KNIGHT / PAWN )
354   */
355  TypePiece m_type;
356  };
357
358  #include "Piece.hpp"
359  #include "get_list_move.hpp"
360

```

```

361 /**
362  * @brief L'objet représentant le roi blanc
363  */
364 constexpr Piece W_KING{TypePiece::KING, Color::WHITE};
365
366 /**
367  * @brief L'objet représentant le roi noir
368  */
369 constexpr Piece B_KING{TypePiece::KING, Color::BLACK};
370
371 /**
372  * @brief L'objet représentant la renne blanche
373  */
374 constexpr Piece W_QUEEN{TypePiece::QUEEN, Color::WHITE};
375
376 /**
377  * @brief L'objet représentant la renne noire
378  */
379 constexpr Piece B_QUEEN{TypePiece::QUEEN, Color::BLACK};
380
381 /**
382  * @brief L'objet représentant le fou blanc
383  */
384 constexpr Piece W_BISHOP{TypePiece::BISHOP, Color::WHITE};
385
386 /**
387  * @brief L'objet représentant le fou noir
388  */
389 constexpr Piece B_BISHOP{TypePiece::BISHOP, Color::BLACK};
390

```

```

391 /**
392  * @brief L'objet représentant la tour blanche
393  */
394 constexpr Piece W_ROOK{TypePiece::ROOK, Color::WHITE};
395
396 /**
397  * @brief L'objet représentant la tour noire
398  */
399 constexpr Piece B_ROOK{TypePiece::ROOK, Color::BLACK};
400
401 /**
402  * @brief L'objet représentant le cavalier blanc
403  */
404 constexpr Piece W_KNIGHT{TypePiece::KNIGHT, Color::WHITE};
405
406 /**
407  * @brief L'objet représentant le cavalier noir
408  */
409 constexpr Piece B_KNIGHT{TypePiece::KNIGHT, Color::BLACK};
410
411 /**
412  * @brief L'objet représentant le pion blanc
413  */
414 constexpr Piece W_PAWN{TypePiece::PAWN, Color::WHITE};
415
416 /**
417  * @brief L'objet représentant le pion noir
418  */
419 constexpr Piece B_PAWN{TypePiece::PAWN, Color::BLACK};
420
421

```

# Piece.hpp

```
1  #include "Piece.hpp"
2  #include <Coord.hpp>
3  #include <check_path.hpp>
4  #include <cmath>
5  #include <forward_list>
6  #include <observer_ptr.hpp>
7  #include <math_utility.hpp>
8  #include <Constexpr.hpp>
9  #include <Move.hpp>
10
11 #define POW2(x) ((x) * (x))
12
13 constexpr inline Piece::Piece() noexcept
14     : m_color(Color::BLACK),
15       m_type(TypePiece::EMPTY)
16 {
17 }
18
19 constexpr inline Piece::Piece(TypePiece piece, Color color) noexcept
20     : m_color(color),
21       m_type(piece)
22 {
23 }
24
25 constexpr inline bool Piece::is_white() const noexcept
26 {
27     return m_color == Color::WHITE;
28 }
29
30 constexpr inline bool Piece::is_black() const noexcept
```

```

31 {
32     return m_color == Color::BLACK;
33 }
34
35 constexpr inline bool
36 Piece::same_color(Piece const &other) const noexcept
37 {
38     return m_color == other.m_color;
39 }
40
41 constexpr inline TypePiece Piece::get_type() const noexcept
42 {
43     return m_type;
44 }
45
46 constexpr inline Color Piece::get_color() const noexcept
47 {
48     return m_color;
49 }
50
51 constexpr inline std::size_t
52 Piece::abs_substracte(std::size_t x, std::size_t y) noexcept
53 {
54     if (x >= y)
55     {
56         return x - y;
57     }
58     return y - x;
59 }
60

```



```

61 constexpr inline double
62 Piece::norm(Coord const &x, Coord const &y) noexcept
63 {
64     return sqrt(
65         static_cast<double>(POW2(abs_substracte(x.n, y.n)) +
66                               POW2(abs_substracte(x.m, y.m))));
67 }
68
69 template <std::size_t N, std::size_t M>
70 CONSTEXPR bool
71 Piece::check_if_use_move_promote(Board<N, M> const &board,
72                                   Coord const &pos) const noexcept
73 {
74     if constexpr (N >= 2)
75     {
76         if (board(pos.n, pos.m).get_type() == TypePiece::PAWN)
77         {
78             auto sign_color{
79                 [](Color color) -> int
80                 {
81                     return (color == Color::WHITE) ? -1 : 1;
82                 }
83             };
84             std::size_t required_line{(get_color() == Color::WHITE) ? 1 : N - 2};
85             if (pos.n == required_line)
86             {
87                 return true;
88             }
89         }
90     }
91     return false;

```

# get-list-move.hpp

```
1  #include "Piece.hpp"
2  #include <Coord.hpp>
3  #include <check_path.hpp>
4  #include <cmath>
5  #include <forward_list>
6  #include <observer_ptr.hpp>
7  #include <math_utility.hpp>
8  #include <Constexpr.hpp>
9  #include <Move.hpp>
10
11 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
12 constexpr std::forward_list<Coord>
13 Piece::get_list_move_king(Board<N, M> const &board,
14                          Coord const &pos) const noexcept
15 {
16     constexpr int P{M + 2};
17     std::array<int, 8> const
18         move_king{{-P - 1, -P, -P + 1, -1, 1, P - 1, P, P + 1}};
19     std::size_t current_pos{board.offset(pos.n, pos.m)};
20     int posBox{board.m_S_mailbox[current_pos]};
21     std::forward_list<Coord> list_move{};
22     for (int const e : move_king)
23     {
24         int arv{board.m_L_mailbox[posBox + e]};
25         if (arv >= 0)
26         {
27             std::size_t n{arv / M}, m{arv % M};
28             Piece const &arvPiece{board(n, m)};
29             bool eval{false};
30             if constexpr (MOVE == Move_Mode::NORMAL)
```

```

31     {
32         eval = arvPiece.get_type() == TypePiece::EMPTY ||
33             !same_color(arvPiece) ||
34             board.get_proba(Coord(n, m)) < 1. - EPSILON;
35     }
36     #if !defined(KING_NO_SPLIT)
37         else
38         {
39             eval = arvPiece.get_type() == TypePiece::EMPTY;
40         }
41     #endif
42     if (eval)
43     {
44         list_move.push_front(Coord(n, m));
45     }
46 }
47 }
48 if CONSTEXPR (N == 8 && M == 8 && MOVE == Move_Mode::NORMAL)
49 {
50     if (board.m_q_castle[static_cast<int>(get_color())] &&
51         check_path_straight(board, pos, Coord(pos.n, pos.m - 4)))
52     {
53         list_move.push_front(Coord(pos.n, pos.m - 2));
54     }
55     if (board.m_k_castle[static_cast<int>(get_color())] &&
56         check_path_straight(board, pos, Coord(pos.n, pos.m + 3)))
57     {
58         list_move.push_front(Coord(pos.n, pos.m + 2));
59     }
60 }

```

```

61     return list_move;
62 }
63
64 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
65 CONSTEXPR std::forward_list<Coord>
66 Piece::get_list_move_knight(Board<N, M> const &board,
67                             Coord const &pos) const noexcept
68 {
69     CONSTEXPR int P{M + 2};
70     std::array<int, 8> const
71         move_knight{{-2 * P - 1, -P - 2, -2 * P + 1, -P + 2,
72                     P - 2, 2 * P - 1, 2 * P + 1, P + 2}};
73     std::size_t current_pos{board.offset(pos.n, pos.m)};
74     int posBox{board.m_S_mailbox[current_pos]};
75     std::forward_list<Coord> list_move{};
76     for (int const e : move_knight)
77     {
78         int arv{board.m_L_mailbox[posBox + e]};
79         if (arv >= 0)
80         {
81             std::size_t n{arv / M}, m{arv % M};
82             Piece const &arvPiece{board(n, m)};
83             bool eval;
84             if CONSTEXPR (MOVE == Move_Mode::NORMAL)
85             {
86                 eval = arvPiece.get_type() == TypePiece::EMPTY ||
87                     !same_color(arvPiece) ||
88                     board.get_proba(Coord(n, m)) < 1. - EPSILON;
89             }
90             else

```

```

91     {
92         eval = arvPiece.get_type() == TypePiece::EMPTY;
93     }
94     if (eval)
95     {
96         list_move.push_front(Coord(n, m));
97     }
98 }
99 }
100 return list_move;
101 }
102
103 template <Piece::Move_Mode MOVE,
104           std::size_t N,
105           std::size_t M,
106           std::size_t SIZE>
107 CONSTEXPR std::forward_list<Coord>
108 Piece::get_list_move_rec(
109     Board<N, M> const &board,
110     Coord const &pos,
111     std::array<int, SIZE> const &list_move) const noexcept
112 {
113     std::size_t current_pos{board.offset(pos.n, pos.m)};
114     int const posBox{board.m_S_mailbox[current_pos]};
115     std::forward_list<Coord> move{};
116     for (int const e : list_move)
117     {
118         int arv{board.m_L_mailbox[posBox + e]};
119         int posBox_tmp{posBox};
120         while (arv >= 0)

```

```

121 {
122     std::size_t n{arv / M}, m{arv % M};
123     Piece const &arvPiece{board(n, m)};
124     if CONSTEXPR (MOVE == Move_Mode::NORMAL)
125     {
126         if (arvPiece.get_type() == TypePiece::EMPTY ||
127             board.get_proba(Coord(n, m)) < 1. - EPSILON)
128         {
129             move.push_front(Coord(n, m));
130         }
131         else if (!same_color(arvPiece))
132         {
133             move.push_front(Coord(n, m));
134             break;
135         }
136         else
137         {
138             break;
139         }
140     }
141     else
142     {
143         if (arvPiece.get_type() == TypePiece::EMPTY ||
144             (arvPiece.get_type() == board(pos.n, pos.m).get_type() &&
145              arvPiece.get_color() == board(pos.n, pos.m).get_color()))
146         {
147             move.push_front(Coord(n, m));
148         }
149         else
150         {

```

```

151         break;
152     }
153 }
154 posBox_tmp = board.m_S_mailbox[arv];
155 arv = board.m_L_mailbox[posBox_tmp + e];
156 }
157 }
158 return move;
159 }
160
161 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
162 constexpr std::forward_list<Coord>
163 Piece::get_list_move_bishop(Board<N, M> const &board,
164                             Coord const &pos) const noexcept
165 {
166     constexpr int P{M + 2};
167     std::array<int, 4> const move_bishop{{-P - 1, -P + 1, P - 1, P + 1}};
168     return get_list_move_rec<MOVE>(board, pos, move_bishop);
169 }
170
171 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
172 constexpr std::forward_list<Coord>
173 Piece::get_list_move_rook(Board<N, M> const &board,
174                           Coord const &pos) const noexcept
175 {
176     constexpr int P{M + 2};
177     std::array<int, 4> const move_rook{{-P, 1, P, -1}};
178     return get_list_move_rec<MOVE>(board, pos, move_rook);
179 }
180

```

```

181 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
182 CONSTEXPR std::forward_list<Coord>
183 Piece::get_list_move_queen(Board<N, M> const &board,
184                           Coord const &pos) const noexcept
185 {
186     CONSTEXPR int P{M + 2};
187     std::array<int, 8> const
188         move_queen{{-P - 1, -P, -P + 1, -1, 1, P - 1, P, P + 1}};
189     return get_list_move_rec<MOVE>(board, pos, move_queen);
190 }
191
192 template <std::size_t N, std::size_t M>
193 CONSTEXPR std::forward_list<Coord>
194 Piece::get_list_move_pawn(Board<N, M> const &board,
195                           Coord const &pos) const noexcept
196 {
197     CONSTEXPR int P{M + 2};
198     auto sign_color{
199         [](Color color)
200         {
201             return (color == Color::WHITE) ? -1 : 1;
202         }
203     };
204     int sign{sign_color(get_color())};
205     std::array<int, 3> move_pawn{sign * P, sign * P - 1, sign * P + 1};
206     std::forward_list<Coord> list_move{};
207     std::size_t current_pos{board.offset(pos.n, pos.m)};
208     int posBox{board.m_S_mailbox[current_pos]};
209     int arv{board.m_L_mailbox[posBox + move_pawn[0]]};
210     if (arv >= 0)
211     {

```



```

211     std::size_t n{arv / N}, m{arv % N};
212     if (board(n, m).get_type() == TypePiece::EMPTY ||
213         board.get_proba(Coord(n, m)) < 1. - EPSILON)
214     {
215         list_move.push_front(Coord(n, m));
216         if ((pos.n == 1 && get_color() == Color::BLACK) ||
217             (pos.n == N - 2 && get_color() == Color::WHITE)) &&
218             (board(n + sign, m).get_type() == TypePiece::EMPTY ||
219              board.get_proba(Coord(n + sign, m)) < 1. - EPSILON))
220         {
221             list_move.push_front(Coord(n + sign, m));
222         }
223     }
224 }
225 for (std::size_t i{1}; i < 3; i++)
226 {
227     arv = board.m_L_mailbox[posBox + move_pawn[i]];
228     if (arv >= 0)
229     {
230         std::size_t n{arv / N}, m{arv % N};
231         if ((board(n, m).get_type() != TypePiece::EMPTY &&
232             !same_color(board(n, m))) ||
233             (board.m_ep != std::nullopt &&
234              board.m_ep->n == pos.n &&
235              board.m_ep->m == pos.m))
236         {
237             list_move.push_front(Coord(n, m));
238         }
239     }
240 }

```

```

241     return list_move;
242 }
243
244 template <std::size_t N, std::size_t M>
245 CONSTEXPR std::forward_list<Move>
246 Piece::get_list_promote(Board<N, M> const &board, Coord const &pos) const noexcept
247 {
248     std::forward_list<Coord> list_move{get_list_move_pawn(board, pos)};
249     std::forward_list<Move> list_promote;
250     std::array<TypePiece, 4> promote_choice{TypePiece::QUEEN,
251                                             TypePiece::ROOK,
252                                             TypePiece::BISHOP,
253                                             TypePiece::KNIGHT};
254     for (Coord const &e : list_move)
255     {
256         for (TypePiece p : promote_choice)
257         {
258             list_promote.push_front(Move_promote(pos, e, p));
259         }
260     }
261     return list_promote;
262 }
263
264 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
265 CONSTEXPR std::forward_list<Coord>
266 Piece::get_list_move(Board<N, M> const &board, Coord const &pos) const noexcept
267 {
268     switch (get_type())
269     {
270     case TypePiece::PAWN:

```

```

271     if CONSTEXPR (MOVE == Move_Mode::NORMAL)
272     {
273         return get_list_move_pawn(board, pos);
274     }
275     else
276     {
277         return std::forward_list<Coord>{};
278     }
279 case TypePiece::KNIGHT:
280     return get_list_move_knight<MOVE>(board, pos);
281 case TypePiece::BISHOP:
282     return get_list_move_bishop<MOVE>(board, pos);
283 case TypePiece::ROOK:
284     return get_list_move_rook<MOVE>(board, pos);
285 case TypePiece::QUEEN:
286     return get_list_move_queen<MOVE>(board, pos);
287 case TypePiece::KING:
288     return get_list_move_king<MOVE>(board, pos);
289 case TypePiece::EMPTY:
290 default:
291     return std::forward_list<Coord>{};
292 }
293 }
294
295 template <std::size_t N, std::size_t M>
296 CONSTEXPR std::forward_list<Coord>
297 Piece::get_list_normal_move(Board<N, M> const &board,
298                             Coord const &pos) const noexcept
299 {
300     return get_list_move<Move_Mode::NORMAL>(board, pos);

```

```
301 }
302
303 template <std::size_t N, std::size_t M>
304 CONSTEXPR std::forward_list<Coord>
305 Piece::get_list_split_move(Board<N, M> const &board,
306                           Coord const &pos) const noexcept
307 {
308     return get_list_move<Move_Mode::SPLIT>(board, pos);
309 }
```

# TypePiece.hpp

```
1  #ifndef TYPE_PIECE_HPP
2  #define TYPE_PIECE_HPP
3
4  /**
5   * @brief Enumère toutes les sortes de pièces
6   */
7  enum class TypePiece
8  {
9      EMPTY,
10     PAWN,
11     KNIGHT,
12     BISHOP,
13     ROOK,
14     QUEEN,
15     KING
16 };
17
18
```

# Coord.hpp

```
1  #ifndef COORD_HPP
2  #define COORD_HPP
3
4  #include <cstddef>
5
6  /**
7   * @brief La structure qui représente une coordonnée
8   * sur le plateau
9   */
10 struct Coord
11 {
12     Coord() = default;
13     Coord(std::size_t i, std::size_t j) : n(i), m(j){};
14     Coord(Coord const &) = default;
15     Coord &operator=(Coord const &) = default;
16     Coord(Coord &&) = default;
17     Coord &operator=(Coord &&) = default;
18
19     /**
20      * @brief l'indice sur les lignes
21      */
22     std::size_t n;
23
24     /**
25      * @brief l'indice sur les colonnes
26      */
27     std::size_t m;
28 };
29
30 struct Coord_hash
```

```
31 {  
32     std::size_t operator()(Coord const &coord) const;  
33 };  
34  
35 bool operator==(Coord const &lhs, Coord const &rhs);  
36  
37
```

# Coord.cpp

```
1  #include "Coord.hpp"
2  #include <functional>
3
4  std::size_t Coord_hash::operator()(Coord const &coord) const
5  {
6      std::size_t h1 = std::hash<std::size_t>()(coord.n);
7      std::size_t h2 = std::hash<std::size_t>()(coord.m);
8
9      return h1 ^ h2;
10 }
11
12 bool operator==(Coord const &lhs, Coord const &rhs)
13 {
14     return lhs.n == rhs.n && lhs.m == rhs.m;
15 }
```



# Move.hpp

```
1  #ifndef MOVE_HPP
2  #define MOVE_HPP
3
4  #include <Coord.hpp>
5  #include <TypePiece.hpp>
6
7  /**
8   * @brief Énumération représentant tout les types de mouvements
9   */
10 enum class TypeMove
11 {
12     /**
13      * @brief Représente le mouvement classic
14      */
15     NORMAL,
16
17     /**
18      * @brief Représente le mouvement de split de deux pièces
19      */
20     SPLIT,
21
22     /**
23      * @brief Représente le mouvement de fusion de deux pièces
24      */
25     MERGE,
26
27     /**
28      * @brief Représente le mouvement de promotion du pion
29      */
30 }
```

```

31  PROMOTE
32 };
33
34 /**
35  * @brief Stoque tout les mouvements possibles
36  */
37 struct Move
38 {
39     Move() = default;
40     Move(Move const &) = default;
41     Move &operator=(Move const &) = default;
42     Move(Move &&) = default;
43     Move &operator=(Move &&) = default;
44     /**
45      * @brief Indique quel est le mouvement stoqué par l'union
46      */
47     TypeMove type;
48
49     /**
50      * @brief Stoque au choix l'un des trois mouvements possible
51      */
52     union
53     {
54         /**
55          * @brief Stoque les coordonnées pour un mouvement classic
56          */
57         struct
58         {
59             /**
60              * @brief La coordonnée de depart

```

```

61     */
62     Coord src;
63
64     /**
65      * @brief La coordonnées d'arrivée
66      */
67     Coord arv;
68 } normal;
69
70 /**
71  * @brief stocke l'ensemble des coordonnées pour un
72  * mouvement splité
73  */
74 struct
75 {
76     /**
77      * @brief La coordonnée de depart
78      */
79     Coord src;
80
81     /**
82      * @brief Une coordonnée d'arrivée
83      */
84     Coord arv1;
85
86     /**
87      * @brief Une seconde coordonnée d'arrivée
88      *
89      */
90     Coord arv2;

```

```

91     } split;
92
93     /**
94      * @brief stocke l'ensemble des coordonnées pour un
95      * mouvement de fusion
96      */
97     struct
98     {
99         /**
100          * @brief La coordonnée de la première pièce à fusionner
101          */
102         Coord src1;
103
104         /**
105          * @brief La coordonnée de la seconde pièce à fusionner
106          */
107         Coord src2;
108
109         /**
110          * @brief La coordonnée de la position d'arrivée
111          */
112         Coord arv;
113     } merge;
114
115     /**
116      * @brief stocke l'ensemble des coordonnées et la pièce choisi
117      * pour un mouvement de promotion
118      */
119     struct
120     {

```

```

121     /**
122      * @brief La case de depart
123      */
124     Coord src;
125
126     /**
127      * @brief La case d'arrivée
128      */
129     Coord arv;
130
131     /**
132      * @brief La piece choisi pour le move
133      */
134     TypePiece piece;
135 } promote;
136 };
137
138 bool operator==(Move const &m) const noexcept;
139 };
140
141 constexpr inline Move Move_classic(Coord const &src, Coord const &arv)
142 {
143     Move move;
144     move.type = TypeMove::NORMAL;
145     move.normal.src = src;
146     move.normal.arv = arv;
147     return move;
148 }
149
150 constexpr inline Move Move_split(Coord const &src, Coord const &arv1, Coord const &arv2)

```

```

151 {
152     Move move;
153     move.type = TypeMove::SPLIT;
154     move.split.src = src;
155     move.split.arv1 = arv1;
156     move.split.arv2 = arv2;
157     return move;
158 }
159
160 constexpr inline Move Move_merge(Coord const &src1, Coord const &src2, Coord const &arv)
161 {
162     Move move;
163     move.type = TypeMove::MERGE;
164     move.merge.src1 = src1;
165     move.merge.src2 = src2;
166     move.merge.arv = arv;
167     return move;
168 }
169
170 constexpr inline Move Move_promote(Coord const &src, Coord const &arv, TypePiece promotion)
171 {
172     Move move;
173     move.type = TypeMove::PROMOTE;
174     move.promote.src = src;
175     move.promote.arv = arv;
176     move.promote.piece = promotion;
177     return move;
178 }
179
180

```

# Move.cpp

```
1  #include <Coord.hpp>
2  #include <TypePiece.hpp>
3  #include "Move.hpp"
4
5  bool Move::operator==(Move const &m) const noexcept
6  {
7      if (type == m.type)
8      {
9          switch (type)
10         {
11             case TypeMove::NORMAL:
12                 return normal.src == m.normal.src &&
13                     normal.arv == m.normal.arv;
14             case TypeMove::SPLIT:
15                 return split.src == m.split.src &&
16                     split.arv1 == m.split.arv1 &&
17                     split.arv2 == m.split.arv2;
18             case TypeMove::MERGE:
19                 return merge.src1 == m.merge.src1 &&
20                     merge.src2 == m.merge.src2 &&
21                     merge.arv == m.merge.arv;
22             case TypeMove::PROMOTE:
23                 return promote.src == m.promote.src &&
24                     promote.arv == m.promote.arv &&
25                     promote.piece == m.promote.piece;
26             default:
27                 return false;
28         }
29     }
30     return false;
31 }
```

# Qubit.hpp

```
1  #ifndef QUBIT_HPP
2  #define QUBIT_HPP
3
4  #include <cstddef>
5  #include <complex>
6  #include <array>
7  #include <CMatrix.hpp>
8  #include <Constexpr.hpp>
9
10 /**
11  * @brief Représente un qubit
12  *
13  * @tparam N La taille du Qubit
14  */
15 template <std::size_t N>
16 class Qubit final
17 {
18 public:
19     // Constructeur
20     CONSTEXPR Qubit() = default;
21
22     /**
23     * @brief Construit un nouveau qubit à l'aide d'un tableau de booléen
24     *
25     * @param data le tableau de n booléen utilisé pour
26     * construire un n-qubit ex : /10>
27     */
28     CONSTEXPR Qubit(std::array<bool, N> const &data);
29     CONSTEXPR Qubit(std::array<std::complex<double>, _2POW(N)> &&init_list);
30 }
```



```

31 // Copie
32 CONSTEXPR Qubit(Qubit const &) = delete;
33 CONSTEXPR Qubit &operator=(Qubit const &) = delete;
34
35 // Mouvement
36 CONSTEXPR Qubit(Qubit &&) = delete;
37 CONSTEXPR Qubit &operator=(Qubit &&) = delete;
38
39 // Destructeur
40 CONSTEXPR ~Qubit() = default;
41
42 template <std::size_t M>
43 friend CONSTEXPR Qubit<M> operator*(
44     CMatrix<_2POW(M)> const &lhs,
45     Qubit<M> const &rhs);
46
47 template <std::size_t M>
48 friend std::ostream &operator<<(
49     std::ostream &out,
50     Qubit<M> const &qubit);
51
52 template <std::size_t M>
53 friend CONSTEXPR std::array<
54     std::pair<
55         std::array<bool, M>,
56         std::complex<double>>>,
57     2>
58     qubitToArray(Qubit<M> const &qubit);
59
60 private:

```

```

61  std::array<std::complex<double>, _2POW(N)> m_data;
62  };
63
64  /**
65   * @brief Transforme un qubit dans sa représentation sous forme de
66   * vecteur en sa représentation classique.
67   * Si le qubit contient deux cases non nuls, c'est une combinaison
68   * linéaires de deux qubits que l'on peut écrire avec la forme classique,
69   * c'est-à-dire avec des "0" et des "1".
70   * @warning Cette fonction ne permet de faire la transformation que
71   * pour au plus deux cases non nuls dans le qubits
72   * @tparam N La taille du qubit
73   * @param[in] qubit Le qubit à transformer
74   * @return Dans chaque cases du tableau, on a le qubits
75   * sous sa représentation classique que l'on modélise par
76   * un tableau de booléen, et un complexe qui est la scalaire.
77   */
78  template <std::size_t N>
79  CONSTEXPR std::array<std::pair<std::array<bool, N>, std::complex<double>>>, 2>
80  qubitToArray(Qubit<N> const &qubit);
81
82  /**
83   * @brief Opérateur du produit entre un qubit et une matrice
84   * @warning le produit n'est pas associatif
85   * @tparam M La taille du qubit, la matrice est de taille  $2^M$ 
86   * @param[in] lhs La matrice carré complexe de taille  $2^M$ 
87   * @param[in] rhs Le M-qubit
88   * @return Qubit<M> renvoie le qubit résultant du produit
89   */
90  template <std::size_t N>

```

```

91 CONSTEXPR Qubit<N> operator*(
92     CMatrix<_2POW(N)> const &lhs,
93     Qubit<N> const &rhs);
94
95 /**
96  * @brief Fonction d'affichage des qubit sous forme de
97  * liste de nombre complexe
98  *
99  * @tparam M La taille du qubit
100  * @param[out] out Le flux de destination
101  * @param[in] qubit Le qubit à afficher
102  * @return std::ostream& le flux de destination
103  */
104 template <std::size_t N>
105 std::ostream &operator<<(std::ostream &out, Qubit<N> const &qubit);
106
107 #include "Qubit.hpp"
108
109

```

# Qubit.hpp

```
1  #include "Qubit.hpp"
2  #include <Matrix.hpp>
3  #include <algorithm>
4  #include <numeric>
5  #include <math_utility.hpp>
6  #include <Constexpr.hpp>
7  #include <Coord.hpp>
8
9  template <std::size_t N>
10 CONSTEXPR Qubit<N>::Qubit(
11     std::array<std::complex<double>, _2POW(N)> &&init_list)
12     : m_data(std::move(init_list))
13 {
14 }
15
16 template <std::size_t N>
17 CONSTEXPR Qubit<N>::Qubit(std::array<bool, N> const &data) : m_data()
18 {
19
20     auto sum{[] (std::array<bool, N> const &tab) -> int
21         {
22             int acc{0};
23             for (std::size_t i{0}; i < std::size(tab); i++)
24             {
25                 acc += _2POW(i) * tab[std::size(tab) - i - 1];
26             }
27             return acc;
28         }};
29     m_data[sum(data)] = 1;
30 }
```

```

31
32 template <std::size_t N>
33 constexpr Qubit<N> operator*(
34     CMatrix<_2POW(N)> const &lhs,
35     Qubit<N> const &rhs)
36 {
37     std::array<std::complex<double>, _2POW(N)> tab{};
38     for (std::size_t i{0}; i < lhs.numberLines(); i++)
39     {
40         for (std::size_t j{0}; j < lhs.numberColumns(); j++)
41         {
42             tab[i] += lhs(i, j) * rhs.m_data[j];
43         }
44     }
45
46     return Qubit<N>(std::move(tab));
47 }
48
49 template <std::size_t N>
50 std::ostream &operator<<(
51     std::ostream &out,
52     Qubit<N> const &qubit)
53 {
54     for (std::size_t i{0}; i < _2POW(N); i++)
55     {
56         out << qubit.m_data[i].real() << "+" << qubit.m_data[i].imag() << "i ";
57     }
58     return out;
59 }
60

```

```

61 template <std::size_t N>
62 constexpr std::array<
63     std::pair<std::array<bool, N>,
64         std::complex<double>>>,
65     2>
66 qubitToArray(Qubit<N> const &qubit)
67 {
68     std::array<
69         std::pair<std::array<bool, N>,
70             std::complex<double>>>,
71         2>
72     tab{};
73     bool b{true};
74     std::size_t compteur{N};
75     std::size_t pow = _2POW(N);
76     for (std::size_t i{0}; i < pow; i++)
77     {
78         using namespace std::complex_literals;
79         if (complex_equal(qubit.m_data[i], 0i))
80         {
81             if (i == pow - 1)
82             {
83                 return tab;
84             }
85             else
86             {
87                 if (b)
88                 {
89                     while (tab[1].first[compteur - 1])
90                     {

```

```

91         tab[0].first[compteur - 1] = false;
92         tab[1].first[compteur - 1] = false;
93         compteur--;
94     }
95     tab[0].first[compteur - 1] = true;
96     tab[1].first[compteur - 1] = true;
97     compteur = N;
98 }
99 else
100 {
101     while (tab[1].first[compteur - 1])
102     {
103         tab[1].first[compteur - 1] = false;
104         compteur--;
105     }
106     tab[1].first[compteur - 1] = true;
107     compteur = N;
108 }
109 }
110 }
111 else
112 {
113     if (b)
114     {
115         tab[0].second = qubit.m_data[i];
116         b = false;
117         if (i != pow - 1)
118         {
119             while (tab[1].first[compteur - 1])
120             {

```

```
121         tab[1].first[compteur - 1] = false;
122         compteur--;
123     }
124     tab[1].first[compteur - 1] = true;
125     compteur = N;
126 }
127 }
128 else
129 {
130     tab[1].second = qubit.m_data[i];
131     return tab;
132 }
133 }
134 }
135 return tab;
136
```



# Random.hpp

```
1  #ifndef RANDOM_HPP
2  #define RANDOM_HPP
3
4  namespace rnd
5  {
6      /**
7       * @brief Renvoie un entier dans l'intervale [a, b]
8       *
9       * @param a, b Entier pour spécifier la plage
10      * @return int Un entier entre a et b inclu
11      */
12      int randint(int a, int b);
13
14      /**
15       * @brief Renvoie un réel dans l'intervale [a, b)
16       *
17       * @param a, b Réel pour spécifier la plage
18       * @return double Un réel entre a et b inclu
19       */
20      double randreal(double a, double b);
21  }
22
23
24
```

# Random.cpp

```
1  #include <random>
2  #include "Random.hpp"
3  #include <iostream>
4  namespace
5  {
6      std::random_device rd;
7  }
8
9  namespace rnd
10 {
11     int randint(int a, int b)
12     {
13         static std::uniform_int_distribution<> distrib(a, b);
14         return distrib(rd);
15     }
16
17     double randreal(double a, double b)
18     {
19         static std::uniform_real_distribution<> distrib(a, b);
20         return distrib(rd);
21     }
22 }
```

# math-utility.hpp

```
1  #ifndef MATH_UTILITY_HPP
2  #define MATH_UTILITY_HPP
3
4  #define EPSILON 10e-3
5
6  #include <complex>
7  #include <Constepr.hpp>
8
9  namespace
10 {
11     CONSTEXPR bool double_equal(double x, double y);
12     CONSTEXPR bool complex_equal(
13         std::complex<double> const &z1,
14         std::complex<double> const &z2);
15 }
16
17 #include "math_utility.hpp"
18
```

# math-utility.hpp

```
1 #include <cmath>
2 #include <complex>
3 #include <Constexpr.hpp>
4 #include "math_utility.hpp"
5
6 namespace
7 {
8     CONSTEXPR bool double_equal(double x, double y)
9     {
10         return std::abs(x - y) <= EPSILON;
11     }
12
13     CONSTEXPR bool complex_equal(
14         std::complex<double> const &z1,
15         std::complex<double> const &z2)
16     {
17         return double_equal(z1.real(), z2.real()) &&
18             double_equal(z1.imag(), z2.imag());
19     }
20 }
```

# check-path.hpp

```
1  #ifndef CHECK_PATH_HPP
2  #define CHECK_PATH_HPP
3
4  #include <Coord.hpp>
5  #include <Board.hpp>
6  #include <Constexpr.hpp>
7
8  template <std::size_t N, std::size_t M>
9  CONSTEXPR static bool
10 check_path_straight(
11     Board<N, M> const &board,
12     Coord const &dpt,
13     Coord const &arv) noexcept;
14
15 template <std::size_t N, std::size_t M>
16 CONSTEXPR static bool
17 check_path_diagonal(
18     Board<N, M> const &board,
19     Coord const &dpt,
20     Coord const &arv) noexcept;
21
22 /**
23  * @brief Vérifie si il y a une pièce entre deux cases sur une instance
24  * du plateau pour un mouvement orthogonale
25  *
26  * @tparam N Le nombre de ligne du plateau
27  * @tparam M Le nombre de colonnes du plateau
28  * @param[in] board Le plateau
29  * @param[in] dpt Les coordonnées de la case de départ
30  * @param[in] arv Les coordonnées de la case d'arrivée
```

```

31 * @param[in] position L'instance du plateau
32 * @return true si le mouvement est possible
33 * @return false si le mouvement est bloqué,
34 * on si ce n'est pas un mouvementorthogonal
35 */
36 template <std::size_t N, std::size_t M>
37 CONSTEXPR bool
38 check_path_straight_1_instance(
39     Board<N, M> const &board,
40     Coord const &dpt,
41     Coord const &arv,
42     std::size_t position,
43     std::optional<Coord> position_other_piece_merge);
44
45 /**
46  * @brief Vérifie si il y a une pièce entre deux cases sur une
47  * instance du plateau pour un mouvement diagonale.
48  *
49  * @tparam N Le nombre de ligne du plateau
50  * @tparam M Le nombre de colonnes du plateau
51  * @param[in] board Le plateau
52  * @param[in] dpt Les coordonnées de la case de départ
53  * @param[in] arv Les coordonnées de la case d'arrivée
54  * @param[in] position L'instance du plateau
55  * @return true si le mouvement est possible
56  * @return false si le mouvement est bloqué, on si ce n'est pas
57  * un mouvement diagonale.
58  */
59 template <std::size_t N, std::size_t M>
60 CONSTEXPR bool check_path_diagonal_1_instance(

```

```

61     Board<N, M> const &board,
62     Coord const &dpt,
63     Coord const &arv,
64     std::size_t position,
65     std::optional<Coord> position_other_piece_merge);
66
67 /**
68  * @brief Vérifie si il y a une pièce entre deux cases
69  * sur une instance du plateau pour un mouvement orthogonal ou diagonale.
70  *
71  * @tparam N Le nombre de ligne du plateau
72  * @tparam M Le nombre de colonnes du plateau
73  * @param[in] board Le plateau
74  * @param[in] dpt Les coordonnées de la case de départ
75  * @param[in] arv Les coordonnées de la case d'arrivée
76  * @param[in] position L'instance du plateau
77  * @return true si le mouvement est possible
78  * @return false si le mouvement est bloqué, on si ce n'est pas
79  * un mouvement orthogonal ou diagonale.
80  */
81 template <std::size_t N, std::size_t M>
82 CONSTEXPR bool check_path_queen_1_instance(
83     Board<N, M> const &board,
84     Coord const &dpt,
85     Coord const &arv,
86     std::size_t position,
87     std::optional<Coord> position_other_piece_merge);
88
89 #include "check_path.tpp"
90

```

# check-path.hpp

```
1  #include <algorithm>
2  #include <Board.hpp>
3  #include <Coord.hpp>
4  #include <Constexpr.hpp>
5  #include "check_path.hpp"
6
7  template <std::size_t N, std::size_t M>
8  CONSTEXPR bool
9  check_path_straight(
10     Board<N, M> const &board,
11     Coord const &dpt,
12     Coord const &arv) noexcept
13  {
14     if (dpt.n == arv.n)
15     {
16         for (std::size_t i{std::min(dpt.m, arv.m) + 1};
17              i < std::max(dpt.m, arv.m);
18              i++)
19         {
20             if (board(dpt.n, i).get_type() != TypePiece::EMPTY ||
21                 board.get_proba(Coord(dpt.n, i)) < 1.)
22             {
23                 return false;
24             }
25         }
26         return true;
27     }
28     else if (dpt.m == arv.m)
29     {
30         for (std::size_t i{std::min(dpt.n, arv.n) + 1};
```



```

31         i < std::max(dpt.n, arv.n);
32         i++)
33     {
34         if (board(i, dpt.m).get_type() != TypePiece::EMPTY ||
35             board.get_proba(Coord(i, dpt.m)) < 1.)
36         {
37             return false;
38         }
39     }
40     return true;
41 }
42 return false;
43 }
44
45 template <std::size_t N, std::size_t M>
46 CONSTEXPR bool
47 check_path_diagonal(
48     Board<N, M> const &board,
49     Coord const &dpt,
50     Coord const &arv) noexcept
51 {
52     std::size_t const max_lines{std::max(dpt.n, arv.n)};
53     std::size_t const min_lines{std::min(dpt.n, arv.n)};
54     std::size_t const max_columns{std::max(dpt.m, arv.m)};
55     std::size_t const min_columns{std::min(dpt.m, arv.m)};
56
57     std::size_t const dist{max_lines - min_lines};
58
59     if (dist == max_columns - min_columns)
60     {

```

```

61     for (std::size_t i{1}; i < dist; i++)
62     {
63         if (board(min_lines + i, min_columns + i).get_type() != TypePiece::EMPTY)
64         {
65             return false;
66         }
67     }
68     return true;
69 }
70 return false;
71 }
72
73 template <std::size_t N, std::size_t M>
74 CONSTEXPR bool
75 check_path_straight_1_instance(
76     Board<N, M> const &board,
77     Coord const &dpt,
78     Coord const &arv,
79     std::size_t position,
80     std::optional<Coord> position_other_piece_merge)
81 {
82     if (dpt.n == arv.n)
83     {
84         for (std::size_t i{std::min(dpt.m, arv.m) + 1};
85             i < std::max(dpt.m, arv.m);
86             i++)
87         {
88             if (board.m_board[position].first[board.offset(dpt.n, i)])
89             {
90                 if (position_other_piece_merge.has_value())

```

```

91     {
92         if (!(board.offset(position_other_piece_merge.value().n,
93                             position_other_piece_merge.value().m) == board.offset(dpt.n, i)))
94             {
95                 return false;
96             }
97     }
98     else
99     {
100         return false;
101     }
102 }
103 }
104 return true;
105 }
106 else if (dpt.m == arv.m)
107 {
108     for (std::size_t i{std::min(dpt.n, arv.n) + 1};
109         i < std::max(dpt.n, arv.n);
110         i++)
111     {
112         if (board.m_board[position].first[board.offset(i, dpt.m)])
113         {
114             if (position_other_piece_merge.has_value())
115             {
116                 if (!(board.offset(position_other_piece_merge.value().n,
117                                     position_other_piece_merge.value().m) == board.offset(i, dpt.m)))
118                 {
119                     return false;
120                 }

```

```

121     }
122     else
123     {
124         return false;
125     }
126 }
127 }
128 return true;
129 }
130 return false;
131 }
132
133 template <std::size_t N, std::size_t M>
134 CONSTEXPR bool
135 check_path_diagonal_1_instance(
136     Board<N, M> const &board,
137     Coord const &dpt,
138     Coord const &arv,
139     std::size_t position,
140     std::optional<Coord> position_other_piece_merge)
141 {
142     std::size_t const max_lines{std::max(dpt.n, arv.n)};
143     std::size_t const min_lines{std::min(dpt.n, arv.n)};
144     std::size_t const max_columns{std::max(dpt.m, arv.m)};
145     std::size_t const min_columns{std::min(dpt.m, arv.m)};
146
147     std::size_t const dist{max_lines - min_lines};
148
149     if (dist == max_columns - min_columns)
150     {

```

```

151     for (std::size_t i{1}; i < dist; i++)
152     {
153         if (board.m_board[position]
154             .first[board.offset(min_lines + i, min_columns + i)])
155         {
156             if (position_other_piece_merge.has_value())
157             {
158                 if (!(board.offset(position_other_piece_merge.value().n,
159                                     position_other_piece_merge.value().m) ==
160                     board.offset(min_lines + i,
161                                   min_columns + i)))
162                 {
163                     return false;
164                 }
165             }
166             else
167             {
168                 return false;
169             }
170         }
171     }
172     return true;
173 }
174 return false;
175 }
176
177 template <std::size_t N, std::size_t M>
178 constexpr bool check_path_queen_1_instance(
179     Board<N, M> const &board,
180     Coord const &dpt,

```

```
181     Coord const &arv,  
182     std::size_t position,  
183     std::optional<Coord> position_other_piece_merge)  
184 {  
185     return check_path_straight_1_instance<N, M>(board, dpt, arv, position, position_other_piece_merge) ||  
186         check_path_diagonal_1_instance<N, M>(board, dpt, arv, position, position_other_piece_merge);  
187 }
```

# Unitary.hpp

```
1  #ifndef UNITARY_HPP
2  #define UNITARY_HPP
3  #include <CMatrix.hpp>
4  #include <numbers>
5  #include <complex>
6  #include <Constexpr.hpp>
7
8  using namespace std::complex_literals;
9  using namespace std::numbers;
10
11  constexpr inline
12  CMatrix<4> MATRIX_ISWAP {
13      {
14          {1, 0, 0, 0},
15          {0, 0, 1i, 0},
16          {0, 1i, 0, 0},
17          {0, 0, 0, 1}
18      }
19  };
20
21
22  constexpr inline
23  CMatrix<4> MATRIX_SQRT_ISWAP {
24      {
25          {1, 0, 0, 0},
26          {0, 1./sqrt2, 1i/sqrt2, 0},
27          {0, 1i/sqrt2, 1./sqrt2, 0},
28          {0, 0, 0, 1}
29      }
30  };
```

```

31
32 constexpr inline
33 CMatrix<4> MATRIX_JUMP { MATRIX_ISWAP };
34
35 constexpr inline
36 CMatrix<8> MATRIX_SLIDE {
37     {
38         {1, 0, 0, 0, 0, 0, 0, 0},
39         {0, 0, 1i, 0, 0, 0, 0, 0 },
40         {0, 1i, 0, 0, 0, 0, 0, 0},
41         {0, 0, 0, 1, 0, 0, 0, 0},
42         {0, 0, 0, 0, 1, 0, 0, 0 },
43         {0, 0, 0, 0, 0, 1, 0, 0},
44         {0, 0, 0, 0, 0, 0, 1, 0},
45         {0, 0, 0, 0, 0, 0, 0, 1}
46     }
47 };
48
49 constexpr inline
50 CMatrix<8> MATRIX_ISWAP_8 {
51     {
52         {1, 0, 0, 0, 0, 0, 0, 0},
53         {0, 0, 0, 0, 1i, 0, 0, 0 },
54         {0, 0, 1, 0, 0, 0, 0, 0},
55         {0, 0, 0, 0, 0, 0, 1i, 0},
56         {0, 1i, 0, 0, 0, 0, 0, 0 },// erreur dans le papier?
57         {0, 0, 0, 0, 0, 1, 0, 0},
58         {0, 0, 0, 1i, 0, 0, 0, 0},
59         {0, 0, 0, 0, 0, 0, 0, 1}
60     }

```



```

61 };
62
63 constexpr inline
64 CMatrix<8> MATRIX_SQRT_ISWAP_8 {
65     {
66         {1,      0,      0, 0, 0,      0,      0, 0},
67         {0, 1./sqrt2, 1i/sqrt2, 0, 0,      0,      0, 0},
68         {0, 1i/sqrt2, 1./sqrt2, 0, 0,      0,      0, 0},
69         {0,      0,      0, 1, 0,      0,      0, 0},
70         {0,      0,      0, 0, 1,      0,      0, 0},
71         {0,      0,      0, 0, 0, 1./sqrt2, 1i/sqrt2, 0},
72         {0,      0,      0, 0, 0, 1i/sqrt2, 1./sqrt2, 0},
73         {0,      0,      0, 0, 0,      0,      0, 1}
74     }
75 };
76
77 constexpr inline
78 CMatrix<8> MATRIX_SPLIT {
79     {
80         {1,      0,      0, 0, 0,      0,      0, 0},
81         {0,      0,      0, 0, 1i,      0,      0, 0},
82         {0, 1i/sqrt2, 1./sqrt2, 0, 0,      0,      0, 0},
83         {0,      0,      0, 0, 0, -1./sqrt2, 1i/sqrt2, 0},
84         {0, 1i/sqrt2, -1./sqrt2, 0, 0,      0,      0, 0},
85         {0,      0,      0, 0, 0, 1i/sqrt2, -1/sqrt2, 0},
86         {0,      0,      0, 1i, 0,      0,      0, 0},
87         {0,      0,      0, 0, 0,      0,      0, 1}
88     }
89 };
90

```

```

91 constexpr inline
92 CMatrix<8> MATRIX_MERGE {
93     {
94         {1, 0, 0, 0, 0, 0, 0, 0},
95         {0, 0, -1i/sqrt2, 0, -1i/sqrt2, 0, 0, 0},
96         {0, 0, 1/sqrt2, 0, -1/sqrt2, 0, 0, 0},
97         {0, 0, 0, 0, 0, 0, -1i, 0},
98         {0, -1i, 0, 0, 0, 0, 0, 0},
99         {0, 0, 0, -1/sqrt2, 0, -1i/sqrt2, 0, 0},
100        {0, 0, 0, -1i/sqrt2, 0, -1/sqrt2, 0, 0},
101        {0, 0, 0, 0, 0, 0, 0, 1}
102    }
103 };
104
105 constexpr inline
106 CMatrix<32> MATRIX_SPLIT_SLIDE {
107     CMatrix<16>{
108         MATRIX_SPLIT, CMatrix<8>{},
109         CMatrix<8>{}, MATRIX_ISWAP_8
110     },
111     CMatrix<16>{},
112     CMatrix<2>::identity()
113         .tensoriel_product(MATRIX_ISWAP), CMatrix<8>{},
114     CMatrix<8>{}, CMatrix<8>::identity()
115 }
116 };
117
118 constexpr inline
119 CMatrix<32> MATRIX_MERGE_SLIDE {
120     CMatrix<16>{

```

```

121     MATRIX_MERGE, CMatrix<8>{},
122     CMatrix<8>{}, conj(CMatrix<2>::identity().tensoriel_product(MATRIX_ISWAP).transposed())
123 },
124 CMatrix<16>{},
125 CMatrix<16>{},
126 CMatrix<16>{
127     conj(MATRIX_ISWAP_8.transposed()), CMatrix<8>{},
128     CMatrix<8>{}, CMatrix<8>::identity()
129 }
130 };
131
132
133
134

```

# CMatrix.hpp

```
1  #ifndef CMATRIX_HPP
2  #define CMATRIX_HPP
3  #include <Matrix.hpp>
4  #include <complex>
5  #include <Constexpr.hpp>
6  #include <cstdlib>
7
8  /**
9   * @brief Objet représentant les matrices carrées complexe
10   * de dimension N
11   *
12   * @tparam N La dimension de la matrice
13   */
14 template <std::size_t N>
15 using CMatrix = Matrix<std::complex<double>, N>;
16
17 namespace
18 {
19     template <std::size_t N>
20     CONSTEXPR CMatrix<N> conj(CMatrix<N> const & matrix)
21     {
22         CMatrix<N> matrix_conj{};
23         for(std::size_t i = 0; i<N; i++)
24         {
25             for(std::size_t j = 0; j<N; j++)
26             {
27                 matrix_conj(i, j) = std::conj(matrix(i, j));
28             }
29         }
30         return matrix_conj;
31     }
```

```
31     }
32 }
33
34 /**
35  * @brief Réalise l'opération  $2^n$ 
36  * @warning Est valable uniquement sur les types entiers
37  */
38 #define _2POW(n) (1 << (n))
39
40
```

# Complex-printer.hpp

```
1  #ifndef COMPLEX_PRINTER_HPP
2  #define COMPLEX_PRINTER_HPP
3
4  #include <complex>
5  #include <concepts>
6  #include <string>
7
8  namespace std
9  {
10     /**
11      * @brief convertit un nombre complexe en chaîne de caractère
12      */
13     using namespace std::literals;
14     template <std::floating_point T>
15     std::string to_string(std::complex<T> value)
16     {
17         return std::string{std::to_string(std::real(value))
18                             + ((std::imag(value) >= 0) ? '+' : '\0')
19                             + std::to_string(std::imag(value)) + 'i'};
20     }
21
22     /**
23      * @brief Implémente l'opérateur de flux sortant
24      * sur les nombres complexes
25      *
26      * @tparam T le type de représentation floatante du complexe
27      * @param os Le flux de sorti
28      * @param value Le nombre complexe à transférer
29      * @return std::ostream& retourne le flux passé en paramètre
30      */
```

```
31     template <std::floating_point T>
32     std::ostream& operator<<(std::ostream& os, std::complex<T> value)
33     {
34         os << std::real(value) << ((std::imag(value) >= 0) ? '+' : '\0' ) << std::imag(value) << 'i';
35         return os;
36     }
37 }
38
39
```

# Constexpr.hpp

```
1  #ifndef CONSTEXPR_HPP
2  #define CONSTEXPR_HPP
3
4  /**
5   * @brief Utilisé pour utiliser ou non constexpr
6   */
7  #define CONSTEXPR constexpr
8
9
```



# ConsoleInterface.hpp

```
1  #ifndef CONSOLE_INTERFACE_HPP
2  #define CONSOLE_INTERFACE_HPP
3
4  #include <ostream>
5  #include <Board.hpp>
6
7  const char *getUnicodeChar(TypePiece piece, Color color);
8  TypeMove chr_to_TypeMove(char type_move);
9  char Piece_to_chr(TypePiece piece, Color color);
10 Piece chr_to_Piece(char piece);
11
12 template <std::size_t N, std::size_t M>
13 void print_board_light(std::ostream &os, Board<N, M> const &board);
14
15 template <std::size_t N, std::size_t M>
16 std::ostream& operator<<(std::ostream& os, Board<N, M> const& board);
17
18 #include "ConsoleInterface.hpp"
19
```

# ConsoleInterface.tpp

```
1  #include <iostream>
2  #include <Board.hpp>
3
4  const char *getUnicodeChar(TypePiece piece, Color color)
5  {
6      using enum TypePiece;
7      if (color == Color::WHITE)
8      {
9          #if defined(WIN32)
10             switch (piece)
11             {
12                 case KING:
13                     return "K";
14                 case QUEEN:
15                     return "Q";
16                 case ROOK:
17                     return "R";
18                 case BISHOP:
19                     return "B";
20                 case KNIGHT:
21                     return "N";
22                 case PAWN:
23                     return "P";
24                 case EMPTY:
25                     default:
26                     return " ";
27             }
28         }
29     else
30     {
```

```
31     switch (piece)
32     {
33     case KING:
34         return "k";
35     case QUEEN:
36         return "q";
37     case ROOK:
38         return "r";
39     case BISHOP:
40         return "b";
41     case KNIGHT:
42         return "n";
43     case PAWN:
44         return "p";
45     case EMPTY:
46     default:
47         return " ";
48     }
49 }
50 #else
51     switch (piece)
52     {
53     case KING:
54         return "K";
55     case QUEEN:
56         return "Q";
57     case ROOK:
58         return "R";
59     case BISHOP:
60         return "B";
```

```
61     case KNIGHT:
62         return "N";
63     case PAWN:
64         return "P";
65     case EMPTY:
66     default:
67         return " ";
68     }
69 }
70 else
71 {
72     switch (piece)
73     {
74     case KING:
75         return "k";
76     case QUEEN:
77         return "q";
78     case ROOK:
79         return "r";
80     case BISHOP:
81         return "b";
82     case KNIGHT:
83         return "n";
84     case PAWN:
85         return "p";
86     case EMPTY:
87     default:
88         return " ";
89     }
90 }
```

```

91 #endif
92 }
93
94 TypeMove chr_to_TypeMove(char type_move)
95 {
96     using enum TypeMove;
97     switch (type_move)
98     {
99     case 'N':
100         return NORMAL;
101     case 'S':
102         return SPLIT;
103     case 'M':
104         return MERGE;
105     case 'P':
106         return PROMOTE;
107     default:
108         return NORMAL;
109     }
110 }
111
112 char Piece_to_chr(TypePiece piece, Color color)
113 {
114     using enum TypePiece;
115     if (color == Color::WHITE)
116     {
117         switch (piece)
118         {
119         case KING:
120             return 'K';

```

```
121     case QUEEN:
122         return 'Q';
123     case ROOK:
124         return 'R';
125     case BISHOP:
126         return 'B';
127     case KNIGHT:
128         return 'N';
129     case PAWN:
130         return 'P';
131     case EMPTY:
132         default:
133         return ' ';
134     }
135 }
136 else
137 {
138     switch (piece)
139     {
140     case KING:
141         return 'k';
142     case QUEEN:
143         return 'q';
144     case ROOK:
145         return 'r';
146     case BISHOP:
147         return 'b';
148     case KNIGHT:
149         return 'n';
150     case PAWN:
```

```

151     return 'p';
152     case EMPTY:
153     default:
154         return ' ';
155     }
156 }
157 }
158
159 Piece chr_to_Piece(char piece)
160 {
161     using enum TypePiece;
162     using enum Color;
163
164     switch (piece)
165     {
166     case 'p':
167         return Piece(PAWN, BLACK);
168     case 'P':
169         return Piece(PAWN, WHITE);
170     case 'n':
171         return Piece(KNIGHT, BLACK);
172     case 'N':
173         return Piece(KNIGHT, WHITE);
174     case 'b':
175         return Piece(BISHOP, BLACK);
176     case 'B':
177         return Piece(BISHOP, WHITE);
178     case 'r':
179         return Piece(ROOK, BLACK);
180     case 'R':

```

```

181     return Piece(ROOK, WHITE);
182 case 'q':
183     return Piece(QUEEN, BLACK);
184 case 'Q':
185     return Piece(QUEEN, WHITE);
186 case 'k':
187     return Piece(KING, BLACK);
188 case 'K':
189     return Piece(KING, WHITE);
190 case ' ':
191     default:
192     return Piece(EMPTY, BLACK);
193 }
194 }
195
196 template <std::size_t N, std::size_t M>
197 void print_board_light(std::ostream &os, Board<N, M> const &board)
198 {
199     for (std::size_t i{0}; i < N; i++)
200     {
201         for (std::size_t j{0}; j < M; j++)
202         {
203             Piece p {board(i, j)};
204             os << Piece_to_chr(p.get_type(), p.get_color());
205         }
206         os << '\n';
207     }
208 }
209
210 template <std::size_t N, std::size_t M>

```



```

211 std::ostream &operator<<(std::ostream &os, Board<N, M> const &board)
212 {
213     for (std::size_t i{0}; i < M; i++)
214     {
215         os << "|-----";
216     }
217     os << "\n";
218     for (std::size_t i{0}; i < N; i++)
219     {
220         for (std::size_t j{0}; j < M; j++)
221         {
222             TypePiece piece{board(i, j).get_type()};
223             Color color{board(i, j).get_color()};
224             os << "| " << getUnicodeChar(piece, color) << " ";
225         }
226         os << "\n";
227         for (std::size_t j{0}; j < M; j++)
228         {
229             if (board(i, j).get_type() != TypePiece::EMPTY)
230             {
231                 double proba{board.get_proba(Coord(i, j))};
232                 int p{static_cast<int>(std::round(100. * proba))};
233                 if (p != 100)
234                 {
235                     os << "| ";
236                     if (p < 10)
237                     {
238                         os << '0';
239                     }
240                     os << p << "% ";

```

```
241     }
242     else
243     {
244         os << " |      ";
245     }
246 }
247 else
248 {
249     os << " |      ";
250 }
251 }
252 os << "|\n";
253 for (std::size_t j{0}; j < M; j++)
254 {
255     os << " |-----";
256 }
257 os << "|\n";
258 }
259 (void)board;
260 return os;
261
```

# ComputerPlayer.hpp

```
1  #ifndef COMPUTER_PLAYER_HPP
2  #define COMPUTER_PLAYER_HPP
3
4  #include <cstddef>
5  #include <Board.hpp>
6  #include <Constexpr.hpp>
7
8  namespace computer
9  {
10     template <std::size_t N, std::size_t M>
11     CONSTEXPR Move
12     get_best_move(
13         Board<N, M> const &board,
14         int profondeur);
15 }
16
17 #include "ComputerPlayer.cpp"
18
```

# ComputerPlayer.hpp

```
1  #include <Constexpr.hpp>
2  #include <cstdint>
3  #include <forward_list>
4  #include <functional>
5  #include <limits>
6  #include <tuple>
7  #include <algorithm>
8  #include <thread>
9  #include <concepts>
10 #include <unordered_map>
11 #include <Board.hpp>
12 #include <TypePiece.hpp>
13 #include <Color.hpp>
14 #include <Coord.hpp>
15 #include <Move.hpp>
16 #include <mutex>
17 #include <Random.hpp>
18
19 #define WIN_VALUE 15.
20
21 namespace computer
22 {
23     namespace __utility
24     {
25         /**
26          * @brief Renvoie la valeur d'une pièce
27          *
28          * @param piece Le type de la pièce
29          * @return La valeur de la pièce
30          */
```

```

31 CONSTEXPR int value_piece(TypePiece piece)
32 {
33     switch (piece)
34     {
35     case TypePiece::PAWN:
36         return 1;
37     case TypePiece::KNIGHT:
38         return 3;
39     case TypePiece::BISHOP:
40         return 3;
41     case TypePiece::ROOK:
42         return 5;
43     case TypePiece::QUEEN:
44         return 9;
45     case TypePiece::KING:
46         return 5;
47     case TypePiece::EMPTY:
48     default:
49         return 0;
50     }
51 }
52
53 /**
54  * @brief Renvoi le signe d'un joueur
55  *
56  * @param color La couleur du joueur
57  * @return 1 si c'est le joueur blanc
58  * @return -1 si c'est le joueur noir
59  */
60 CONSTEXPR int sign_color(Color color)

```

```

61 {
62     return (color == Color::WHITE) ? 1 : -1;
63 }
64
65 /**
66  * @brief Teste si un plateau est gagnant et indique le gagnant
67  *
68  * @tparam N Le nombre de ligne du plateau
69  * @tparam M Le nombre de colonne du plateau
70  * @param[in] board Le plateau à tester
71  * @param[out] winner_color prend la couleur du joueur gagnant.
72  * Contenu non garanti si il n'y a pas de gagnant
73  * @return true si il y a un gagnant
74  * @return false sinon
75  */
76 template <std::size_t N, std::size_t M>
77 constexpr bool winner(Board<N, M> const &board, Color &winner_color)
78 {
79     bool found_W_king{false}, found_B_king{false};
80     for (std::size_t i{0}; i < board.numberLines(); i++)
81     {
82         for (std::size_t j{0}; j < board.numberColumns(); j++)
83         {
84             auto piece = board(i, j);
85             if (piece.get_type() == TypePiece::KING)
86             {
87                 if (piece.get_color() == Color::WHITE)
88                 {
89                     found_W_king = true;
90                 }

```

```

91         else
92         {
93             found_B_king = true;
94         }
95         if (found_W_king == found_B_king)
96         {
97             return false;
98         }
99     }
100 }
101 }
102 if (found_B_king)
103 {
104     winner_color = Color::BLACK;
105 }
106 else
107 {
108     winner_color = Color::WHITE;
109 }
110 return true;
111 }
112
113 /**
114  * @brief Renvoi la fonction permettant de calculer le meilleur
115  * score du joueur
116  *
117  * @param[in] board Le plateau de jeu
118  * @param[in] x Le premier score
119  * @param[in] y Le second score
120  * @return true si le second score est plus

```

```

121     * interessant pour le joueur
122     * @return false sinon
123     */
124     CONSTEXPR bool
125     get_player_calc_best_score(Color color,
126                               double x,
127                               double y)
128     {
129         if (double_equal(x, y))
130         {
131             return static_cast<bool>(rnd::randint(0, 1));
132         }
133         if (color == Color::WHITE)
134         {
135             return (y > x) ? true : false;
136         }
137         return (y < x) ? true : false;
138     }
139
140     template <std::size_t N, std::size_t M>
141     bool check_alpha_beta(
142         Board<N, M> const &board,
143         std::forward_list<double> const &best_all_round_score,
144         double &best_score,
145         double score)
146     {
147         if (get_player_calc_best_score(
148             board.get_current_player(),
149             best_score, score))
150         {

```



```

151     best_score = score;
152     Color current{board.get_current_player()};
153     for (double const alpha : best_all_round_score)
154     {
155         if (get_player_calc_best_score(
156             current,
157             alpha,
158             best_score))
159         {
160             return true;
161         }
162         if (current == Color::WHITE)
163         {
164             current = Color::BLACK;
165         }
166         else
167         {
168             current = Color::WHITE;
169         }
170     }
171 }
172 return false;
173 }
174
175 template <std::size_t N, std::size_t M>
176 constexpr double evalCase(std::size_t i, std::size_t j) noexcept
177 {
178     if constexpr (N % 2 == 0)
179     {
180         if constexpr (M % 2 == 0)

```

```

181     {
182         return 2 * std::min(std::min(static_cast<double>(i), static_cast<double>(N - i - 1)) / (N - 2),
183                             std::min(static_cast<double>(j), static_cast<double>(M - j - 1)) / (M - 2))
184     }
185     else
186     {
187         return 2 * std::min(std::min(static_cast<double>(i), static_cast<double>(N - i - 1)) / (N - 2),
188                             std::min(static_cast<double>(j), static_cast<double>(M - j - 1)) / (M));
189     }
190 }
191 else
192 {
193     if constexpr (M % 2 == 0)
194     {
195         return 2 * std::min(std::min(static_cast<double>(i), static_cast<double>(N - i - 1)) / (N - 2),
196                             std::min(static_cast<double>(j), static_cast<double>(M - j - 1)) / (M - 2))
197     }
198     else
199     {
200         return 2 * std::min(std::min(static_cast<double>(i), static_cast<double>(N - i - 1)) / (N - 2),
201                             std::min(static_cast<double>(j), static_cast<double>(M - j - 1)) / (M));
202     }
203 }
204 }
205
206 /**
207  * @brief Renvoie une évaluation du plateau
208  * @details L'évaluation du plateau est calculé
209  *          en sommant toutes les pièces d'un joueur
210  *          * coefficienté par leurs probabilités et en soustrayant

```

```

211 * avec ce meme calcul les piéces de l'autre joueur
212 * @tparam N Le nombre de ligne du plateau
213 * @tparam M Le nombre de colonne du plateau
214 * @param board Le plateau de jeu
215 * @return double L'évaluation du plateau
216 */
217 template <std::size_t N, std::size_t M>
218 constexpr double heuristic(Board<N, M> const &board) noexcept
219 {
220     Color win;
221     if (__utility::winner(board, win))
222     {
223         return __utility::sign_color(win) * WIN_VALUE;
224     }
225     double h{0};
226     for (std::size_t i{0}; i < board.numberLines(); i++)
227     {
228         for (std::size_t j{0}; j < board.numberColumns(); j++)
229         {
230             auto p = board(i, j);
231             if (p.get_type() != TypePiece::EMPTY)
232             {
233                 h += sign_color(p.get_color()) *
234                     (__utility::value_piece(p.get_type()) +
235                      evalCase<N, M>(i, j)) *
236                     board.get_proba(Coord(i, j));
237             }
238         }
239     }
240     return h;

```

```

241     }
242
243     template <std::size_t N, std::size_t M>
244     CONSTEXPR double rec_get_best_move(
245         Board<N, M> const &board,
246         std::forward_list<double> &best_score_alpha_beta,
247         int profondeur);
248
249     template <std::size_t N, std::size_t M>
250     CONSTEXPR double deterministic_eval_move(
251         Board<N, M> const &board,
252         std::forward_list<double> &best_score_alpha_beta,
253         Move const &move,
254         int profondeur)
255     {
256         double proba_move{board.get_proba_move(move)};
257         Board<N, M> board_cpy1{board};
258         if (double_equal(proba_move, 1.))
259         {
260             board_cpy1.move(move, true);
261             board_cpy1.change_player();
262             return rec_get_best_move(board_cpy1, best_score_alpha_beta, profondeur - 1);
263         }
264         else if (double_equal(proba_move, 0.))
265         {
266             board_cpy1.move(move, false);
267             board_cpy1.change_player();
268             return rec_get_best_move(board_cpy1, best_score_alpha_beta, profondeur - 1);
269         }
270         else

```

```

271 {
272     Board<N, M> board_cpy2{board};
273     board_cpy1.move(move, true);
274     board_cpy2.move(move, false);
275     board_cpy1.change_player();
276     board_cpy2.change_player();
277     double eval1{rec_get_best_move(board_cpy1, best_score_alpha_beta, profondeur - 1)};
278     double eval2{rec_get_best_move(board_cpy2, best_score_alpha_beta, profondeur - 1)};
279
280     return proba_move * eval1 + (1. - proba_move) * eval2;
281 }
282 }
283
284 template <std::size_t N, std::size_t M>
285 CONSTEXPR double rec_get_best_move(
286     Board<N, M> const &board,
287     std::forward_list<double> &best_score_alpha_beta,
288     int profondeur)
289 {
290     Color win;
291     if (profondeur <= 0)
292     {
293         return heuristic(board);
294     }
295     else if (winner(board, win))
296     {
297         return __utility::sign_color(win) * WIN_VALUE;
298     }
299     else
300     {

```

```

301     double best_score{-1 * sign_color(board.get_current_player()) *
302                     std::numeric_limits<double>::max()};
303
304     board.all_move(
305         [&board,
306          &best_score_alpha_beta,
307          profondeur,
308          &best_score](Move const &m) mutable -> bool
309     {
310         best_score_alpha_beta
311             .push_front(best_score);
312         double score;
313         try
314         {
315             score = deterministic_eval_move(
316                 board,
317                 best_score_alpha_beta,
318                 m,
319                 profondeur);
320         }
321         catch (std::runtime_error const &e)
322         {
323             return false;
324         }
325         best_score_alpha_beta.pop_front();
326         return check_alpha_beta(
327             board,
328             best_score_alpha_beta,
329             best_score,
330             score);

```

```

331         });
332         return best_score;
333     }
334 }
335
336 template <std::size_t N, std::size_t M>
337 struct Param_get_best_move
338 {
339     Board<N, M> board;
340     std::forward_list<Move> const &move;
341     double &best_eval;
342     Move &best_move;
343     int profondeur;
344 };
345
346 template <std::size_t N, std::size_t M>
347 constexpr void
348 th_get_best_move(Param_get_best_move<N, M> &&param)
349 {
350     param.best_eval = -1 * sign_color(param.board.get_current_player()) *
351         std::numeric_limits<double>::max();
352     for (Move const &move : param.move)
353     {
354         double score;
355         Board<N, M> board_cpy = param.board;
356         if (move.type == TypeMove::PROMOTE)
357         {
358             board_cpy.move_promotion(move);
359         }
360         else

```

```

361     {
362         board_cpy.move(move);
363     }
364     board_cpy.change_player();
365     std::forward_list<double> list_best_score{param.best_eval};
366     score = rec_get_best_move(board_cpy,
367                               list_best_score,
368                               param.profondeur - 1);
369
370     if (get_player_calc_best_score(
371         param.board.get_current_player(),
372         param.best_eval, score))
373     {
374         param.best_eval = score;
375         param.best_move = move;
376     }
377 }
378 }
379 }
380
381 template <std::size_t N, std::size_t M>
382 constexpr Move get_best_move(
383     Board<N, M> const &board,
384     int profondeur)
385 {
386     unsigned int number_thread{std::thread::hardware_concurrency()};
387     std::vector<std::forward_list<Move>> list_move(number_thread);
388     std::size_t last_th_view{0};
389
390     board.all_move(

```



```

391     [&list_move,
392      &last_th_view,
393      number_thread](Move const &m) mutable -> bool
394     {
395         list_move[last_th_view].push_front(std::move(m));
396         last_th_view = (last_th_view + 1) % number_thread;
397         return false;
398     });
399
400     std::vector<std::thread> threads(number_thread);
401
402     std::vector<double> best_eval(number_thread);
403
404     std::vector<Move> best_move(number_thread);
405
406     for (std::size_t i{0}; i < number_thread; i++)
407     {
408
409         __utility::Param_get_best_move param{board,
410                                             std::move(list_move[i]),
411                                             best_eval[i],
412                                             best_move[i],
413                                             profondeur};
414
415         // param.board = board;
416         threads[i] = std::thread(__utility::th_get_best_move<N, M>, param);
417     }
418     double better{-1 * __utility::sign_color(board.get_current_player()) *
419                  std::numeric_limits<double>::max()};
420     Move better_move;
421     for (std::size_t i{0}; i < number_thread; i++)

```

```
421     {
422         threads[i].join();
423         if (__utility::get_player_calc_best_score(
424             board.get_current_player(),
425             better,
426             best_eval[i]))
427         {
428             better = best_eval[i];
429             better_move = best_move[i];
430         }
431     }
432     return better_move;
433 }
434
```

# Matrix.hpp

```
1  #ifndef MATRIX_HPP
2  #define MATRIX_HPP
3
4  #include <array>
5  #include <initializer_list>
6  #include <concepts>
7  #include <type_traits>
8  #include <ostream>
9
10 template <typename T>
11 concept arithmetic = std::is_arithmetic_v<T>;
12
13 template <typename T>
14 concept MathObj = requires(T lhs, T rhs) {
15     lhs + rhs == rhs + lhs;
16     lhs - rhs;
17     lhs * rhs;
18 };
19
20 template <MathObj T, std::size_t LINES, std::size_t COLUMNS = LINES>
21 class Matrix final
22 {
23     static_assert(LINES > 0 || COLUMNS > 0,
24         "The matrix cannot have a zero size");
25
26 public:
27     Matrix() = default;
28     constexpr Matrix(T const initialValue);
29     constexpr Matrix(std::initializer_list<std::initializer_list<T>> const &matrix);
30
```

```

31 using SubBloc = Matrix<T, LINES / 2, COLUMNS / 2>;
32 constexpr Matrix(SubBloc const &a, SubBloc const &b, SubBloc const &c, SubBloc const &d);
33
34 constexpr ~Matrix() noexcept = default;
35
36 Matrix(Matrix const &matrix) = default;
37 Matrix &operator=(Matrix const &matrix) = default;
38
39 Matrix(Matrix &&matrix) noexcept = default;
40 Matrix &operator=(Matrix &&matrix) = default;
41
42 constexpr std::size_t numberLines() const noexcept;
43 constexpr std::size_t numberColumns() const noexcept;
44
45 constexpr T const &operator()(std::size_t const lines,
46                               std::size_t const columns) const noexcept;
47 constexpr T &operator()(std::size_t const lines, std::size_t const columns);
48
49 constexpr Matrix<T, LINES, COLUMNS>
50 operator+=(Matrix<T, LINES, COLUMNS> const &matrix) noexcept;
51
52 template <typename U, std::size_t N, std::size_t M>
53 constexpr friend Matrix<U, N, M>
54 operator+(Matrix<U, N, M> matrix_left, Matrix<U, N, M> const &matrix_right) noexcept;
55
56 constexpr Matrix<T, LINES, COLUMNS>
57 operator-=(Matrix<T, LINES, COLUMNS> const &matrix) noexcept;
58
59 template <MathObj U, std::size_t N, std::size_t M>
60 constexpr friend Matrix<U, N, M>

```

```

61 operator-(Matrix<U, N, M> matrix_left, Matrix<U, N, M> const &matrix_right) noexcept;
62
63 constexpr Matrix<T, LINES, COLUMNS> &operator*=(int const multiplier) noexcept;
64
65 template <MathObj U, std::size_t N, std::size_t M, arithmetic V>
66 constexpr friend Matrix<U, N, M> operator*(Matrix<U, N, M> matrix,
67      V const multiplier) noexcept;
68
69 template <MathObj U, std::size_t N, std::size_t M, arithmetic V>
70 constexpr friend Matrix<U, N, M> operator*(V const multiplier,
71      Matrix<U, N, M> matrix) noexcept;
72
73 template <MathObj U, std::size_t LINES_M1, std::size_t SHARED,
74      std::size_t COLUMNS_M2>
75 constexpr friend Matrix<U, LINES_M1, COLUMNS_M2>
76 operator*(Matrix<U, LINES_M1, SHARED> const &matrix_left,
77      Matrix<U, SHARED, LINES_M1> const &matrix_right) noexcept;
78
79 template <MathObj U, std::size_t N, std::size_t M>
80 friend std::ostream &operator<<(std::ostream &out,
81      Matrix<U, N, M> const &matrix);
82
83 constexpr Matrix<T, COLUMNS, LINES> transposed() const noexcept;
84
85 constexpr Matrix<T, 1, COLUMNS>
86 line(std::size_t const indexLine) const noexcept;
87
88 constexpr Matrix<T, LINES, 1>
89 column(std::size_t const indexColumn) const noexcept;
90

```

```

91 constexpr static Matrix<T, LINES, COLUMNS> identity() noexcept;
92
93 template <std::size_t N, std::size_t M>
94 constexpr Matrix<T, LINES*N, COLUMNS*M> tensoriel_product(Matrix<T, N, M> const& rhs) const noexcept;
95
96 private:
97     constexpr std::size_t offset(std::size_t const lines,
98                                   std::size_t const columns) const noexcept;
99
100    constexpr std::array<std::size_t, COLUMNS> strSizeMax() const;
101    constexpr std::size_t strSize(T const value) const;
102
103    constexpr std::array<T, COLUMNS * LINES>
104    array_matrix_with_initializer_list(std::initializer_list<std::initializer_list<T>> const &matrix);
105
106    std::array<T, LINES * COLUMNS> m_matrix{0};
107 };
108
109 #include "Matrix.tpp"
110
111

```

# Matrix.hpp

```
1  #include <cassert>
2  #include <algorithm>
3  #include <string>
4  #include <ostream>
5  #include <iomanip>
6  #include <cmath>
7  #include "Matrix.hpp"
8
9  template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
10 constexpr Matrix<T, LINES, COLUMNS>::Matrix(T const initialValue)
11 {
12     std::fill(std::begin(m_matrix), std::end(m_matrix), initialValue);
13 }
14
15 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
16 constexpr Matrix<T, LINES, COLUMNS>::Matrix(std::initializer_list<std::initializer_list<T>> const & matrix)
17 : m_matrix()
18 {
19     m_matrix = array_matrix_with_initializer_list(matrix);
20 }
21
22 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
23 constexpr Matrix<T, LINES, COLUMNS>::Matrix(SubBloc const& a, SubBloc const& b, SubBloc const& c, SubBloc
24 : m_matrix()
25 {
26     static_assert( (LINES % 2 == 0 || COLUMNS % 2 == 0)
27         && "Le constructeur par bloc n'est valable que pour des matrices de taille pair" );
28     constexpr std::size_t N { LINES/2 };
29     constexpr std::size_t M { COLUMNS/2 };
30 }
```

```

31  for (std::size_t i {0}; i < N; i++)
32  {
33      for (std::size_t j {0}; j < M; j++)
34      {
35          m_matrix[offset(i, j)] = a(i, j);
36          m_matrix[offset(i, j + M)] = b(i, j);
37          m_matrix[offset(i + N, j)] = c(i, j);
38          m_matrix[offset(i + N, j + M)] = d(i, j);
39      }
40  }
41 }
42
43 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
44 constexpr std::size_t
45 Matrix<T, LINES, COLUMNS>::numberLines() const noexcept
46 {
47     return LINES;
48 }
49
50 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
51 constexpr std::size_t
52 Matrix<T, LINES, COLUMNS>::numberColumns() const noexcept
53 {
54     return COLUMNS;
55 }
56
57 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
58 constexpr std::size_t
59 Matrix<T, LINES, COLUMNS>::offset(std::size_t const lines,
60                                     std::size_t const columns) const noexcept

```



```

61 {
62     assert(lines < numberLines() && "Column requested invalid.");
63     assert(columns < numberColumns() && "Line requested invalid.");
64     return lines * numberColumns() + columns;
65 }
66
67 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
68 constexpr T const &
69 Matrix<T, LINES, COLUMNS>::operator()(std::size_t const lines,
70                                         std::size_t const columns) const noexcept
71 {
72     return m_matrix[offset(lines, columns)];
73 }
74
75 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
76 constexpr T &
77 Matrix<T, LINES, COLUMNS>::operator()(std::size_t const lines,
78                                         std::size_t const columns)
79 {
80     return m_matrix[offset(lines, columns)];
81 }
82
83 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
84 constexpr Matrix<T, LINES, COLUMNS>
85 Matrix<T, LINES, COLUMNS>::operator+=(Matrix<T, LINES, COLUMNS> const &matrix) noexcept
86 {
87     for (std::size_t i{0}; i < numberLines(); i++)
88     {
89         for (std::size_t j{0}; j < numberColumns(); j++)
90         {

```

```

91     (*this)(i, j) += matrix(i, j);
92 }
93 }
94 return *this;
95 }
96
97 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
98 constexpr Matrix<T, LINES, COLUMNS>
99 operator+(Matrix<T, LINES, COLUMNS> matrix_left,
100           Matrix<T, LINES, COLUMNS> const &matrix_right) noexcept
101 {
102     return matrix_left += matrix_right;
103 }
104
105 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
106 constexpr Matrix<T, LINES, COLUMNS>
107 Matrix<T, LINES, COLUMNS>::operator-(Matrix<T, LINES, COLUMNS> const &matrix) noexcept
108 {
109     assert(matrix.numberLines() == numberLines() && "The subtraction of two matrixes, requires that they be
110             "same size");
111     assert(matrix.numberColumns() == numberColumns() && "The subtraction of two matrixes, requires that the
112             "same size");
113
114     for (std::size_t i{0}; i < numberLines(); i++)
115     {
116         for (std::size_t j{0}; j < numberColumns(); j++)
117         {
118             (*this)(i, j) -= matrix(i, j);
119         }
120     }

```

```

121     return *this;
122 }
123
124 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
125 constexpr Matrix<T, LINES, COLUMNS>
126 operator-(Matrix<T, LINES, COLUMNS> matrix_left,
127           Matrix<T, LINES, COLUMNS> const &matrix_right) noexcept
128 {
129     return matrix_left -= matrix_right;
130 }
131
132 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
133 constexpr Matrix<T, LINES, COLUMNS> &
134 Matrix<T, LINES, COLUMNS>::operator*=(int const multiplier) noexcept
135 {
136     for (std::size_t i{0}; i < numberLines(); i++)
137     {
138         for (std::size_t j{0}; j < numberColumns(); j++)
139         {
140             (*this)(i, j) *= multiplier;
141         }
142     }
143     return *this;
144 }
145
146 template <MathObj T, std::size_t LINES, std::size_t COLUMNS, arithmetic V>
147 constexpr Matrix<T, LINES, COLUMNS>
148 operator*(Matrix<T, LINES, COLUMNS> matrix, V const multiplier) noexcept
149 {
150     return matrix *= multiplier;

```

```

151 }
152
153 template <MathObj T, std::size_t LINES, std::size_t COLUMNS, arithmetic V>
154 constexpr Matrix<T, LINES, COLUMNS>
155 operator*(V const multiplier, Matrix<T, LINES, COLUMNS> matrix) noexcept
156 {
157     return matrix * multiplier;
158 }
159
160 template <MathObj T, std::size_t LINES_M1, std::size_t SHARED,
161         std::size_t COLUMNS_M2>
162 constexpr Matrix<T, LINES_M1, COLUMNS_M2>
163 operator*(Matrix<T, LINES_M1, SHARED> const &matrix_left,
164         Matrix<T, SHARED, COLUMNS_M2> const &matrix_right) noexcept
165 {
166
167     Matrix<T, LINES_M1, COLUMNS_M2> destination{};
168     for (std::size_t i{0}; i < matrix_left.numberLines(); i++)
169     {
170         for (std::size_t j{0}; j < matrix_right.numberColumns(); j++)
171         {
172             T somme = 0;
173             for (std::size_t k{0}; k < matrix_left.numberColumns(); k++)
174             {
175                 somme += matrix_left(i, k) * matrix_right(k, j);
176             }
177             destination(i, j) = somme;
178         }
179     }
180     return destination;

```

```

181 }
182
183 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
184 constexpr std::size_t Matrix<T, LINES, COLUMNS>::strSize(T const value) const
185 {
186     std::size_t size{1};
187     if constexpr (std::is_integral<T>())
188     {
189         if (value != 0)
190         {
191             size = static_cast<std::size_t>(std::log10(value)) + 1;
192         }
193     }
194     else
195     {
196         std::string str{std::to_string(value)};
197         size = std::size(str);
198     }
199     return size;
200 }
201
202 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
203 constexpr std::array<std::size_t, COLUMNS> Matrix<T, LINES, COLUMNS>::strSizeMax() const
204 {
205     std::array<std::size_t, COLUMNS> maxSize {};
206     for (std::size_t i {0}; i < LINES; i++)
207     {
208         for (std::size_t j {0}; j < COLUMNS; j++)
209         {
210             std::size_t size = strSize(m_matrix[offset(i, j)]);

```

```

211     if (size > maxSize[j])
212     {
213         maxSize[j] = size;
214     }
215 }
216 }
217 return maxSize;
218 }
219
220 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
221 std::ostream &
222 operator<<(std::ostream &out, Matrix<T, LINES, COLUMNS> const &matrix)
223 {
224     out << std::setfill(' ');
225     if constexpr (LINES == 1)
226     {
227         out << "(" ;
228         std::for_each(std::begin(matrix.m_matrix), std::end(matrix.m_matrix),
229             [&out](T const& v) -> void
230             {
231                 out << v << ' ';
232             });
233         out << ')' << std::endl;
234         return out;
235     }
236     else
237     {
238         out << "/";
239     }
240     std::array<std::size_t, COLUMNS> maxSize {matrix.strSizeMax()};

```

```

241 for (std::size_t i{0}; i < matrix.numberLines(); i++)
242 {
243     if (i == matrix.numberLines() - 1)
244     {
245         out << '\\';
246     }
247     else if (i > 0)
248     {
249         out << '|';
250     }
251     for (std::size_t j{0}; j < matrix.numberColumns(); j++)
252     {
253         std::size_t const size = maxSize[j] - matrix.strSize(matrix(i, j));
254         if (size > 0)
255         {
256             out << std::setw(static_cast<int>(size)+1);
257         }
258         out << ' ' << matrix(i, j);
259     }
260     if (i == 0)
261     {
262         out << " \\\" << std::endl;
263     }
264     else if (i < matrix.numberLines() - 1)
265     {
266         out << " |\" << std::endl;
267     }
268 }
269 if constexpr (LINES == 1)
270 {

```

```

271     out << ")" << std::endl;
272 }
273 else
274 {
275     out << " / " << std::endl;
276 }
277 return out;
278 }
279
280 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
281 constexpr Matrix<T, COLUMNS, LINES>
282 Matrix<T, LINES, COLUMNS>::transposed() const noexcept
283 {
284     Matrix<T, COLUMNS, LINES> trans{};
285     for (std::size_t i{0}; i < numberLines(); i++)
286     {
287         for (std::size_t j{0}; j < numberColumns(); j++)
288         {
289             trans(j, i) = (*this)(i, j);
290         }
291     }
292     return trans;
293 }
294
295 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
296 constexpr Matrix<T, 1, COLUMNS>
297 Matrix<T, LINES, COLUMNS>::line(std::size_t const indexLine) const noexcept
298 {
299     assert(indexLine < numberLines() && "index outside the matrix");
300     Matrix<T, 1, COLUMNS> copyLine{};

```



```

301     for (std::size_t i{0}; i < numberColumns(); i++)
302     {
303         cpyLine(0, i) = (*this)(indexLine, i);
304     }
305     return cpyLine;
306 }
307
308 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
309 constexpr Matrix<T, LINES, 1>
310 Matrix<T, LINES, COLUMNS>::column(
311     std::size_t const indexColumn) const noexcept
312 {
313     assert(indexColumn < numberColumns() && "index outside the matrix");
314     Matrix<T, LINES, 1> cpyLine{};
315     for (std::size_t i{0}; i < numberLines(); i++)
316     {
317         cpyLine(i, 0) = (*this)(i, indexColumn);
318     }
319     return cpyLine;
320 }
321
322 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
323 constexpr std::array<T, COLUMNS*LINES>
324 Matrix<T, LINES, COLUMNS>::array_matrix_with_initializer_list(
325     std::initializer_list<std::initializer_list<T>> const & matrix
326 )
327 {
328     std::array<T, COLUMNS*LINES> array_matrix = {};
329     auto it_mat_dst { std::begin(array_matrix) };
330     for (std::initializer_list<T> const & e : matrix)

```

```

331 {
332     assert(std::size(e) <= COLUMNS && "Value entry out of matrix");
333     std::move(std::begin(e), std::end(e), it_mat_dst);
334     it_mat_dst += COLUMNS;
335 }
336 return array_matrix;
337 }
338
339 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
340 constexpr Matrix<T, LINES, COLUMNS> Matrix<T, LINES, COLUMNS>::identity() noexcept
341 {
342     static_assert(LINES == COLUMNS && "not identity matrix in non square matrix");
343     Matrix<T, LINES, COLUMNS> Id {};
344     for (std::size_t i { 0 }; i < LINES; i++)
345     {
346         Id(i, i) = static_cast<T>(1);
347     }
348     return Id;
349 }
350
351 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
352 template <std::size_t N, std::size_t M>
353 constexpr Matrix<T, LINES*N, COLUMNS*M> Matrix<T, LINES, COLUMNS>::tensoriel_product(Matrix<T, N, M> const& mat2) const
354 {
355     Matrix<T, LINES*N, COLUMNS*M> res {};
356     for (std::size_t i {0}; i < LINES; i++)
357     {
358         for (std::size_t j {0}; j < COLUMNS; j++)
359         {
360             for (std::size_t k {0}; k < N; k++)

```

```
361     {
362         for (std::size_t l {0}; l < M; l++)
363         {
364             res(k + i*N, l + j * M) = m_matrix[offset(i, j)] * rhs(k, l);
365         }
366     }
367 }
368 }
369 return res;
370
```

# test-move-promotion.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_promotion(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_ROOK, Piece(), Piece()},
12         {B_PAWN, Piece(), Piece()},
13         {Piece(), Piece(), Piece()}};
14     Move m = Move_promote(Coord(1, 0), Coord(2, 0), TypePiece::BISHOP);
15     board.move_promotion(m);
16     return !(board(2, 0).get_type() == TypePiece::BISHOP &&
17             double_equal(board.get_proba(Coord(1, 0)), 0.) &&
18             board(1, 0).get_type() == TypePiece::EMPTY);
19 }
```

# test-move-epassant-with-capture.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_epassant_with_capture(int argc, char **argv)
9  {
10     Board<5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {B_PAWN, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece(), Piece()},
14         {Piece(), W_PAWN, Piece(), Piece(), Piece()},
15         {Piece(), Piece(), Piece(), W_BISHOP, Piece()}};
16     Move m = Move_split(Coord(4, 3), Coord(3, 2), Coord(2, 1));
17     Move m1 = Move_classic(Coord(3, 1), Coord(1, 1));
18     Move m2 = Move_classic(Coord(1, 0), Coord(2, 1));
19     board.move(m);
20     board.move(m1);
21     board.move(m2);
22     bool equal = double_equal(board.get_proba(Coord(3, 2)), 0.5);
23     return !(board(2, 1).get_type() == TypePiece::PAWN &&
24             equal &&
25             board(1, 1).get_type() == TypePiece::EMPTY &&
26             board(1, 0).get_type() == TypePiece::EMPTY);
27 }
```

# test-move-pawn-two-step.cpp

```
1  #include <math_utility.hpp>
2  #include <Board.hpp>
3  #include <Piece.hpp>
4  #include <Coord.hpp>
5  #include <TypePiece.hpp>
6  #include <Move.hpp>
7
8  int test_move_pawn_two_step(int argc, char **argv)
9  {
10     Board<5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {B_PAWN, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece(), Piece()},
14         {Piece(), Piece(), Piece(), Piece(), Piece()},
15         {Piece(), Piece(), Piece(), Piece(), Piece()}};
16     Move m = Move_classic(Coord(1, 0), Coord(3, 0));
17     board.move(m);
18     return !(board(3, 0).get_type() == TypePiece::PAWN &&
19             double_equal(board.get_proba(Coord(1, 0)), 0.) &&
20             board(1, 0).get_type() == TypePiece::EMPTY);
21 }
```

# test-move-promotion-with-capture.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_promotion_with_capture(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_ROOK, Piece(), Piece()},
12         {B_PAWN, Piece(), Piece()},
13         {Piece(), W_QUEEN, Piece()}};
14     Move m = Move_promote(Coord(1, 0), Coord(2, 1), TypePiece::BISHOP);
15     board.move(m);
16     return !(board(2, 1).get_type() == TypePiece::BISHOP &&
17             board(2, 1).get_color() == Color::BLACK &&
18             double_equal(board.get_proba(Coord(1, 0)), 0.) &&
19             board(1, 0).get_type() == TypePiece::EMPTY);
20 }
```

# test-move-promotion-with-split-before.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_promotion_with_split_before(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_ROOK, Piece(), Piece()},
12         {B_PAWN, Piece(), Piece()},
13         {Piece(), W_QUEEN, Piece()}};
14     Move m1 = Move_split(Coord(2, 1), Coord(2, 0), Coord(2, 2));
15     Move m2 = Move_promote(Coord(1, 0), Coord(2, 0), TypePiece::BISHOP);
16     board.move(m1);
17     board.move(m2);
18     bool res = !((board(2, 0).get_type() == TypePiece::BISHOP &&
19         board(2, 0).get_color() == Color::BLACK &&
20         board(1, 0).get_type() == TypePiece::EMPTY &&
21         double_equal(board.get_proba(Coord(2, 2)), 1.)) ||
22         (board(2, 0).get_type() == TypePiece::QUEEN &&
23         board(2, 0).get_color() == Color::WHITE &&
24         board(1, 0).get_type() == TypePiece::PAWN &&
25         double_equal(board.get_proba(Coord(2, 2)), 0.)));
26     return res;
27 }
```



# test-move-split-with-mesure.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_split_with_mesure(int argc, char **argv)
9  {
10     Board<5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {Piece(), Piece(), Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece(), Piece()},
14         {Piece(), Piece(), Piece(), Piece(), Piece()},
15         {Piece(), Piece(), Piece(), B_ROOK, Piece()}};
16     Move m = Move_split(Coord(4, 3), Coord(4, 1), Coord(4, 4));
17     Move m1 = Move_split(Coord(0, 0), Coord(0, 1), Coord(0, 4));
18     Move m2 = Move_classic(Coord(0, 1), Coord(4, 1));
19     Move m3 = Move_classic(Coord(4, 4), Coord(0, 4));
20     board.move(m);
21     board.move(m1);
22     board.move(m2);
23     board.move(m3);
24     std::size_t acc{0};
25     for (std::size_t i{0}; i < 5; i++)
26     {
27         for (std::size_t j{0}; j < 5; j++)
28         {
29             if (board(i, j).get_type() == TypePiece::ROOK)
30             {
```

```
31         acc++;
32     }
33 }
34 }
35 return acc != 2 && acc != 1;
36
```

# test-split-in-non-empty-case.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Move.hpp>
4  #include <Coord.hpp>
5  #include <math_utility.hpp>
6
7  int test_split_in_non_empty_case(int argc, char **argv)
8  {
9      Board<1, 4> board{
10         {W_ROOK, Piece(), Piece(), Piece()}};
11
12     Move m1{Move_split(Coord(0, 0), Coord(0, 1), Coord(0, 3))};
13     Move m2{Move_split(Coord(0, 1), Coord(0, 2), Coord(0, 3))};
14
15     board.move(m1);
16     board.move(m2);
17
18     bool ret{
19         board(0, 1).get_type() == TypePiece::ROOK &&
20         board(0, 2).get_type() == TypePiece::ROOK &&
21         board(0, 3).get_type() == TypePiece::ROOK &&
22         double_equal(board.get_proba(Coord(0, 1)), 0.5) &&
23         double_equal(board.get_proba(Coord(0, 2)), 0.25) &&
24         double_equal(board.get_proba(Coord(0, 3)), 0.25)};
25     return !ret;
26 }
```

# test-slide-merge-move-through-a-split-piece.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_slide_merge_move_through_a_split_piece(int argc, char **argv)
9  {
10     Board<2, 5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {Piece(), Piece(), B_ROOK, Piece(), Piece()}};
13     Move m = Move_split(Coord(1, 2), Coord(0, 2), Coord(1, 4));
14     Move m1 = Move_split(Coord(0, 0), Coord(0, 1), Coord(0, 4));
15     Move m2 = Move_merge(Coord(0, 1), Coord(0, 4), Coord(0, 3));
16     board.move(m);
17     board.move(m1);
18     board.move(m2);
19     bool res{board(0, 2).get_type() == TypePiece::ROOK &&
20         board(0, 2).get_color() == Color::BLACK &&
21         board(0, 1).get_type() == TypePiece::ROOK &&
22         board(0, 1).get_color() == Color::WHITE &&
23         double_equal(board.get_proba(Coord(0, 1)), 0.5) &&
24         board(0, 3).get_type() == TypePiece::ROOK &&
25         board(0, 3).get_color() == Color::WHITE &&
26         double_equal(board.get_proba(Coord(0, 3)), 0.5) &&
27         board(0, 4).get_type() == TypePiece::EMPTY};
28     return !res;
29 }
```

# test-get-proba-move-slide-capture.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_slide_capture(int argc, char **argv)
9  {
10     Board<3, 5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
14     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
15     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
16     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
17     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
18     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
19     board.move(m1);
20     board.move(m2);
21     board.move(m3);
22     board.move(m4);
23     bool res{double_equal(board.get_proba_move(m5), 1. / 16)};
24     return !res;
25 }
```

# test-get-proba-move-slide-on-same-color.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_slide_on_same_color(int argc, char **argv)
9  {
10     Board<3, 5> board{
11         {Piece(), Piece(), Piece(), Piece(), Piece()},
12         {B_QUEEN, W_ROOK, W_ROOK, W_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
14     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
15     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
16     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
17     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
18     Move m = Move_split(Coord(1, 0), Coord(0, 0), Coord(2, 0));
19     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
20     board.move(m1);
21     board.move(m2);
22     board.move(m3);
23     board.move(m4);
24     board.move(m);
25     bool res{double_equal(board.get_proba_move(m5), 1. / 2)};
26     return !res;
27 }
```

# test-get-proba-move-slide-without-target.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_slide_without_target(int argc, char **argv)
9  {
10     Board<3, 5> board{
11         {Piece(), Piece(), Piece(), Piece(), Piece()}},
12         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()}},
13         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
14     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
15     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
16     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
17     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
18     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
19     board.move(m1);
20     board.move(m2);
21     board.move(m3);
22     board.move(m4);
23     bool res{double_equal(board.get_proba_move(m5), 1.)};
24     return !res;
25 }
```

# test-get-proba-move-jump-same-color.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_jump_same_color(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_KING, Piece(), Piece()},
12         {Piece(), Piece(), Piece()},
13         {Piece(), Piece(), W_KNIGHT}};
14     Move m1 = Move_split(Coord(2, 2), Coord(0, 1), Coord(1, 0));
15     Move m5 = Move_classic(Coord(0, 0), Coord(0, 1));
16     board.move(m1);
17     bool res{double_equal(board.get_proba_move(m5), 1. / 2)};
18     return !res;
19 }
```



# test-get-proba-move-jump-capture.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_jump_capture(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_KING, Piece(), Piece()},
12         {Piece(), Piece(), Piece()},
13         {Piece(), Piece(), B_KNIGHT}};
14     Move m1 = Move_split(Coord(2, 2), Coord(0, 1), Coord(1, 0));
15     Move m5 = Move_classic(Coord(0, 0), Coord(0, 1));
16     board.move(m1);
17     bool res{double_equal(board.get_proba_move(m5), 1.)};
18     return !res;
19 }
```

# test-capture-slide-move-true.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <Color.hpp>
7  #include <math_utility.hpp>
8
9  int test_capture_slide_move_true(int argc, char **argv)
10 {
11     Board<3, 5> board{
12         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()},
14         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
15     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
16     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
17     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
18     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
19     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
20     board.move(m1);
21     board.move(m2);
22     board.move(m3);
23     board.move(m4);
24     board.move(m5, true);
25     bool res{board(0, 0).get_type() == TypePiece::ROOK && board(0, 0).get_color() == Color::BLACK};
26     return !res;
27 }
```

# test-capture-slide-move-false.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <Color.hpp>
7  #include <math_utility.hpp>
8
9  int test_capture_slide_move_false(int argc, char **argv)
10 {
11     Board<3, 5> board{
12         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()},
14         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
15     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
16     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
17     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
18     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
19     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
20     board.move(m1);
21     board.move(m2);
22     board.move(m3);
23     board.move(m4);
24     board.move(m5, false);
25     bool res{board(0, 4).get_type() == TypePiece::ROOK && board(0, 4).get_color() == Color::BLACK};
26     return !res;
27 }
```

# test-move-merge-after-split.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_merge_after_split(int argc, char **argv)
9  {
10     Board<4> board{
11         {B_KING, Piece(), Piece(), Piece()},
12         {B_ROOK, Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece()},
14         {W_QUEEN, Piece(), Piece(), W_KING}};
15     Move m1 = Move_split(Coord(3, 0), Coord(3, 1), Coord(3, 2));
16     Move m2 = Move_merge(Coord(3, 1), Coord(3, 2), Coord(3, 0));
17     board.move(m1);
18     board.move(m2, true);
19     bool res{double_equal(board.get_proba(Coord(3, 0)), 1.)};
20     return !res;
21 }
```

# test-split-after-split.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_split_after_split(int argc, char **argv)
9  {
10     Board<4> board{
11         {Piece(), Piece(), Piece(), Piece()},
12         {Piece(), B_KING, B_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece()},
14         {W_QUEEN, Piece(), Piece(), W_KING}};
15     Move m1 = Move_split(Coord(3, 0), Coord(2, 1), Coord(3, 2));
16     Move m2 = Move_split(Coord(3, 2), Coord(2, 1), Coord(1, 0));
17     board.move(m1);
18     board.move(m2, true);
19     bool res{double_equal(board.get_proba(Coord(3, 3)), 1.) &&
20             double_equal(board.get_proba(Coord(2, 1)), 1. / 2) &&
21             double_equal(board.get_proba(Coord(1, 0)), 1. / 2)};
22     return !res;
23 }
```

# test-split-after-split2.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_split_after_split2(int argc, char **argv)
9  {
10     Board<4> board{
11         {Piece(), Piece(), Piece(), Piece()},
12         {Piece(), B_KING, B_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece()},
14         {W_QUEEN, Piece(), Piece(), W_KING}};
15     Move m1 = Move_split(Coord(3, 0), Coord(2, 1), Coord(3, 2));
16     Move m2 = Move_split(Coord(3, 2), Coord(1, 0), Coord(2, 1));
17     board.move(m1);
18     board.move(m2, true);
19     bool res{double_equal(board.get_proba(Coord(3, 3)), 1.) &&
20             double_equal(board.get_proba(Coord(2, 1)), 1. / 4) &&
21             double_equal(board.get_proba(Coord(1, 0)), 1. / 4) &&
22             double_equal(board.get_proba(Coord(3, 2)), 1. / 2)};
23     return !res;
24 }
```

# test-multiple-split.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_multiple_split(int argc, char **argv)
9  {
10     Board<3> board{
11         {Piece(), Piece(), B_BISHOP},
12         {Piece(), B_KING, Piece()},
13         {Piece(), Piece(), Piece()}};
14     Move m1 = Move_split(Coord(1, 1), Coord(2, 1), Coord(2, 2));
15     Move m2 = Move_split(Coord(2, 2), Coord(1, 1), Coord(1, 2));
16     Move m3 = Move_classic(Coord(1, 2), Coord(2, 2));
17     Move m4 = Move_split(Coord(1, 1), Coord(0, 0), Coord(0, 1));
18     board.move(m1);
19     board.move(m2);
20     board.move(m3);
21     board.move(m4);
22     bool res{double_equal(board.get_proba(Coord(0, 2)), 1.)};
23     return !res;
24 }
```

# test-split-takes.cpp

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_split_takes(int argc, char **argv)
9  {
10     Board<3> board{
11         {Piece(), Piece(), W_BISHOP},
12         {Piece(), B_KING, Piece()},
13         {Piece(), Piece(), Piece()}};
14     Move m1 = Move_split(Coord(1, 1), Coord(2, 1), Coord(2, 0));
15     Move m2 = Move_split(Coord(2, 1), Coord(1, 1), Coord(2, 2));
16     Move m3 = Move_classic(Coord(0, 2), Coord(1, 1));
17     Move m4 = Move_classic(Coord(1, 1), Coord(2, 0));
18     board.move(m1);
19     board.move(m2);
20     board.move(m3);
21     board.move(m4);
22     bool res{double_equal(board.get_proba(Coord(2, 0)), 1.)};
23     return !res;
24 }
```